

Hướng Dẫn Thực Hành

Phát triển Web Kinh Doanh Nâng Cao



Hướng dẫn:

- Bài tập thực hành được chia làm nhiều Module
- Mỗi Module được thiết kế cho thời lượng là 3 hoặc 5 tiết thực hành tại lớp với sự hướng dẫn của Giảng viên.
- Tùy theo số tiết phân bổ, mỗi tuần học có thể thực hiện nhiều Module.
- Sinh viên phải làm tất cả các bài tập trong các Module ở tuần tương ứng. Những sinh viên chưa hoàn tất phần bài tập tại lớp có trách nhiệm tự làm tiếp tục ở nhà.
- Các bài có dấu (*) là các bài tập nâng cao dành cho sinh viên khá giỏi.

Biên soạn và tổng hợp

TS. Trần Duy Thanh

Mục Lục

Module 1: AngularJS – Binding - Component.....	4
Bài 1 – Kiến trúc thành phần của Angular.....	4
Bài 2 – Cài đặt và lập trình Angular.....	4
Bài 3 - Binding	4
Bài 4 – Binding Property	5
Bài 5 – Binding Class	5
Bài 6 – Binding Style.....	6
Bài 7 – Binding Event.....	7
Bài 8 – Binding Two-Way	8
Bài 9 - Binding Two-Way, model for Quadratic Equation	8
Bài 10- Binding Two-Way, model for Lunar Year (*).....	9
Module 2: AngularJS(tt) – Service - Routing.....	10
Bài 11- Json Object Model - Product	10
Bài 12- Json Array Model - Product.....	10
Bài 13- Json Array Model – Product Event (*).....	11
Bài 14- Json Array Model – Product - Catalog	15
Bài 15- Json Array Model – Product– Http Service(*).....	16
Bài 16- Json Array Model – Product– Http Service Handle Error (*)	18
Bài 17- Json Object Model – Customer – Service	20
Bài 18- Json Array Model – Group Customers (*).....	20
Bài 19 – Cấu hình Routing.....	22
Bài 20 – Routing for Page Not Found.....	24
Module 3 – AngularJS (tt) - Form.....	25
Bài 21– Login Screen	25
Bài 22 – Form Đăng ký Khóa học.....	26
Bài 23 – Form Đăng ký Khóa học - Validation.....	26
Bài 24- Mathematics	27
Module 4 – AngularJS (tt) Triệu gọi External Restful API.....	28
Bài 25: Tương tác External Restful API- Ngân hàng Đông Á.....	28
Bài 26: Tương tác External Restful API- Product Service 1.....	35
Bài 27: Tương tác External Restful API- Product Service 2.....	40
Bài 28 – Coindesk API- Bitcoin Price Index in Real-time.....	43
Bài 29 – Get List of Public APIs	43
Bài 30 - Predict the gender of a person based on their name - API.....	44
Bài 31- Get US public data API.....	44
Bài 32- Random dog images API	44
Module 5 –Restful API với NodeJS và ExpressJS	45
Bài 33– Giải thích chi tiết mô hình hoạt động của MVC	45

Bài 34 – API và Restful API	45
Bài 35 – Tạo và thực thi Project Restful API	45
Bài 36 – Cài đặt nodemon cho Web Server Project	49
Bài 37 – Cài đặt morgan cho Web Server Project	51
Bài 38 – Tạo HTTP GET – List of Book	53
Bài 39 - Triệu gọi HTTP GET – List of Book	55
Bài 40 – Tạo HTTP GET – a Book	59
Bài 41– Triệu gọi HTTP GET – a Book	61
Bài 42 – Tạo HTTP POST – Create a Book	64
Bài 43 – Triệu gọi HTTP POST – Create a Book	67
Bài 44 – Tạo HTTP PUT – Update a Book	72
Bài 45 – Triệu gọi HTTP PUT – Update a Book	74
Bài 46– Tạo HTTP DELETE– Remove a Book	77
Bài 47 – Triệu gọi HTTP DELETE– Remove a Book	77
Bài 48 – Upload files to Server-1	80
Bài 49 – Upload files to Server-2	83
Bài 50 – Bài tập tổng hợp Restful API(*)	88
Module 6 –Restful API với MongoDB	89
Bài 51- Cài đặt và sử dụng MongoDB	89
Bài 52 - Cấu hình và Kết nối NodeJS với MongoDB	97
Bài 53 – Tạo và triệu gọi API - HTTP GET – List Fashion	100
Bài 54 – Tạo và triệu gọi API - HTTP GET – A Fashion	104
Bài 55 – Tạo và triệu gọi API - HTTP POST – Create a Fashion	106
Bài 56 – Tạo và triệu gọi API - HTTP PUT – Update a Fashion	110
Bài 57 – Tạo và triệu gọi API - HTTP DELETE – Remove a Fashion	112
Bài 58 – Bài tập tổng hợp(*)	113
Module 7 – Cookie và Session	115
Bài 59 – Trình bày ý nghĩa của Cookie và Session	115
Bài 60 – Lập trình Cookie	115
Bài 61 – Lập trình Cookie – Lưu thông tin đăng nhập	119
Bài 62 – Lập trình Session	120
Bài 63 – Lập trình Session – Bài tập tổng hợp Shopping Cart(*)	122

Module 1: AngularJS – Binding - Component

Nội dung kiến thức thực hành:

- + Cấu hình và cài đặt NodeJS, AngularJS
- + Thực hành các lệnh của AngularJS: Tạo và chạy dự án, tạo component, service
- + Tìm hiểu cấu trúc thành phần của một Project AngularJS
- + Lập trình Binding: Binding, Binding Property, Binding Style, Binding Event, Binding Two Way
- + Lập trình Type Script
- + Lập trình built-in directives: ngIf, ngSwitch, ngFor
- + Lập trình Component

Bài 1 – Kiến trúc thành phần của Angular

Yêu cầu:

Hãy trình bày kiến trúc thành phần của Angular, giải thích chức năng của từng thành phần (ví dụ App, Module, Component, Service...).

Bài 2 – Cài đặt và lập trình Angular

Yêu cầu:

Hãy trình bày cách thức cài đặt AngularJS

Trình bày các lệnh tạo dự án (đặt tên **my-app**), chạy dự án, tạo component, tạo service

Bài 3 - Binding

Yêu cầu:

Tạo một Component tên “MyComponent”. File Type Script (.ts) chứa một biến tên là “myVar” có giá trị “Hello Angular”, hãy viết các lệnh binding để hiển thị dữ liệu lên giao diện (html) bằng 2 cách:

- Truy xuất trực tiếp vào biến “myVar”

- Viết hàm getMyVar() trong (.ts) trả giá trị của “myVar”

File (html) truy suất và hiển thị giá trị của “myVar” trong (.ts) như sau:

- Hiển thị dữ liệu gốc của biến “myVar”
- Hiển thị toàn bộ chữ in hoa của “myVar”
- Hiển thị toàn bộ chữ in thường của “myVar”

Bài 4 – Binding Property

Yêu cầu:

Tạo một “BindingPropertyComponent”, Type Script (.ts) chứa các Property (Attribute/Variable) sau:

```
export class BindingPropertyComponent {  
  public name:string="Trần Duy Thanh"  
  public email:string="thanhtd@uel.edu.vn"  
  public nameid:string="nameid"  
  public emailid:string="emailid"  
  public isDisabled:boolean=true  
}
```

HTML sử dụng 2 cách: tạo cấu trúc HTML trong .ts và cấu trúc HTML trong file HTML để binding các thuộc tính này. Sử dụng cú pháp `[]` để binding cho name và `{{}}` để binding cho email. Thuộc tính isDisabled sẽ thiết lập cho Email, sinh viên thay đổi true hoặc false để quan sát kết quả.

Bài 5 – Binding Class

Yêu cầu:

Tạo một “BindingClassComponent”.

File “binding-class.component.css” định nghĩa các class sau:

```
.text-success{  
  color:darkgreen  
}  
.text-error{  
  color:darkred  
}  
.text-primary{  
  color:navy  
}
```

```
.text-normal{  
  font-style: italic;  
}
```

File “binding-class.component.ts” khai báo:

```
export class BindingClassComponent {  
  public success="text-success"  
  public error="text-error"  
  public primary="text-primary"  
  public normal="text-normal"  
  public multiClass={  
    "text-primary":true,  
    "text-normal":true,  
    "text-error":true  
  }  
}
```

Yêu cầu, file “binding-class.component.html” tạo 5 thẻ <h1> dùng cú pháp [] để binding các class, tạo 5 thẻ <h2> dùng cú pháp {{{}} để binding các class. Nội dung hiển thị trong mỗi <h1> và <h2> là tùy ý.

Sau đó định nghĩa một class “.text-complexity” định dạng chữ Đậm, In Nghiêng, Font chữ “Cambria”, cỡ chữ 18px, màu chữ Blue. Tiến hành định nghĩa tên biến trong .ts và binding trong html để sử dụng class “.text-complexity” này.

Bài 6 – Binding Style

Yêu cầu:

Tạo một “BindingStyleComponent” component.

Định nghĩa Type Script (.ts) như sau:

```
export class BindingStyleComponent {  
  public isError:boolean=false  
  public textStyle={  
    color:'darkorange',  
    fontSize:'26px'  
  }  
}
```

Trong html binding style theo cú pháp sau:

```
<h1 style="color:red;">Trần Duy Thanh</h1>  
<h1 [style.color]="isError? 'red':'darkgreen'">Trần Phạm Thanh Trà</h1>  
<h1 [style]="{'color':'purple','font-size':'26px'}">Trần Phạm Mẫn Nhi</h1>  
<h1 [style]="textStyle">Phạm Thị Xuân Diệu</h1>
```

Chạy và quan sát kết quả.

Mở rộng: Bổ sung style cho chữ tự động in Hoa toàn bộ. Gán binding và kiểm tra kết quả.

Bài 7 – Binding Event

Yêu cầu:

Tạo một “BindingEventComponent” component.

File Type Script “binding-event.component.ts” định nghĩa 2 sự kiện:

```
export class BindingEventComponent {  
  onClick(event:any){  
    alert(event.pointerId)  
  }  
  onSubmit(value:string){  
    alert(value)  
  }  
}
```

File “binding-event.component.html” định nghĩa và binding event:

```
<p>binding-event works!</p>  
<button (click)="onClick($event)">Click Me</button>  
Name:  
<input type="text" #myName>  
<button (click)="onSubmit(myName.value)">Submit</button>
```

Chạy component và quan sát kết quả.

Mở rộng: Chỉnh sửa giao diện cho người dùng nhập 2 số a, b bất kỳ và 5 button:

Button cộng: xử lý xuất kết quả tổng a+b

Button trừ: xử lý xuất kết quả hiệu a-b

Button nhân: xử lý xuất kết quả tích a*b

Button chia: xử lý xuất kết quả thương a/b, lưu ý xử lý mẫu số =0

Button xóa trắng: Xóa dữ liệu trên giao diện.

Chạy component để kiểm tra kết quả sau khi chỉnh sửa.

Bài 8 – Binding Two-Way

Yêu cầu:

Tạo một component “BindingTwoWayComponent”.

File “binding-two-way.component.ts” định nghĩa:

```
export class BindingTwoWayComponent {
  public name:string=''
  public address:string=''
  setDefaultAddress(){
    this.address='13 đường Hùng Vương'
  }
}
```

File “binding-two-way.component.html” định nghĩa:

```
<p>binding-two-way works!</p>
<p>Name: <input [(ngModel)]="name" type="text"></p>
<p>Your name:{{name}}</p>
<p>Address:<input [(ngModel)]="address" type="text"></p>
<p>Your address:{{address}}</p>
<button (click)="setDefaultAddress()">Set default</button>
```

Chạy component và quan sát kết quả.

Mở rộng bổ sung email, số điện thoại, tiến hành binding và chạy lại component, quan sát kết quả sau khi thay đổi.

Bài 9 - Binding Two-Way, model for Quadratic Equation

Yêu cầu:

Tạo một component có giao diện HTML như sau:

Giải phương trình bậc 2 online	
Hệ số a:	1
Hệ số b:	3
Hệ số c:	-4
Kết quả:	x1=-4 ; x2=1
Giải phương trình Xóa trắng	

- Dùng Binding Two-way để mapping giá trị cho các biến trong Type Script và Html.

- Tạo một class tên là “Quadratic” nhận vào 3 hệ số a, b, c. và có một phương thức “findSolution()” để xử lý giải phương trình bậc 2
- Khi nhấn nút “Giải phương trình” thì Type Script sẽ áp dụng cơ chế binding để nhận các giá trị nhập vào từ giao diện Html, sau đó dùng lớp “Quadratic” để giải phương trình, sau khi giải xong thì trả về kết quả lên giao diện html.
- Khi nhấn nút “Xóa trắng” thì xóa các dữ liệu trên các ô nhập liệu và dòng kết quả.

Bài 10- Binding Two-Way, model for Lunar Year (*)

Yêu cầu:

Tạo một component có giao diện như dưới đây:

Chuyển đổi Dương lịch thành Âm lịch	
15	Ngày 5 Tháng 1986 Năm Chuyển đổi
Âm Lịch	
Thứ trong tuần	Ngày thứ 5
Ngày tháng năm âm	7/4/1986
Năm	Bính Dần
Tháng	Quý Tỵ
Ngày	Kỷ Mùi

Người dùng lựa chọn Ngày tháng năm sinh trên giao diện (dùng select/dropdownlist). Bấm nút “Chuyển đổi”, chuyển ngày dương thành ngày âm.

Yêu cầu dùng cơ chế binding để nạp dữ liệu lên dropdownlist. Viết class “LunarYear” nhận vào 3 thông số (ngày, tháng, năm dương lịch) và hàm “findLunarYearDetail()” trả về thông tin chi tiết như trên giao diện.

Hướng dẫn:

Google search để biết cách chuyển 1 năm Dương lịch thành Âm lịch, cũng như cách binding lấy dữ liệu trên dropdownlist mà người dùng chọn.

Binding dữ liệu từ mảng lên dropdownlist. Ví dụ ta có:

```
days=["1","2","3"]
<select class="form-select" id="days" name="days">
  <option *ngFor="let day of days">{{day}}</option>
</select>
```

Module 2: AngularJS(tt) – Service - Routing

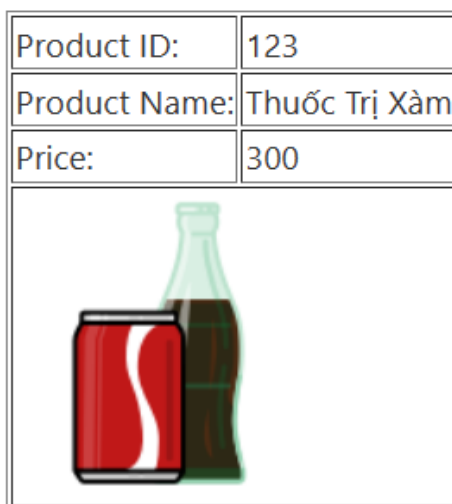
Nội dung kiến thức thực hành:

- + Lập trình built-in directives: ngIf, ngSwitch, ngFor
- + Lập trình Service
- + Lập trình Routing
- + Và các kỹ năng thực hành khác

Bài 11- Json Object Model - Product

Yêu cầu:

Tạo một Component để hiển thị sản phẩm như hình bên dưới. Yêu cầu dùng JsonObject và cơ chế binding. Dữ liệu khai báo trong Type Script.



Bài 12- Json Array Model - Product

Yêu cầu:

Tạo một Component để hiển thị danh sách sản phẩm như hình bên dưới. Yêu cầu dùng JsonArray và cơ chế binding.

Dữ liệu khai báo trong Type Script.

Ma San Pham	Ten San Pham	Gia San Pham	Picture
p1	Coca	100	
p2	Pepsi	300	
p3	Sting	200	

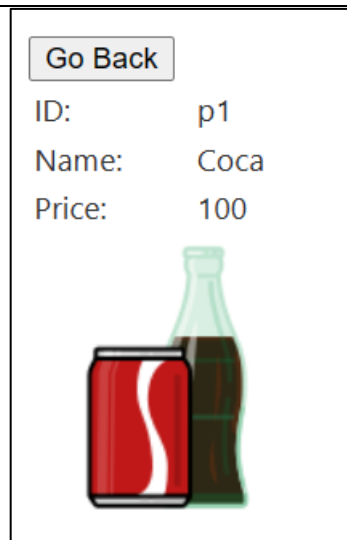
Bài 13- Json Array Model – Product Event (*)

Yêu cầu:

Tạo một Component để hiển thị danh sách sản phẩm như hình bên dưới. Yêu cầu dùng JsonArray và cơ chế binding. Dữ liệu khai báo trong Service.

Ma San Pham	Ten San Pham	Gia San Pham	Picture	#
p1	Coca	100		Details
p2	Pepsi	300		Details
p3	Sting	200		Details

Nhấn vào details sẽ hiển thị chi tiết của sản phẩm đó:



Nhấn “Go Back” sẽ quay lại màn hình danh sách sản phẩm.

Hướng dẫn:

- Tạo ProductService:

```
export class ProductService {
  productsImage=[
    {"ProductId": "p1", "ProductName": "Coca", "Price": 100, "Image": "assets/h1.png"},
    {"ProductId": "p2", "ProductName": "Pepsi", "Price": 300, "Image": "assets/h2.png"},
    {"ProductId": "p3", "ProductName": "Sting", "Price": 200, "Image": "assets/h3.png"},
  ]
  constructor() { }
  getProductsWithImages()
  {
    return this.productsImage
  }
  getProductDetail(id:any){

    return this.productsImage.find(x=>x.ProductId==id)
  }
}
```

- Các hình ảnh sao chép vào assets
- Tạo component “service-product-image-event”:

+ Viết lệnh cho “service-product-image-event.component.ts”:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { ProductService } from '../product.service';

@Component({
  selector: 'app-service-product-image-event',
  templateUrl: './service-product-image-event.component.html',
  styleUrls: ['./service-product-image-event.component.css']
})
export class ServiceProductImageEventComponent {
```

```
public products:any
constructor(pservice: ProductService,private router:Router){
  this.products=pservice.getProductsWithImages()
}
viewDetail(f:any)
{
  this.router.navigate(['service-product-image-event',f.ProductId])
}
}
```

+Viết lệnh cho “service-product-image-event.component.html”:

```
<p>service-product-image-event works!</p>
<table border="1">
  <tr>
    <td>Ma San Pham</td>
    <td>Ten San Pham</td>
    <td>Gia San Pham</td>
    <td>Picture</td>
    <td>#</td>
  </tr>
  <tbody *ngFor="let p of products">
    <tr>
      <td>{{p.ProductId}}</td>
      <td>{{p.ProductName}}</td>
      <td>{{p.Price}}</td>
      <td>
        
      </td>
      <td>
        <a style="cursor:pointer;" (click)="viewDetail(p)" title="Click
here to view detail">Details</a>
      </td>
    </tr>
  </tbody>
</table>
```

- Tạo component “service-product-image-event”:

+ viết lệnh cho “service-product-image-event-detail.component.ts”:

```
import { Component } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { ProductService } from '../product.service';

@Component({
  selector: 'app-service-product-image-event-detail',
  templateUrl: './service-product-image-event-detail.component.html',
  styleUrls: ['./service-product-image-event-detail.component.css']
})
export class ServiceProductImageEventDetailComponent {
  selectedProduct:any
```

```

    constructor(private activateRoute:ActivatedRoute,private _fs:ProductService,
private router:Router)
    {
        activateRoute.paramMap.subscribe(
            (param)=>{
                let id=param.get('id')

                if(id!=null)
                {
                    this.selectedProduct=_fs.getProductDetail(id)
                }
            }
        )
    }
    goBack(){
        this.router.navigate(['service-product-image-event'])
    }
}

```

+ viết lệnh cho “service-product-image-event-detail.component.html”:

```

<p>service-product-image-event-detail works!</p>
<button (click)="goBack()">Go Back</button>
<table>
  <tr>
    <td>ID:</td>
    <td>{{selectedProduct.ProductId}}</td>
  </tr>
  <tr>
    <td>Name:</td>
    <td>{{selectedProduct.ProductName}}</td>
  </tr>
  <tr>
    <td>Price:</td>
    <td>{{selectedProduct.Price}}</td>
  </tr>
  <tr>
    <td colspan="2">
      <img [src]="selectedProduct.Image"/>
    </td>
  </tr>
</table>

```

- Cấu hình Routing:

```

{path:'service-product-image-event',
component:ServiceProductImageEventComponent},
{path:'service-product-image-event/:id',
component:ServiceProductImageEventDetailComponent},

```

Bài 14- Json Array Model – Product - Catalog

Yêu cầu:

Cho CatalogService có cấu trúc như dưới đây:

```
export class CatalogService {
  datas=[
    { "Cateid": "cate1", "CateName": "nuoc ngot",
      "Products": [
        { "ProductId": "p1", "ProductName": "Coca", "Price": 100,
          "Image": "assets/h1.png" },
        { "ProductId": "p2", "ProductName": "Pepsi", "Price": 300,
          "Image": "assets/h2.png" },
        { "ProductId": "p3", "ProductName": "Sting", "Price": 200,
          "Image": "assets/h3.png" },
      ]
    },
    { "Cateid": "cate2", "CateName": "Bia",
      "Products": [
        { "ProductId": "p4", "ProductName": "Heleiken", "Price": 500,
          "Image": "assets/h4.png" },
        { "ProductId": "p5", "ProductName": "333", "Price": 400,
          "Image": "assets/h5.png" },
        { "ProductId": "p6", "ProductName": "Sai Gon", "Price": 600,
          "Image": "assets/h6.png" },
      ]
    },
  ]

  constructor() { }
  getCategories()
  {
    return this.datas
  }
}
```

- Tạo component và dùng ngFor lồng nhau để hiển thị danh mục và sản phẩm như hình dưới đây:

Ma Danh Muc		Ten Danh Muc	
cate1		nuoc ngot	
Ma San Pham	Ten San Pham	Gia San Pham	Picture
p1	Coca	100	
p2	Pepsi	300	
p3	Sting	200	
cate2		Bia	
Ma San Pham	Ten San Pham	Gia San Pham	Picture
p4	Heleiken	500	
p5	333	400	
p6	Sai Gon	600	

Bài 15- Json Array Model – Product– Http Service(*)

Yêu cầu:

Tạo một cơ sở dữ liệu json trong “assets/data/products.json” có cấu trúc mẫu:

```
[
  {"ProductId": "p1", "ProductName": "Coca", "Price": 100, "Image": "assets/h1.png"},
  {"ProductId": "p2", "ProductName": "Pepsi", "Price": 300, "Image": "assets/h2.png"},
  {"ProductId": "p3", "ProductName": "Sting", "Price": 200, "Image": "assets/h3.png"}
]
```

Hãy dùng cơ chế Service dạng Http để hiển thị dữ liệu như hình:

Ma San Pham	Ten San Pham	Gia San Pham	Picture
p1	Coca	100	
p2	Pepsi	300	
p3	Sting	200	

Hướng dẫn:

-Tạo Interface:

```
export interface IProduct{  
  ProductId:string,  
  ProductName:string,  
  Price:number,  
  Image:string  
}
```

(Có thể để trống toàn bộ các property):

```
export interface IProduct{  
  
}
```

-Tạo “ProductHttpService”:

```
export class ProductHttpService {  
  
  private _url:string="./assets/data/products.json";  
  constructor(private _http: HttpClient) { }  
  
  getProducts():Observable<IProduct[]>{  
    return this._http.get<IProduct[]>(this._url)  
  }  
}
```

-Tạo component “service-product-http”:

+ File “service-product-http.component.ts”:

```
export class ServiceProductHttpComponent {  
  products:any;  
  constructor(private _service: ProductHttpService){  
    this._service.getProducts().subscribe({  
      next:(data)=>{this.products=data}  
    })  
  }  
}
```

+ File “service-product-http.component.html”:

```
<p>service-product-http works!</p>  
<table border="1">  
  <tr>  
    <td>Ma San Pham</td>  
    <td>Ten San Pham</td>  
    <td>Gia San Pham</td>
```

```
<td>Picture</td>
</tr>
<tbody>
  <tr *ngFor="let p of products">
    <td>{{p.ProductId}}</td>
    <td>{{p.ProductName}}</td>
    <td>{{p.Price}} </td>
    <td>
      
    </td>
  </tr>
</tbody>
</table>
```

Bài 16- Json Array Model – Product– Http Service Handle Error (*)

Yêu cầu:

Tương tự như bài tập trước, tuy nhiên bài này xử lý lỗi ngoại lệ xảy ra. Ví dụ như cơ sở dữ liệu không tồn tại.

Hướng dẫn:

-Bổ sung “ProductHttpService”, và cố tình sửa sai đường dẫn dữ liệu:

```
export class ProductHttpService {

  private _url:string="./assets/data/productsXXX.json";
  constructor(private _http: HttpClient) { }

  getProducts():Observable<IProduct[]>{
    return this._http.get<IProduct[]>(this._url)
  }
  getProductsHandleError()
  {
    return this._http.get<IProduct[]>(this._url)
      .pipe(retry(3),
        catchError(this.handleError))
  }

  handleError(error:HttpErrorResponse){
    return throwError(()=>new Error(error.message))
  }

}
```

-Tạo component “service-product-http-handle-error”:

+ File “service-product-http-handle-error.component.ts”:

```
export class ServiceProductHttpHandleErrorComponent {
  products:any
  errMessage:string=''
  constructor(_service:ProductHttpService){
    _service.getProductsHandleError().subscribe({
      next:(data)=>{this.products=data},
      error:(err)=>{this.errMessage=err}
    })
  }
}
```

+ File “service-product-http-handle-error.component.html”:

```
<p>service-product-http-handle-error works!</p>
{{errMessage}}
<table border="1">
  <tr>
    <td>Ma San Pham</td>
    <td>Ten San Pham</td>
    <td>Gia San Pham</td>
    <td>Picture</td>
  </tr>
  <tbody>
    <tr *ngFor="let p of products">
      <td>{{p.ProductId}}</td>
      <td>{{p.ProductName}}</td>
      <td>{{p.Price}}</td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>
```

Chạy lên ta có kết quả (trường hợp ta đã sửa sai đường dẫn cơ sở dữ liệu):

service-product-http-handle-error works!

Error: Http failure response for http://localhost:4200/assets/data/products1.json: 404 Not Found

Ma San Pham	Ten San Pham	Gia San Pham	Picture
-------------	--------------	--------------	---------

Nếu sửa lại cho đúng thì ta có kết quả như bài trước đó.

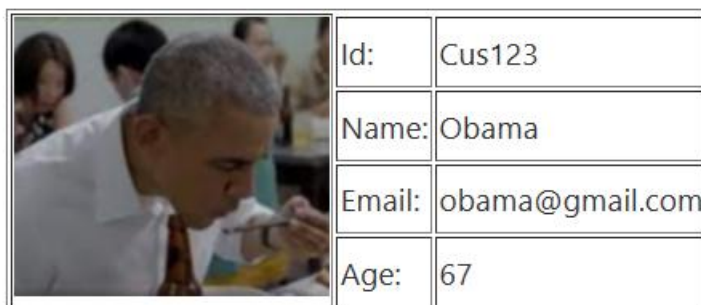
Bài 17- Json Object Model – Customer – Service

Yêu cầu:

Tạo một Component để hiển thị Khách hàng như hình bên dưới. Yêu cầu dùng JsonObject và cơ chế binding. Dữ liệu khai báo trong Type Script.

```
customer={
  "Id": "Cus123",
  "Name": "Obama",
  "Email": "obama@gmail.com",
  "Age": 67,
  "Image": "assets/avatars/obama-avatar.png"
}
```

Giao diện khi chạy component:



Bài 18- Json Array Model – Group Customers (*)

Yêu cầu:

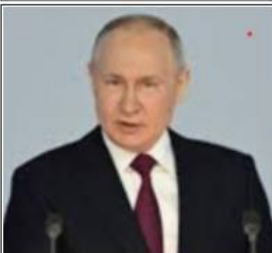
Tạo 1 file Json lưu danh sách Khách hàng trong “assets/data/customers.json” có cấu trúc mẫu như dưới đây:

```
[
  {
    "CustomerTypeId": 1, "CustomerTypeName": "VIP",
    "Customers": [
      {
        "Id": "Cus123",
        "Name": "Obama",
        "Email": "obama@gmail.com",
        "Age": 67,
        "Image": "assets/avatars/obama-avatar.png"
      },
      {
        "Id": "Cus456",
        "Name": "Kim jong Un",
        "Email": "unun@gmail.com",
        "Age": 38,

```

```
"Image": "assets/avatars/unun-avatar.png"
}
,
{
  "Id": "Cus789",
  "Name": "Putin",
  "Email": "putin@gmail.com",
  "Age": 77,
  "Image": "assets/avatars/putin-avatar.png"
}]
},
{ "CustomerId": 2, "CustomerTypeName": "Normal",
  "Customers": [{
    "Id": "Cus000",
    "Name": "Hồ Cẩm Đào",
    "Email": "hodao@gmail.com",
    "Age": 16,
    "Image": "assets/avatars/hodao-avatar.png"
  },
  {
    "Id": "Cus111",
    "Name": "Tap Can Binh",
    "Email": "binhbinh@gmail.com",
    "Age": 67,
    "Image": "assets/avatars/binhbinh-avatar.png"
  },
  ],
},
]
```

Áp dụng bài 14 (nhóm) và bài 16(service http) để làm bài này.

Id	Name	Email	Age	#
1 - VIP				
Cus123	Obama	obama@gmail.com	67	
Cus456	Kim jong Un	unun@gmail.com	38	
Cus789	Putin	putin@gmail.com	77	
2 - Normal				
Cus000	Hồ Cẩm Đào	hodao@gmail.com	16	
Cus111	Tập Cận Bình	binhbinh@gmail.com	67	

Bài 19 – Cấu hình Routing

Hiểu đại khái Routing là điều hướng trang, ví dụ ta thấy:

<http://localhost:4200/products>

<http://localhost:4200/customers>

<http://localhost:4200/login>

Khi truy suất vào các link trên và nó điều hướng vào các trang nội dung được định nghĩa thì hiểu là Routing.

Để điều hướng Page ta dùng kỹ thuật Routing, các bước làm như sau:

Bước 1: Tạo các Component, ví dụ ta tạo ProductComponent, ListProductComponent, ServiceProductComponent, ...

Sau đó ta muốn gõ các link sau khi mở các component này:

<http://localhost:4200/product> => Mở ProductComponent

<http://localhost:4200/list-product> => Mở ListProductComponent

<http://localhost:4200/service-product> => Mở ServiceProductComponent

Bước 2: Chỉnh sửa file “app-routing.module.ts”

Khai báo lệnh path và component cho mảng routes như dưới đây:

```
const routes: Routes = [  
  {path: 'product', component: ProductComponent},  
  {path: 'list-product', component: ListProductComponent},  
  {path: 'service-product', component: ServiceProductComponent},  
];
```

Cuối file này tiếp tục khai báo biến “RoutingComponent”:

```
export const RoutingComponent=[  
ProductComponent,  
ListProductComponent,  
ServiceProductComponent,  
]
```

Bước 3: Chỉnh sửa file “app.module.ts”:

Thẻ “declarations” bổ sung RoutingComponent mà ta khai báo ở bước 2

```
@NgModule({  
  declarations: [  
    AppComponent,  
    RoutingComponent,  
  ],  
  imports: [  
    BrowserModule,  
    AppRoutingModule,  
    FormsModule,  
    HttpClientModule  
  ],  
  providers: [],  
  bootstrap: [AppComponent]  
})
```

Tới đây chương trình đã hiểu các Routing rồi.

Bước 4: (Bước này không cần thiết)

Giả sử trong ứng dụng (trang chủ) ta muốn điều hướng tới các Routing mà ta đã khai báo thì làm như sau:

Mở file “app.component.html”

```
<table>
  <tr>
    <td>
      <input type="button" value="Product" [routerLink]='product'>
    </td>
    <td>
      <input type="button" value="List-Product" [routerLink]='list-product'>
    </td>
    <td>
      <input type="button" value="Service-Product" [routerLink]='service-product'>
    </td>
  </tr>
</table>
```

Bài 20 – Routing for Page Not Found

Để điều hướng qua Page Not found khi người dùng chọn sai, ta cấu hình dòng cuối trong routes:

```
const routes: Routes = [
  {path: 'product', component: ProductComponent},
  {path: 'list-product', component: ListProductComponent},
  {path: 'service-product', component: ServiceProductComponent},
  {path: "**", component: PageNotFoundComponent},
];
```

Module 3 – AngularJS (tt) - Form

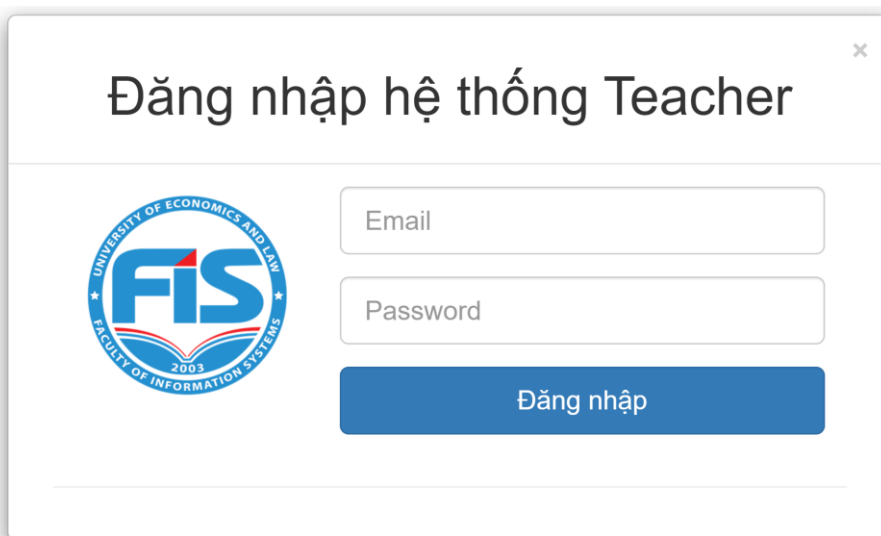
Nội dung kiến thức thực hành:

- + Hiểu được ý nghĩa và chức năng của Form, Reactive Form
- + Sử dụng FormsModule, ngModel
- + Binding Data, Validation
- + Hiểu và sử dụng được các control trên Form: Label, Textbox, Button, Combo, ListBox, TextArea....

Bài 21– Login Screen

Yêu cầu:

Thiết kế giao diện login tương tự như dưới đây:



The image shows a login form titled "Đăng nhập hệ thống Teacher". On the left is the logo of the Faculty of Information Systems (FIS) at the University of Economics and Law (UEL), established in 2003. To the right of the logo are two input fields: "Email" and "Password". Below these fields is a blue button labeled "Đăng nhập".

- Yêu cầu tạo 1 model có thuộc tính email và password
- Khi nhấn nút “Đăng nhập” thì hiển thị JSONObject as string của thông tin và người dùng nhập ngay bên dưới nút “Đăng nhập” (thẻ p, chữ màu đỏ).
- Mở rộng 1: Bổ sung validation: email không hợp lệ, mật khẩu tối thiểu 5 ký tự
- Mở rộng 2: Bổ sung giả lập nếu nhập đúng email và mật khẩu thì navigate tới một component bất kỳ nào đó.
- Mở rộng 3: Bổ sung thêm nút Checkbox(lưu thông tin đăng nhập), tìm từ khóa “Angular Local Storage”. Nếu trước đó checked thì lần sau sẽ nạp lại thông tin đăng nhập khi chạy lại màn hình login.

Bài 22 – Form Đăng ký Khóa học

Yêu cầu:

Thiết kế màn hình Đăng ký khóa học như hình dưới đây:

Đăng ký khóa học

Tên:

Email:

Điện thoại:

Chọn khóa học:

▼

Chọn ca học: ☐ Sáng ☒ Tối

☒ Đồng ý với điều khoản

{ "name": "Nam Anh", "email": "anh@gmail.com", "phone": "0909090909", "course": "Ruby", "time": "tối", "agree": true }

- Yêu cầu khai báo lớp model để binding dữ liệu
- Khi bấm Submit, thông tin JsonObject as string sẽ hiển thị ngay bên dưới nút Submit.

Hướng dẫn:

- Sử dụng ReactiveFormsModule

Bài 23 – Form Đăng ký Khóa học - Validation

Yêu cầu:

Mở rộng bài tập Form Đăng ký khóa học, bổ sung Validation

Đăng ký khóa học

Tên:

Nhập tên



Bạn chưa nhập tên

Email:

anh@gmail.com

Điện thoại:

0909090909

Chọn khóa học:



Vui lòng chọn khóa học

Chọn ca học:



Sáng



Tối



Đồng ý với điều khoản

Submit

Khi nhấn nút “Submit”:

- Nếu tên rỗng sẽ báo lỗi
- Nếu khóa học chưa chọn sẽ báo lỗi
- Nếu chưa bấm vào “đồng ý với điều khoản” sẽ báo lỗi

Hướng dẫn:

Bài 24- Mathematics

Yêu cầu:

Tạo trang web với Table và Textbox, Button để tính MAX, MIN, sin, cos... Theo mô tả dưới đây:

a(So Thu 1):	30
b(So Thu 2)	45
c(So Thu 3)	90
KET QUA	-0.9880316240928618
MAX	MIN
sin(a)	Cos(a)
a^b	

- Báo lỗi (tô nền đỏ) các ô nhập liệu nếu sai định dạng dữ liệu số.

Module 4 – AngularJS (tt) Triệu gọi External Restful API

Nội dung kiến thức thực hành:

- + Luyện tập CSS
- + Nhúng Bootstrap
- + Luyện tập Service
- + Luyện tập Component
- + Luyện tập HTTP Call external restful API
- + Luyện tập xử lý Http Handling Error
- + Xử lý CORS Policy

Bài 25: Tương tác External Restful API- Ngân hàng Đông Á

Yêu cầu:

Ngân hàng Đông Á cung cấp Restful API sau để truy suất tỉ giá hối đoái của một số quốc gia:

<https://www.dongabank.com.vn/exchange/export>

Dưới đây là dữ liệu của API trả về:

```
{
  "items": [
    {
      "type": "GBP",
      "imageurl": "https://www.dongabank.com.vn/images/flag/GBP.gif",
      "muatienmat": "28050",
      "muack": "28170",
      "bantienmat": "28760",
      "banck": "28710"
    },
    {
      "type": "AUD",
      "imageurl": "https://www.dongabank.com.vn/images/flag/AUD.gif",
      "muatienmat": "15800",
      "muack": "15890",
      "bantienmat": "16230",
      "banck": "16190"
    },
    {
      "type": "CAD",
      "imageurl": "https://www.dongabank.com.vn/images/flag/CAD.gif",
      "muatienmat": "17220",
      "muack": "17320",
      "bantienmat": "17690",
      "banck": "17650"
    },
    {
      "type": "CHF",
      "imageurl": "https://www.dongabank.com.vn/images/flag/CHF.gif",
      "muatienmat": "22610",
      "muack": "25000",
      "bantienmat": "23080",
      "banck": "25500"
    },
    {
      "type": "CNY",
      "imageurl": "https://www.dongabank.com.vn/images/flag/CNY.gif",
      "muatienmat": "3000",
      "muack": "3000",
      "bantienmat": "3500",
      "banck": "3500"
    },
    {
      "type": "EUR",
      "imageurl": "https://www.dongabank.com.vn/images/flag/EUR.gif",
      "muatienmat": "24860",
      "muack": "24970",
      "bantienmat": "25490",
      "banck": "25440"
    },
    {
      "type": "HKD",
      "imageurl": "https://www.dongabank.com.vn/images/flag/HKD.gif",
      "muatienmat": "2410",
      "muack": "2900",
      "bantienmat": "2920",
      "banck": "3060"
    },
    {
      "type": "XAU",
      "imageurl": "https://www.dongabank.com.vn/images/flag/XAU.gif",
      "muatienmat": "6620000",
      "muack": "6655000",
      "bantienmat": "6690000",
      "banck": "6655000"
    },
    {
      "type": "USD",
      "imageurl": "https://www.dongabank.com.vn/images/flag/USD.gif",
      "muatienmat": "23590",
      "muack": "23590",
      "bantienmat": "23920",
      "banck": "23870"
    },
    {
      "type": "THB",
      "imageurl": "https://www.dongabank.com.vn/images/flag/THB.gif",
      "muatienmat": "610",
      "muack": "670",
      "bantienmat": "700",
      "banck": "690"
    },
    {
      "type": "SGD",
      "imageurl": "https://www.dongabank.com.vn/images/flag/SGD.gif",
      "muatienmat": "17310",
      "muack": "17470",
      "bantienmat": "17820",
      "banck": "17800"
    },
    {
      "type": "PNJ_DAB",
      "imageurl": "https://www.dongabank.com.vn/images/flag/PNJ_DAB.gif",
      "muatienmat": "5330000",
      "muack": "5330000"
    }
  ]
}
```

```
, "bantienmat": "5430000", "banck": "5430000"}, {"type": "NZD", "imageurl": "https://www.dongabank.com.vn/images/flag/NZD.gif", "muatienmat": "", "muack": "14650", "bantienmat": "", "banck": "15060"}, {"type": "JPY", "imageurl": "https://www.dongabank.com.vn/images/flag/JPY.gif", "muatienmat": "169", "muack": "172.4", "bantienmat": "176.1", "banck": "175.6"}]]
```

Yêu cầu Sinh viên sử dụng HTTP và handling error để triệu gọi API này và hiển thị lên giao diện như hình dưới đây:

Dong A Bank					
Type	ImageURL	MuaTienMat	MuaCK	BanTienMat	BanCK
CAD		17220	17320	17690	17650
XAU		6620000	6655000	6690000	6655000
AUD		15800	15890	16230	16190
CHF		22610	25000	23080	25500
CNY		3000	3000	3500	3500
EUR		24860	24970	25490	25440
GBP		28050	28170	28760	28710
HKD		2410	2900	2920	3060
JPY		169	172.4	176.1	175.6
NZD			14650		15060
PNJ_DAB		5330000	5330000	5430000	5430000
SGD		17310	17470	17820	17800
THB		610	670	700	690
USD		23590	23590	23920	23870

Hướng dẫn chi tiết:

Như đã được học, dữ liệu Json có 2 loại: JsonObject {} và JsonArray []. Tuy nhiên dữ liệu của Ngân hàng đông á gửi về lại được bao bọc bởi ngoặc tròn (). Như vậy Ta cần tải dữ liệu và xóa ngoặc () đi để nó trở thành JsonObject (được bao bọc bởi ngoặc nhọn {}).

Quan sát thấy rằng “items” chứa mảng các JsonObject:

```
{"type", "imageurl", "muatienmat", "muack", "bantienmat", "banck"}
```

Như vậy ta cần tạo 2 interface (hoặc 2 class) để mô hình hóa và mapping dữ liệu.

Bước 1: Tạo các lớp mô hình hóa cho dữ liệu

Tạo file “DongABankItem.ts” và khai báo interface như dưới đây:

```
export interface IDongABankItem
{
    type:string,
    imageurl:string,
    muatienmat:string,
    muack:string,
    bantienmat:string,
    banck:string,
}
```

Tiếp tục tạo file “” và khai báo interface như dưới đây:

```
import { IDongABankItem } from "../DongABankItem";

export interface IDongABankData
{
    items:Array<IDongABankItem>
}
```

Như vậy với cách khai báo 2 interface ở trên thì chương trình sẽ mapping được dữ liệu Json mà Ngân Hàng Đông Á gửi về thành mô hình hướng đối tượng.

Điều quan trọng nhất là chúng ta phải hiểu đúng cấu trúc dữ liệu Json mà bất kỳ 1 API nào đó trả về, việc hiểu đúng cấu trúc Json thì ta mới khai báo các mô hình lớp chính xác được.

Các thuộc tính khai báo trong interface hay trong class Ta nên copy + paste từ dữ liệu mà API trả về nhằm tránh bị sai chính tả (phải đúng cả chữ Hoa, chữ Thường).

Bước 2: Khai báo Service

Tạo Service tên “DongABankService” lúc này file “dong-abank.service.ts” sẽ được tạo ra, ta tiến hành coding cho file này.

```
import { JsonPipe } from '@angular/common';
import { HttpClient, HttpResponse, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, map, Observable, retry, throwError } from 'rxjs';
import { IDongABankData } from '../interfaces/DongABankData';
import { IDongABankItem } from '../interfaces/DongABankItem';

@Injectable({
    providedIn: 'root'
})
export class DongABankService {

    private _url:string="/exchange/export"
    constructor(private _http: HttpClient) { }

    getDongABankData()
```

```
{
  const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
  const requestOptions:Object={
    headers:headers,
    responseType:"text"
  }
  return this._http.get<any>(this._url,requestOptions).pipe(
    map(res=>JSON.parse(res.slice(1,-1)) as IDongABankData),
    retry(3),
    catchError(this.handleError))
}

handleError(error:HttpErrorResponse){
  return throwError(()=>new Error(error.message))
}
}
```

res.slice(1,-1) xóa các ngoặc tròn ở đầu và ở cuối dữ liệu mà API trả về

JSON.parse(res.slice(1,-1)) đưa dữ liệu về JSon

JSON.parse(res.slice(1,-1)) as IDongABankData mô hình hóa Json thành mô hình hướng đối tượng.

handleError giúp kiểm tra lỗi trong quá trình triệu gọi API

Bước 3: Tạo Component

Tạo “DongABankComponent” component, lúc này ta coding trong file “dong-abank.component.ts” như sau:

```
import { Component } from '@angular/core';
import { DongABankService } from '../dong-abank.service';

@Component({
  selector: 'app-dong-abank',
  templateUrl: './dong-abank.component.html',
  styleUrls: ['./dong-abank.component.css']
})
export class DongABankComponent {
  data:any
  errMessage:string=''
  constructor(_service:DongABankService){
    _service.getDongABankData().subscribe({
      next:(data)=>{
        this.data=data
      },
      error:(err)=>{
        this.errMessage=err}
    })
  }
}
```

Tiếp tục coding html trong file “dong-abank.component.html”:

```
<h3 class="text-center text-white bg-dark p-3">Dong A Bank</h3>
{{errorMessage}}
<div class="container">
  <table class="table table -hover">
    <thead>
      <tr>
        <th>Type</th>
        <th>ImageURL</th>
        <th>MuaTienMat</th>
        <th>MuaCK</th>
        <th>BanTienMat</th>
        <th>BanCK</th>
      </tr>
    </thead>
    <tbody>
      <tr *ngFor="let item of data.items">
        <td>{{item.type}}</td>
        <td></td>
        <td>{{item.muatienmat}}</td>
        <td>{{item.muack}}</td>
        <td>{{item.bantienmat}}</td>
        <td>{{item.banck}}</td>
      </tr>
    </tbody>
  </table>
</div>
```

Bước 4: Xử lý CORS Policy

Lưu ý rằng, khi gọi external restful API thông thường ta sẽ gặp lỗi tương tự:

✖ Access to XMLHttpRequest at 'https://www.dongabank.com.vn/e_exchange/export' from origin 'http://localhost:4200' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

Do đó ta cần xử lý Xử lý CORS Policy như sau:

- Tạo file src/**proxy.conf.json** và khai báo cú pháp:

Vì thông thường một dự án có thể kết nối nhiều API khác nhau (tức là mảng) nên ta khai báo nội dung của file proxy này dạng mảng cho tiện lợi:

```
[
  {
    "context": ["/exchange"],
    "target": "https://www.dongabank.com.vn",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  }
]
```

“context” trỏ tới API mà ta muốn truy suất

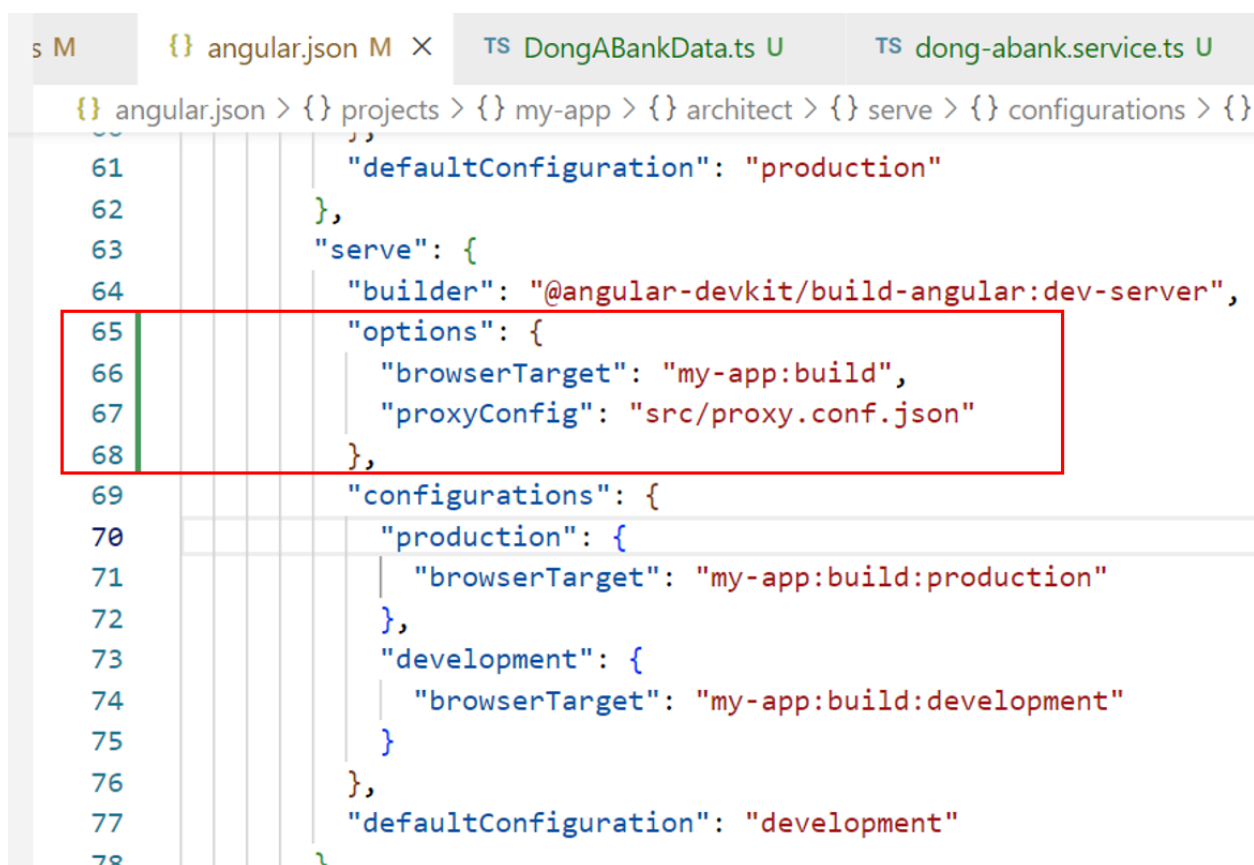
“target” là domain mà API được cấu hình

Các thông số khác thiết lập như minh họa ở trên. Khi triệu gọi nhiều API khác thì cứ Copy + paste cấu trúc {} thành nhiều cái, rồi chỉnh lại context và target là được.

Trong trường hợp nếu dự án chỉ truy suất tới 1 API duy nhất thì ta có thể khai báo dạng Object như dưới đây cho “proxy.conf.json” (nhưng chắc là hiếm, vì 1 dự án thường truy suất tới nhiều API):

```
{
  "/exchange":{
    "target":"https://www.dongabank.com.vn",
    "secure":true,
    "changeOrigin":true,
    "logLevel":"debug"
  }
}
```

Sau khi khai báo xong file “proxy.conf.json”, ta tiến hành cấu hình “angular.json”:



```
s M    {} angular.json M X    TS DongABankData.ts U    TS dong-abank.service.ts U
{} angular.json > {} projects > {} my-app > {} architect > {} serve > {} configurations > {}
61      "defaultConfiguration": "production"
62    },
63    "serve": {
64      "builder": "@angular-devkit/build-angular:dev-server",
65      "options": {
66        "browserTarget": "my-app:build",
67        "proxyConfig": "src/proxy.conf.json"
68      },
69      "configurations": {
70        "production": {
71          "browserTarget": "my-app:build:production"
72        },
73        "development": {
74          "browserTarget": "my-app:build:development"
75        }
76      },
77      "defaultConfiguration": "development"
78    }
```

Bổ sung khung màu đỏ.

```
"serve": {
  "builder": "@angular-devkit/build-angular:dev-server",
  "options": {
    "browserTarget": "my-app:build",
    "proxyConfig": "src/proxy.conf.json"
  },
  "configurations": {
    "production": {
      "browserTarget": "my-app:build:production"
```

```
    },
    "development": {
      "browserTarget": "my-app:build:development"
    }
  },
  "defaultConfiguration": "development"
},
```

"browserTarget" trỏ tới tên ứng dụng mà mình tạo, ví dụ "my-app:build"

"proxyConfig" trỏ tới file proxy mà mình định nghĩa, ví dụ "src/proxy.conf.json"

Bước 5: Cài đặt BootStrap – JQuery (không cần thiết nếu không dùng)

Thực hiện lệnh:

```
npm install --save bootstrap jquery
```

Tiếp tục cấu hình "angular.json":

```
12     "architect": {
13       "build": {
14         "builder": "@angular-devkit/build-angular:browser",
15         "options": {
16           "outputPath": "dist/my-app",
17           "index": "src/index.html",
18           "main": "src/main.ts",
19           "polyfills": [
20             "zone.js"
21           ],
22           "tsConfig": "tsconfig.app.json",
23           "assets": [
24             "src/favicon.ico",
25             "src/assets"
26           ],
27           "styles": [
28             "src/styles.css",
29             "node_modules/bootstrap/dist/css/bootstrap.min.css"
30           ],
31           "scripts": [
32             "node_modules/jquery/dist/jquery.min.js",
33             "node_modules/bootstrap/dist/js/bootstrap.min.js"
34           ]
35         },
```

Bổ sung khung màu đỏ.

```
"styles": [
  "src/styles.css",
  "node_modules/bootstrap/dist/css/bootstrap.min.css"
],
"scripts": [
  "node_modules/jquery/dist/jquery.min.js",
  "node_modules/bootstrap/dist/js/bootstrap.min.js"
]
```

Chạy chương trình và ta có được kết quả như mong muốn.

Bài 26: Tương tác External Restful API- Product Service 1

Yêu cầu:

Sinh viên thực hiện tương tự như bài tập triệu gọi API của ngân hàng Đông Á, load danh sách sản phẩm từ API: <https://fakestoreapi.com/products> hiển thị giao diện như sau:

Fake Product Data						
Id	title	price	description	category	image	rating
1	Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops	109.95	Your perfect pack for everyday use and walks in the forest. Stash your laptop (up to 15 inches) in the padded sleeve, your everyday	men's clothing		3.9
2	Mens Casual Premium Slim Fit T-Shirts	22.3	Slim-fitting style, contrast raglan long sleeve, three-button henley placket, light weight & soft fabric for breathable and comfortable wearing. And Solid stitched shirts with round neck made for durability and a great fit for casual fashion wear and diehard baseball fans. The Henley style round neckline includes a three-button placket.	men's clothing		4.1
3	Mens Cotton Jacket	55.99	great outerwear jackets for Spring/Autumn/Winter, suitable for many occasions, such as working, hiking, camping, mountain/rock climbing, cycling, traveling or other outdoors. Good gift choice for you or your family member. A warm hearted love to Father, husband or son in this thanksgiving or Christmas Day.	men's clothing		4.7
4	Mens Casual Slim Fit	15.99	The color could be slightly different between on the screen and in practice. / Please note that body builds vary by person, therefore, detailed size information should be reviewed below on the product description.	men's clothing		2.1

Hướng dẫn chi tiết:

Quan sát dữ liệu trả về của API <https://fakestoreapi.com/products>:

```
[{"id":1,"title":"Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops","price":109.95,"description":"Your perfect pack for everyday use and walks in the forest. Stash your laptop (up to 15 inches) in the padded sleeve, your everyday","category":"men's clothing","image":"https://fakestoreapi.com/img/81fPKd-2AYL.AC_SL1500.jpg","rating":{"rate":3.9,"count":120}},{ "id":2,"title":"Mens Casual Premium Slim Fit T-Shirts","price":22.3,"description":"Slim-fitting style, contrast raglan long sleeve, three-button henley placket, light weight & soft fabric for breathable and comfortable wearing. And Solid stitched shirts with round neck made for durability and a great fit for casual fashion wear and diehard baseball fans. The Henley style round neckline includes a three-button placket.","category":"men's clothing","image":"https://fakestoreapi.com/img/71-3HjGNDUL.AC_SY879.SX.UX.SY.UY.jpg","rating":{"rate":4.1,"count":259}},{ "id":3,"title":"Mens Cotton Jacket","price":55.99,"description":"great outerwear jackets for Spring/Autumn/Winter, suitable for many occasions, such as working, hiking, camping, mountain/rock climbing, cycling, traveling or other outdoors. Good gift choice for you or your family member. A warm hearted love to Father, husband or son in this thanksgiving or Christmas Day.","category":"men's clothing","image":"https://fakestoreapi.com/img/71li-ujt1UL.AC_UX679.jpg","rating":{"rate":4.7,"count":500}},{ "id":4,"title":"Mens Casual Slim Fit","price":15.99,"description":"The color could be slightly different between on the screen and in practice. / Please note that body builds vary by person, therefore, detailed size information should be reviewed below on the product description.","category":"men's clothing","image":"https://fakestoreapi.com/img/71YXzeOus1L.AC_UY879.jpg","rating":{"rate":2.1,"count":430}},{ "id":5,"title":"John Hardy Women's Legends Naga Gold & Silver Dragon Station Chain Bracelet","price":695,"description":"From our Legends Collection, the Naga was inspired by the
```

mythical water dragon that protects the ocean's pearl. Wear facing inward to be bestowed with love and abundance, or outward for protection.", "category": "jewelery", "image": "https://fakestoreapi.com/img/71pWzhdJNwL._AC_UL640_QL65_ML3_.jpg", "rating": {"rate": 4.6, "count": 400}, {"id": 6, "title": "Solid Gold Petite Micropave", "price": 168, "description": "Satisfaction Guaranteed. Return or exchange any order within 30 days. Designed and sold by Hafeez Center in the United States. Satisfaction Guaranteed. Return or exchange any order within 30 days.", "category": "jewelery", "image": "https://fakestoreapi.com/img/61sbMiUnoGL._AC_UL640_QL65_ML3_.jpg", "rating": {"rate": 3.9, "count": 70}, {"id": 7, "title": "White Gold Plated Princess", "price": 9.99, "description": "Classic Created Wedding Engagement Solitaire Diamond Promise Ring for Her. Gifts to spoil your love more for Engagement, Wedding, Anniversary, Valentine's Day...", "category": "jewelery", "image": "https://fakestoreapi.com/img/71YAIFU48IL._AC_UL640_QL65_ML3_.jpg", "rating": {"rate": 3, "count": 400}, {"id": 8, "title": "Pierced Owl Rose Gold Plated Stainless Steel Double", "price": 10.99, "description": "Rose Gold Plated Double Flared Tunnel Plug Earrings. Made of 316L Stainless Steel", "category": "jewelery", "image": "https://fakestoreapi.com/img/51UDEZMJVpL._AC_UL640_QL65_ML3_.jpg", "rating": {"rate": 1.9, "count": 100}, {"id": 9, "title": "WD 2TB Elements Portable External Hard Drive - USB 3.0", "price": 64, "description": "USB 3.0 and USB 2.0 Compatibility Fast data transfers Improve PC Performance High Capacity; Compatibility Formatted NTFS for Windows 10, Windows 8.1, Windows 7; Reformattting may be required for other operating systems; Compatibility may vary depending on user's hardware configuration and operating system", "category": "electronics", "image": "https://fakestoreapi.com/img/61IBBVJvSDL._AC_SY879_.jpg", "rating": {"rate": 3.3, "count": 203}, {"id": 10, "title": "SanDisk SSD PLUS 1TB Internal SSD - SATA III 6 Gb/s", "price": 109, "description": "Easy upgrade for faster boot up, shutdown, application load and response (As compared to 5400 RPM SATA 2.5\" hard drive; Based on published specifications and internal benchmarking tests using PCMark vantage scores) Boosts burst write performance, making it ideal for typical PC workloads The perfect balance of performance and reliability Read/write speeds of up to 535MB/s/450MB/s (Based on internal testing; Performance may vary depending upon drive capacity, host device, OS and application.)", "category": "electronics", "image": "https://fakestoreapi.com/img/61U7T1koQgL._AC_SX679_.jpg", "rating": {"rate": 2.9, "count": 470}, {"id": 11, "title": "Silicon Power 256GB SSD 3D NAND A55 SLC Cache Performance Boost SATA III 2.5\"", "price": 109, "description": "3D NAND flash are applied to deliver high transfer speeds Remarkable transfer speeds that enable faster bootup and improved overall system performance. The advanced SLC Cache Technology allows performance boost and longer lifespan 7mm slim design suitable for Ultrabooks and Ultra-slim notebooks. Supports TRIM command, Garbage Collection technology, RAID, and ECC (Error Checking & Correction) to provide the optimized performance and enhanced reliability.", "category": "electronics", "image": "https://fakestoreapi.com/img/71kWyMz+c+L._AC_SX679_.jpg", "rating": {"rate": 4.8, "count": 319}, {"id": 12, "title": "WD 4TB Gaming Drive Works with Playstation 4 Portable External Hard Drive", "price": 114, "description": "Expand your PS4 gaming experience, Play anywhere Fast and easy, setup Sleek design with high capacity, 3-year manufacturer's limited warranty", "category": "electronics", "image": "https://fakestoreapi.com/img/61mtL65D4cL._AC_SX679_.jpg", "rating": {"rate": 4.8, "count": 400}, {"id": 13, "title": "Acer SB220Q bi 21.5 inches Full HD (1920 x 1080) IPS Ultra-Thin", "price": 599, "description": "21.5 inches Full HD (1920 x 1080) widescreen IPS display And Radeon free Sync technology. No compatibility for VESA Mount Refresh Rate: 75Hz - Using HDMI port Zero-frame design | ultra-thin | 4ms response time | IPS panel Aspect ratio - 16:9. Color Supported - 16.7 million colors. Brightness - 250 nit Tilt angle -5 degree to 15 degree. Horizontal viewing angle-178 degree. Vertical viewing angle-178 degree 75 hertz", "category": "electronics", "image": "https://fakestoreapi.com/img/81QpKtqPL._AC_SX679_.jpg", "rating": {"rate": 2.9, "count": 250}, {"id": 14, "title": "Samsung 49-Inch CHG90 144Hz Curved Gaming Monitor (LC49HG90DMNXZA) - Super Ultrawide Screen QLED", "price": 999.99, "description": "49 INCH SUPER ULTRAWIDE 32:9 CURVED GAMING MONITOR with dual 27 inch screen side by side QUANTUM DOT (QLED) TECHNOLOGY, HDR support and factory calibration provides stunningly realistic and accurate color and contrast 144HZ HIGH REFRESH RATE and 1ms ultra fast response time work to eliminate motion blur, ghosting, and reduce input lag", "category": "electronics", "image": "https://fakestoreapi.com/img/81Zt42ioCgL._AC_SX679_.jpg", "rating": {"rate": 2.2, "count": 140}, {"id": 15, "title": "BIYIACLESEN Women's 3-in-1 Snowboard Jacket Winter Coats", "price": 56.99, "description": "Note:The Jackets is US standard size, Please choose size as your usual wear Material: 100% Polyester; Detachable Liner Fabric: Warm Fleece. Detachable Functional Liner: Skin Friendly, Lightweight and Warm. Stand Collar Liner jacket, keep you warm in cold weather. Zippered Pockets: 2 Zippered Hand Pockets, 2 Zippered Pockets on Chest (enough to keep cards or keys) and 1 Hidden Pocket Inside. Zippered Hand Pockets and Hidden Pocket keep your things secure. Humanized Design: Adjustable and Detachable Hood and Adjustable cuff to prevent the wind and water, for a comfortable fit. 3 in 1 Detachable Design provide more convenience, you can separate the coat and inner as needed, or wear it together. It is suitable for different season and help you adapt to different climates", "category": "women's clothing", "image": "https://fakestoreapi.com/img/51Y5NI-I5jL._AC_UX679_.jpg", "rating": {"rate": 2.6, "count": 235}, {"id": 16, "title": "Lock and Love Women's Removable Hooded Faux Leather Moto Biker Jacket", "price": 29.95, "description": "100% POLYURETHANE(shell) 100% POLYESTER(lining) 75% POLYESTER 25% COTTON (SWEATER), Faux leather material for style and comfort / 2 pockets of front, 2-For-One Hooded denim style faux leather jacket, Button detail on waist / Detail stitching at sides, HAND WASH ONLY / DO NOT BLEACH / LINE DRY / DO NOT IRON", "category": "women's clothing", "image": "https://fakestoreapi.com/img/81XH0e8fefL._AC_UY879_.jpg", "rating": {"rate": 2.9, "count": 340}, {"id": 17, "title": "Rain Jacket Women Windbreaker Striped Climbing Raincoats", "price": 39.99, "description": "Lightweight perfect for trip or casual wear---Long sleeve with hooded, adjustable drawstring waist design. Button and zipper front closure raincoat, fully stripes Lined and The Raincoat has 2 side pockets are a good size to hold all kinds of things, it covers the hips, and the hood is generous but doesn't overdo it. Attached Cotton Lined Hood with

```
Adjustable Drawstrings give it a real styled look.", "category": "women's clothing", "image": "https://fakestoreapi.com/img/71HblAHs5xL._AC_UY879_-2.jpg", "rating": {"rate": 3.8, "count": 679}}, {"id": 18, "title": "MBJ Women's Solid Short Sleeve Boat Neck V", "price": 9.85, "description": "95% RAYON 5% SPANDEX, Made in USA or Imported, Do Not Bleach, Lightweight fabric with great stretch for comfort, Ribbed on sleeves and neckline / Double stitching on bottom hem", "category": "women's clothing", "image": "https://fakestoreapi.com/img/71z3kpMAYsL._AC_UY879_.jpg", "rating": {"rate": 4.7, "count": 130}}, {"id": 19, "title": "Opna Women's Short Sleeve Moisture", "price": 7.95, "description": "100% Polyester, Machine wash, 100% cationic polyester interlock, Machine Wash & Pre Shrunk for a Great Fit, Lightweight, roomy and highly breathable with moisture wicking fabric which helps to keep moisture away, Soft Lightweight Fabric with comfortable V-neck collar and a slimmer fit, delivers a sleek, more feminine silhouette and Added Comfort", "category": "women's clothing", "image": "https://fakestoreapi.com/img/51leg55uWmdL._AC_UX679_.jpg", "rating": {"rate": 4.5, "count": 146}}, {"id": 20, "title": "DANVOUY Womens T Shirt Casual Cotton Short", "price": 12.99, "description": "95%Cotton,5%Spandex, Features: Casual, Short Sleeve, Letter Print,V-Neck,Fashion Tees, The fabric is soft and has some stretch., Occasion: Casual/Office/Beach/School/Home/Street. Season: Spring,Summer,Autumn,Winter.", "category": "women's clothing", "image": "https://fakestoreapi.com/img/61pHAEJ4NML._AC_UX679_.jpg", "rating": {"rate": 3.6, "count": 145}}]
```

Nhìn cấu trúc trên thì nó đơn giản hơn dữ liệu của Ngân Hàng Đông Á. Nó là mảng các đối tượng và ngay từ đầu nó đã đúng với cấu trúc của Json Data.

Bước 1: Tạo các interface/lớp mô hình

Đối với "rating":{"rate":3.6,"count":145} cần có một interface, đặt tên file "FakeProductRating.ts" và khai báo interface "IFakeProductRating" có nội dung:

```
export interface IFakeProductRating{
  rate:number,
  count:number
}
```

Đối với cấu trúc ở ngoài, tạo file "FakeProduct.ts" và khai báo interface "IFakeProduct" có nội dung:

```
import {IFakeProductRating } from "../FakeProductRating";

export interface IFakeProduct{
  id:number,
  title:string,
  price:number,
  description:string,
  category:string,
  image:string ,
  rate:IFakeProductRating
}
```

Bước 2: Tạo "FakeProductService" Service, viết mã lệnh trong file "fake-product.service.ts" như sau:

```
import { HttpClient, HttpResponseError, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
```

```
import { catchError, map, Observable, retry, throwError } from 'rxjs';
import { IFakeProduct } from '../interfaces/FakeProduct';
@Injectable({
  providedIn: 'root'
})
export class FakeProductService {
  private _url:string="/products"
  constructor(private _http: HttpClient) { }
  getFakeProductData():Observable<any> {
    const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
    const requestOptions:Object={
      headers:headers,
      responseType:"text"
    }
    return this._http.get<any>(this._url,requestOptions).pipe(
      map(res=>JSON.parse(res) as Array<IFakeProduct>),
      retry(3),
      catchError(this.handleError))
  }
  handleError(error:HttpErrorResponse){
    return throwError(()=>new Error(error.message))
  }
}
```

Bước 3: Tạo Component

Tạo “FakeProductComponent” component, lúc này ta coding trong file “fake-product.component.ts” như sau:

```
import { Component, OnInit } from '@angular/core';
import { FakeProductService } from '../fake-product.service';
@Component({
  selector: 'app-fake-product',
  templateUrl: './fake-product.component.html',
  styleUrls: ['./fake-product.component.css']
})
export class FakeProductComponent {
  data:any
  errMessage:string=''
  constructor(_service:FakeProductService){
    _service.getFakeProductData().subscribe({
      next:(data)=>{ this.data=data},
      error:(err)=>{
        this.errMessage=err
      }
    })
  }
}
```

Chỉnh sửa HTML trong file “fake-product.component.html”:

```
<p>binding-style works!</p>
<table>
  <thead>
    <tr>
      <th>Id</th>
      <th>title</th>
      <th>price</th>
      <th>description</th>
      <th>category</th>
      <th>image</th>
      <th>rating</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let item of data">
      <td>{{item.id}}</td>
      <td>{{item.title}}</td>
      <td>{{item.price}}</td>
      <td>{{item.description}}</td>
      <td>{{item.category}}</td>
      <td></td>
      <td>{{item.rating.rate}}</td>
    </tr>
  </tbody>
</table>
```

Bước 4: Xử lý CORS Policy

Khai báo và cấu hình tương tự như bài triệu gọi API của Ngân Hàng Đông Á. Ta chỉnh sửa (hoặc tạo mới) “proxy.conf.json”. Như đã giải thích ở bài trước, vì trong 1 dự án có thể kết nối nhiều API nên nội dung file này như sau:

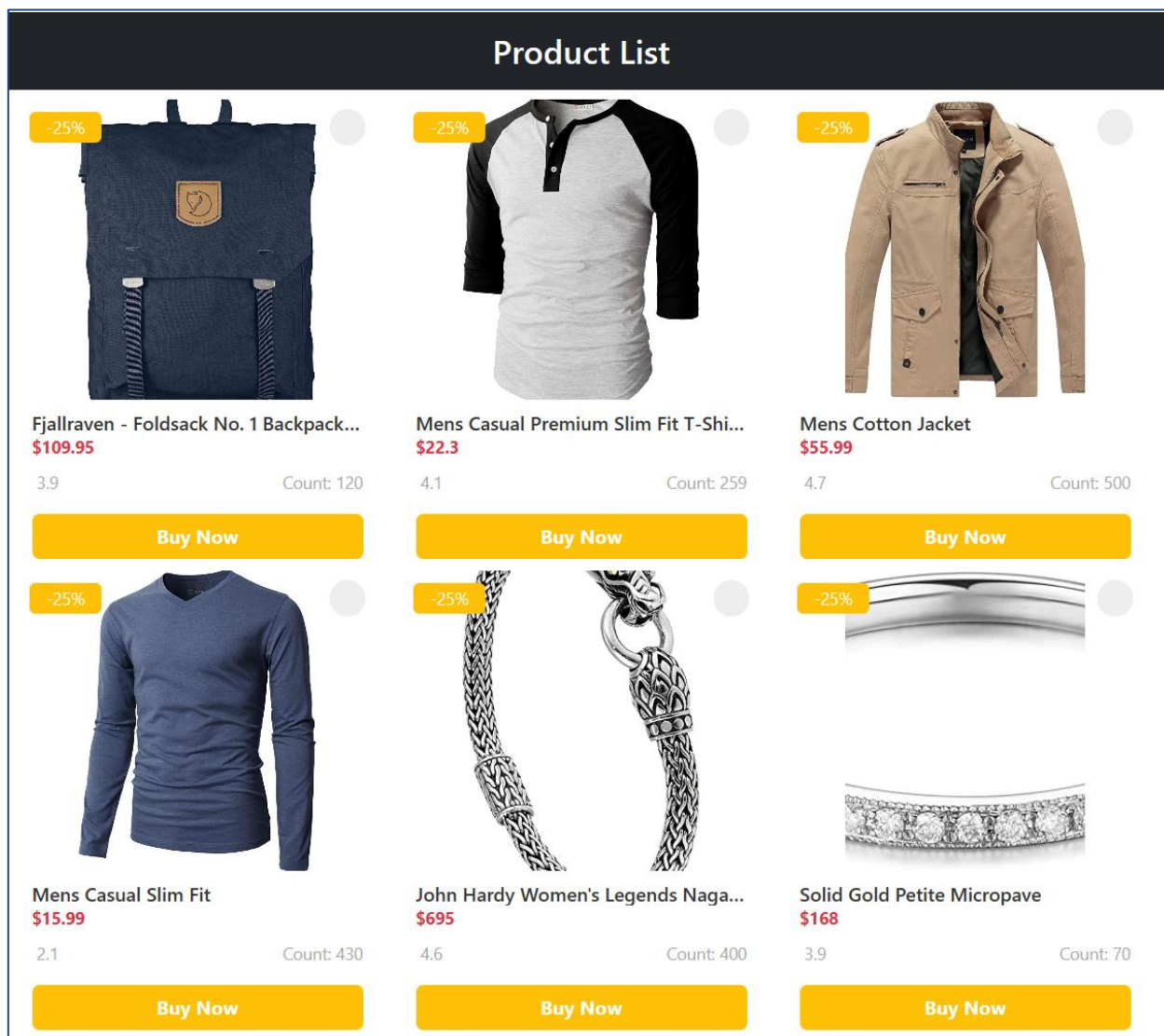
```
[
  {
    "context": ["/exchange"],
    "target": "https://www.dongabank.com.vn",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  },
  {
    "context": ["/products"],
    "target": "https://fakestoreapi.com",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  }
]
```

Chạy ứng dụng ta có kết quả như mong muốn.

Bài 27: Tương tác External Restful API- Product Service 2

Yêu cầu:

Tiếp tục cải tiến bài <https://fakestoreapi.com/products> ở trên. Cụ thể hiển thị ảnh giao diện kết quả như hình dưới đây:



Hướng dẫn chi tiết:

Ta thực hiện mọi bước giống như bài trước đã hướng dẫn. Chỉ cần bổ sung 2 công việc chính:

- Đầu tiên là cài đặt Bootstrap JQuery (xem lại bước 5 của bài trước)
- Sau đó hiệu chỉnh “fake-product.component.html”

```
<p>fake-product works!</p>
<p>{{errorMessage}}</p>
<h3 class="text-center text-white bg-dark p-3">Product List</h3>
<div class="container">
```

```
<div class="row">
  <div class="col-md-4" *ngFor="let p of data">
    <div class="card">
      <div class="img-container">
        <div class="d-flex justify-content-between align-items-center
p-2 first">
          <span class="percent">-25%</span>
          <span class="wishlist"><i class="bi bi-heart-
fill"></i></span>
        </div>
        <div class="text-center">
          
        </div>
      </div>
      <div class="product-detail-container">
        <div class="d-flex justify-content-between align-items-
center">
          <h6 class="mb-0 text-truncate">{{p.title}}</h6>
        </div>
        <div>
          <span class="text-danger fw-bold">${{p.price}}</span>
        </div>
        <div class="d-flex justify-content-between align-items-center
mt-2">
          <div class="ratings"><i class="bi bi-star-fill me-1"></i>
<span>{{p.rating.rate}}</span> </div>
          <div class="count">Count:
{{p.rating.count}}</div>
        </div>
        <div class="mt-3"> <button class="btn btn-warning d-block w-
100 text-white fw-bold">Buy Now</button> </div>
      </div>
    </div>
  </div>
</div>
```

File CSS “fake-product.component.css”:

```
@import
url("https://fonts.googleapis.com/css2?family=Poppins:wght@100;200;300;400;500;600;700;800&dis
play=swap");
body {
  background-color: #d6d2d2;
  font-family: "Poppins", sans-serif;
  font-weight: 300
}

.card {
  border: none;
  border-radius: 10px
}

.card:hover {
  box-shadow: 5px 6px 6px 2px #ccecf;
  /* transform: scale(1.1) */
}
```

```
.percent {
  padding: 5px;
  background-color: #ffc107;
  border-radius: 5px;
  color: #fff;
  font-size: 14px;
  height: 25px;
  width: 60px;
  display: flex;
  justify-content: center;
  align-items: center;
  cursor: pointer
}

.wishlist {
  height: 30px;
  width: 30px;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 50%;
  background-color: #eee;
  padding: 10px;
  cursor: pointer
}

.img-container {
  position: relative
}

.img-container .first {
  position: absolute;
  width: 100%
}

.img-container img {
  border-top-left-radius: 5px;
  border-top-right-radius: 5px
}

.img-scale {
  width: 200px;
  height: 250px;
  object-fit: cover
}

.product-detail-container {
  padding: 10px
}

.ratings i {
  color: #a9a6a6
}

.ratings span {
  color: #a9a6a6
}

.count {
  color: #a9a6a6;
}

label.radio {
  cursor: pointer
}

label.radio input {
  position: absolute;
  top: 0;
}
```

```
left: 0;
visibility: hidden;
pointer-events: none
}
label.radio span {
height: 25px;
width: 25px;
display: flex;
justify-content: center;
align-items: center;
border: 2px solid #dc3545;
color: #dc3545;
font-size: 10px;
border-radius: 50%;
text-transform: uppercase
}
label.radio input:checked+span {
border-color: #dc3545;
background-color: #dc3545;
color: #fff
}
```

Chạy ứng dụng ta có kết quả như mong muốn.

Bài 28 – Coindesk API- Bitcoin Price Index in Real-time

Yêu cầu:

Sử dụng AngularJS để hiển thị Bitcoin Price Index (BPI) thời gian thực theo public API sau:

<https://api.coindesk.com/v1/bpi/currentprice.json>

```
{
  "time": {
    "updated": "Mar 12, 2023 10:43:00 UTC",
    "updatedISO": "2023-03-12T10:43:00+00:00",
    "updateduk": "Mar 12, 2023 at 10:43 GMT"
  },
  "disclaimer": "This data was produced from the CoinDesk Bitcoin Price Index (USD). Non-USD currency data converted using hourly conversion rate from openexchangerates.org",
  "chartName": "Bitcoin",
  "bpi": {
    "USD": {
      "code": "USD",
      "symbol": "$",
      "rate": "20,520.4346",
      "description": "United States Dollar",
      "rate_float": 20520.4346
    },
    "GBP": {
      "code": "GBP",
      "symbol": "£",
      "rate": "17,146.7110",
      "description": "British Pound Sterling",
      "rate_float": 17146.711
    },
    "EUR": {
      "code": "EUR",
      "symbol": "€",
      "rate": "19,989.8993",
      "description": "Euro",
      "rate_float": 19989.8993
    }
  }
}
```

Hướng dẫn:

Bài 29 – Get List of Public APIs

Yêu cầu:

Viết chương trình hiển thị danh sách các public API ở link sau <https://api.publicapis.org/entries>, bố trí giao diện phù hợp.

Hướng dẫn:

Bài 30 - Predict the gender of a person based on their name - API.

Yêu cầu:

Thiết kế giao diện để sử dụng public API:

<https://api.genderize.io/?name=hoa>

API này nhận vào tên và dự đoán giới tính. Giao diện cần có ô nhập tên và nút kiểm tra giới tính.

Hướng dẫn:

Bài 31- Get US public data API

Yêu cầu:

Cho public US API:

<https://datausa.io/api/data?drilldowns=Nation&measures=Population>

Viết chương để hiển thị dữ liệu lên giao diện, yêu cầu tạo cấu trúc model và bố trí giao diện phù hợp.

Hướng dẫn:

Bài 32- Random dog images API

Yêu cầu:

Cho public API: <https://dog.ceo/api/breeds/image/random>

Nó sẽ hiển thị ngẫu nhiên dữ liệu là link hình của một chú Dog, hãy viết chương trình hiển thị hình ảnh ngẫu nhiên này lên giao diện (mỗi lần nhấn vào nút hiển thị thì gọi hình ảnh trong API ngẫu nhiên này).

```
{ "message": "https:\\\\images.dog.ceo\\breeds\\poodle-miniature\\n02113712_9682.jpg", "status": "success" }
```



Hướng dẫn:

Module 5 –Restful API với NodeJS và ExpressJS

Nội dung kiến thức thực hành:

- + Thực hành mô hình Model-View-Controller (MVC)
- + Cài đặt và sử dụng ExpressJS
- + Cấu hình Webserver
- + Tạo các Restful API: GET, POST, PUT, DELETE
- + Sử dụng Postman để kiểm thử Restful API

Bài 33– Giải thích chi tiết mô hình hoạt động của MVC

Yêu cầu:

Giải thích chi tiết mô hình hoạt động của MVC, ý nghĩa của từng thành phần.

Bài 34 – API và Restful API

Yêu cầu:

API là gì? Hãy giải thích ý nghĩa của Restful API

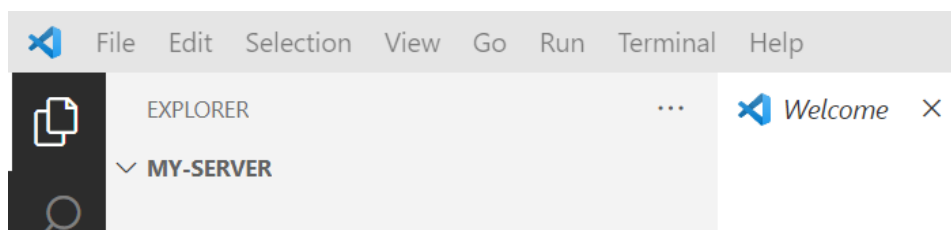
Bài 35 – Tạo và thực thi Project Restful API

Yêu cầu:

Tạo một Project tên “my-server” để chạy trên nền tảng Web Server, sử dụng NodeJS và ExpressJS. Minh họa default API xuất thông báo “Hello Restful API”

Hướng dẫn:

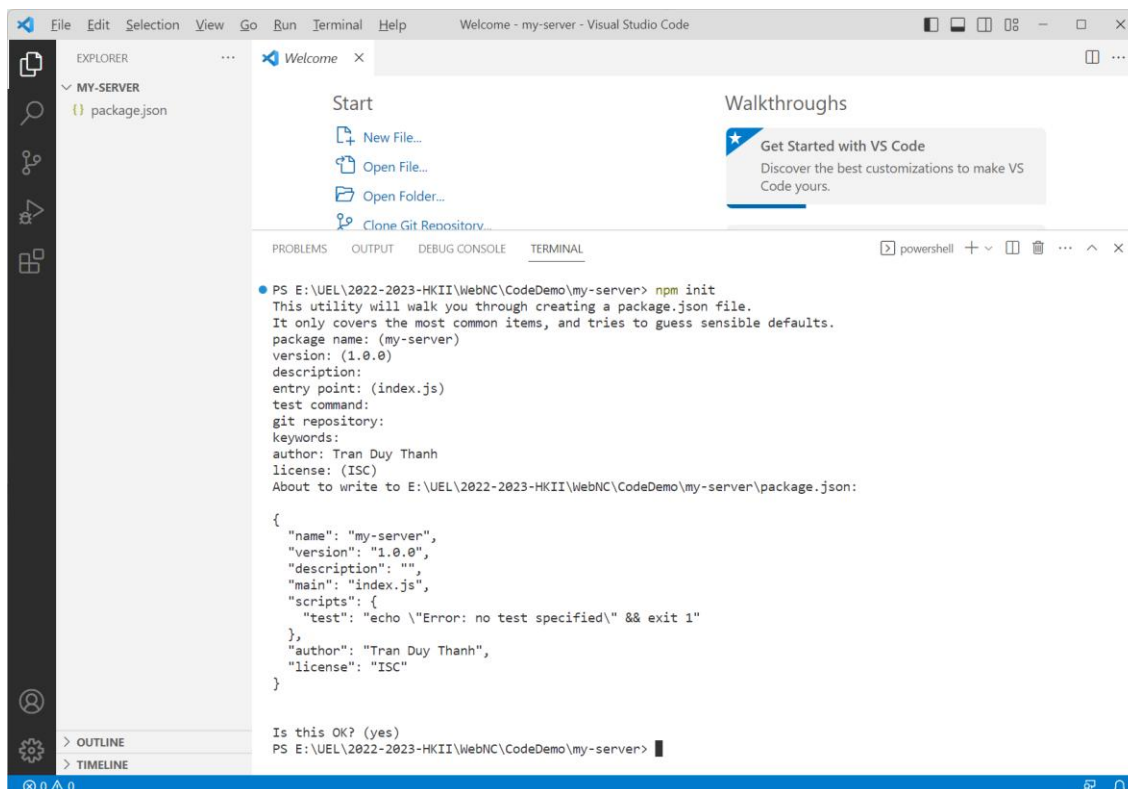
Bước 1: Tạo thư mục “my-server” và dùng Visual Studio Code để mở thư mục này:



Bước 2: Sau đó dùng terminal trong Visual Studio Code hoặc command line prompt để viết lệnh, hướng dẫn này sử dụng Terminal:

npm init

Minh họa quá trình chạy mã lệnh này:

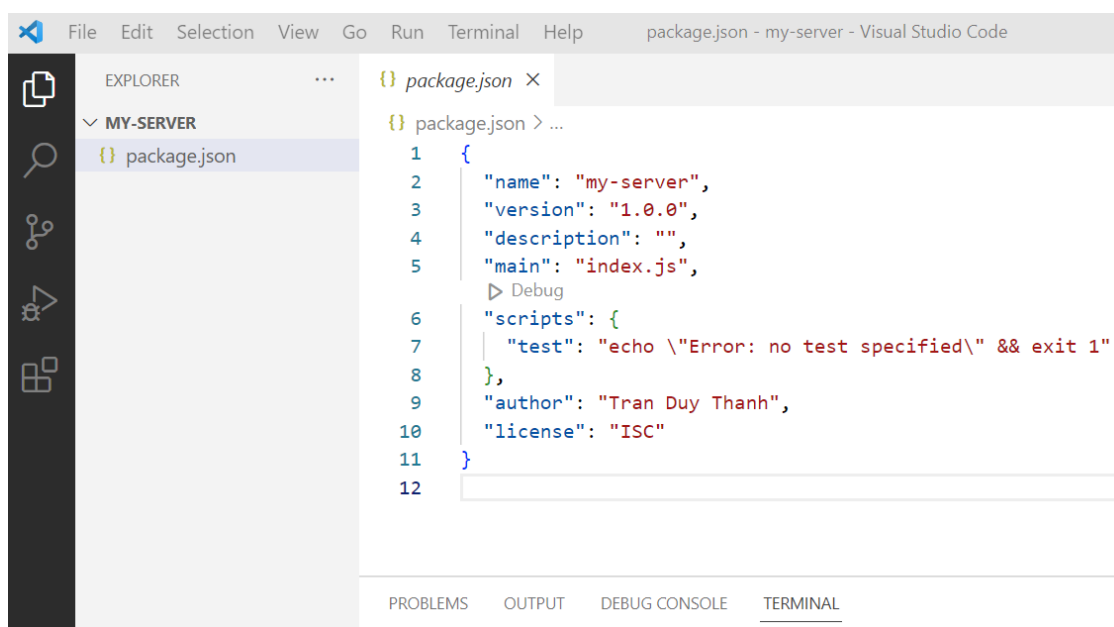


```
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.
package name: (my-server)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author: Tran Duy Thanh
license: (ISC)
About to write to E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server\package.json:

{
  "name": "my-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Tran Duy Thanh",
  "license": "ISC"
}

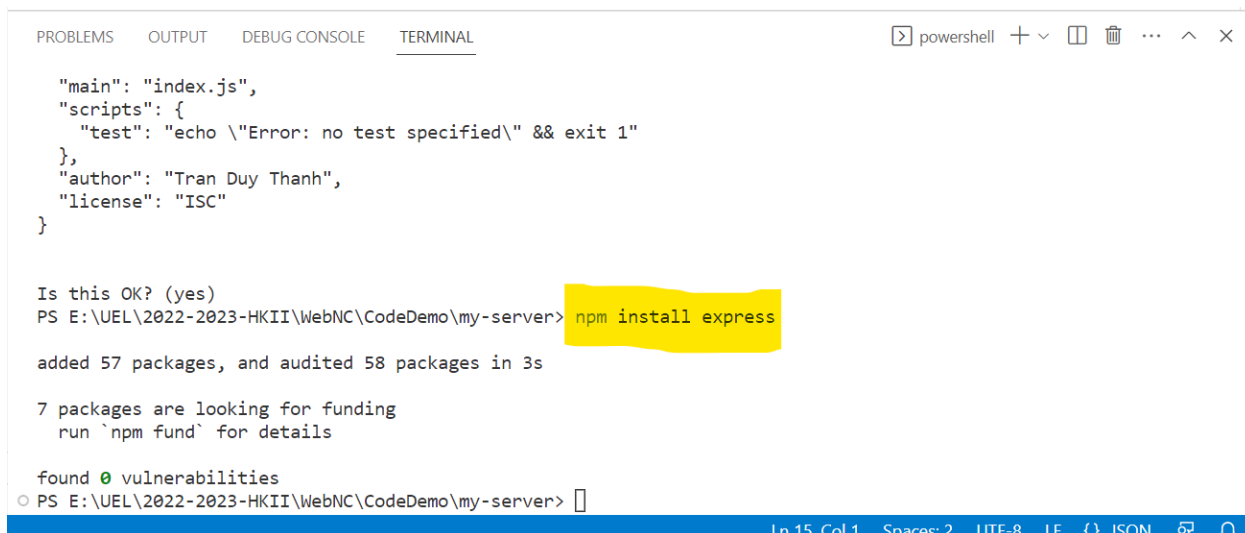
Is this OK? (yes)
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server>
```

Sau khi gõ npm init xong, thì cứ nhấn Enter liên tục (có thể cung cấp các thông tin ở từng dấu nhắc lệnh). Sau khi hoàn tất nó sẽ tạo ra một file **package.json**.



```
{
  "name": "my-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Tran Duy Thanh",
  "license": "ISC"
}
```

Bước 3: thực hiện lệnh: `npm install express` để cài đặt các thư viện ExpressJS



```

"main": "index.js",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"author": "Tran Duy Thanh",
"license": "ISC"
}

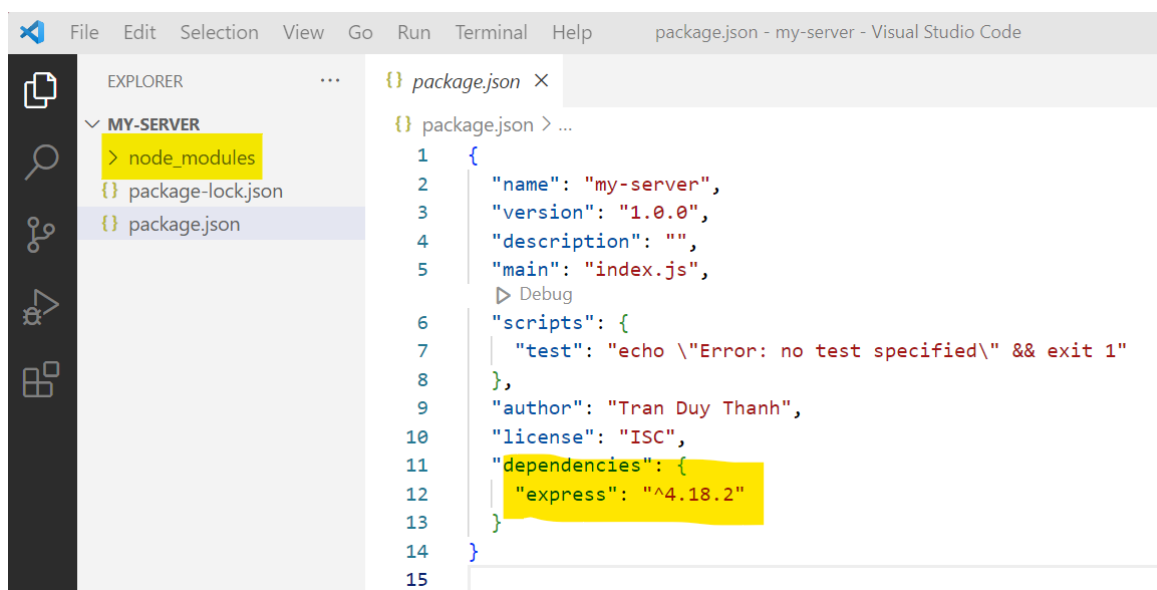
Is this OK? (yes)
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm install express

added 57 packages, and audited 58 packages in 3s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server>
  
```

Sau khi thực hiện xong, thư viện express được cài đặt vào Project:



Visual Studio Code Explorer shows the project structure:

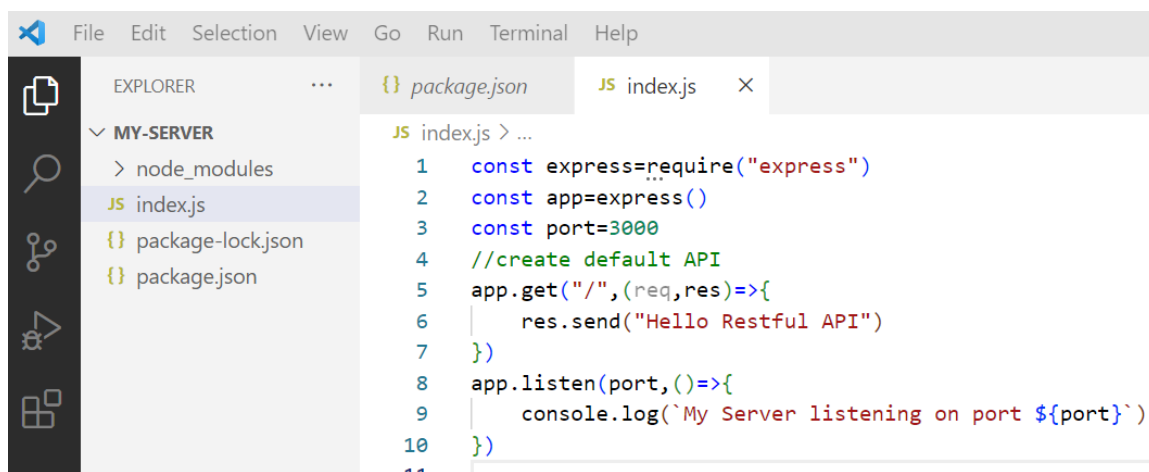
- MY-SERVER
 - node_modules
 - package-lock.json
 - package.json

The `package.json` file content is as follows:

```

{
  "name": "my-server",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Tran Duy Thanh",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2"
  }
}
  
```

Bước 4: Tạo một file “index.js” để tạo API mặc định như yêu cầu, coding như dưới đây:



Visual Studio Code Explorer shows the project structure:

- MY-SERVER
 - node_modules
 - index.js
 - package-lock.json
 - package.json

The `index.js` file content is as follows:

```

const express=require("express")
const app=express()
const port=3000
//create default API
app.get("/",(req,res)=>{
  res.send("Hello Restful API")
})
app.listen(port,()=>{
  console.log(`My Server listening on port ${port}`)
})
  
```

Mã lệnh ở trên chạy dịch vụ Web (Restful API) ở port 3000, với API mặc định là xuất dữ liệu “Hello Restful API” khi nó được triệu gọi.

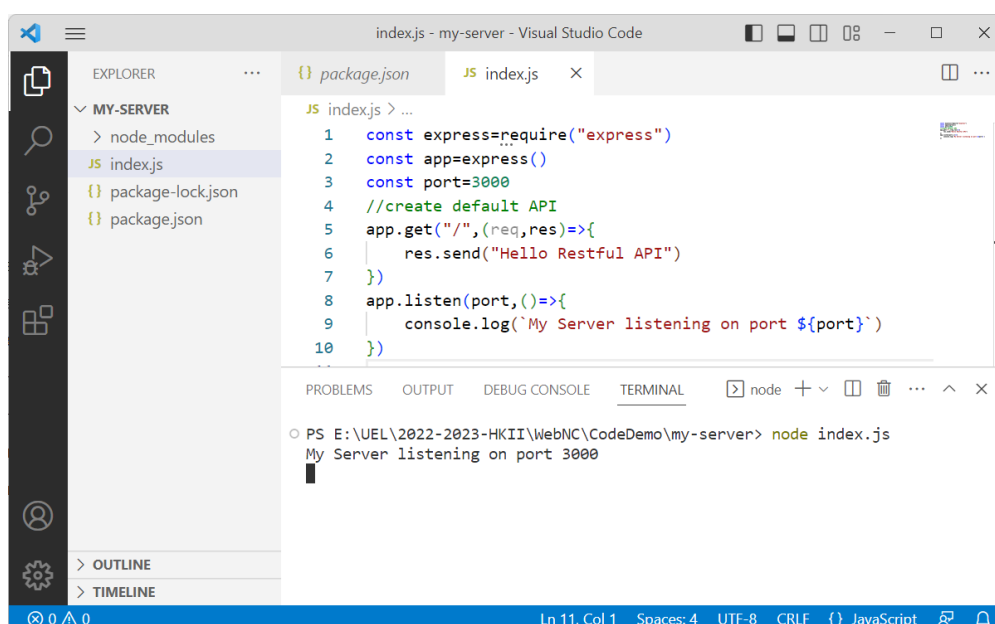
req(tên biến đại diện cho Request), là đối tượng để gửi các dữ liệu từ Client lên Server.

res(tên biến đại diện cho Response), là đối tượng để gửi các dữ liệu từ Server xuống Client.

Mã lệnh trên viết: **res.send**(“Hello Restful API”) tức là khi gọi mặc định dịch vụ web này thì nó trả về chuỗi “Hello Restful API”.

Bước 5: Khởi chạy Web Server:

Gọi lệnh **node index.js**



```

1  const express=require("express")
2  const app=express()
3  const port=3000
4  //create default API
5  app.get("/",(req,res)=>{
6    res.send("Hello Restful API")
7  })
8  app.listen(port,()=>{
9    console.log(`My Server listening on port ${port}`)
10 })

```

Terminal Output:

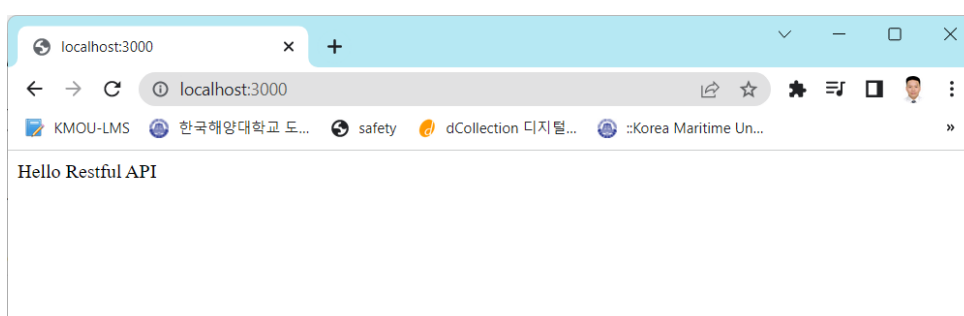
```

PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> node index.js
My Server listening on port 3000

```

Lúc này hàm listen sẽ được thực hiện, nếu Web Server được khởi tạo thành công thì lệnh ở dòng số 9 sẽ thực hiện, cụ thể ở đây là ta thấy thông báo “My Server listening on port 3000”.

Mở trình duyệt để xem kết quả:



Bài 36 – Cài đặt nodemon cho Web Server Project

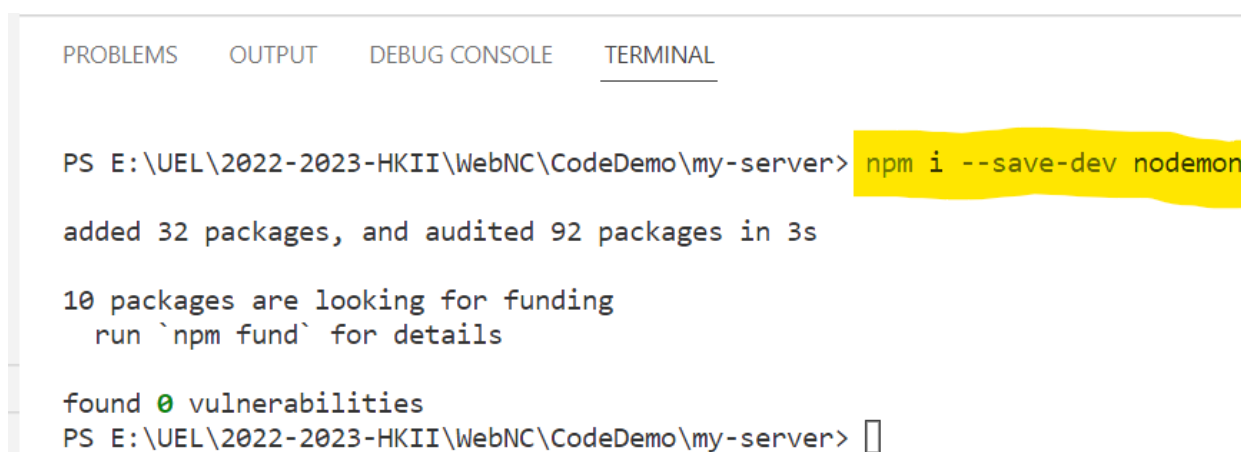
Yêu cầu:

Thông thường chúng ta phải thay đổi mã lệnh liên tục trong dự án, mỗi lần thay đổi thì phải chạy lại Web server cho dự án này, dẫn tới mất thời gian và không cần thiết. Kỹ thuật nodemon giúp restart lại server khi thay đổi mã lệnh trong dự án (phát âm **node-mon /nəʊd mən/**).

Hướng dẫn:

Bước 1: Thực hiện lệnh

npm i --save-dev nodemon



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm i --save-dev nodemon

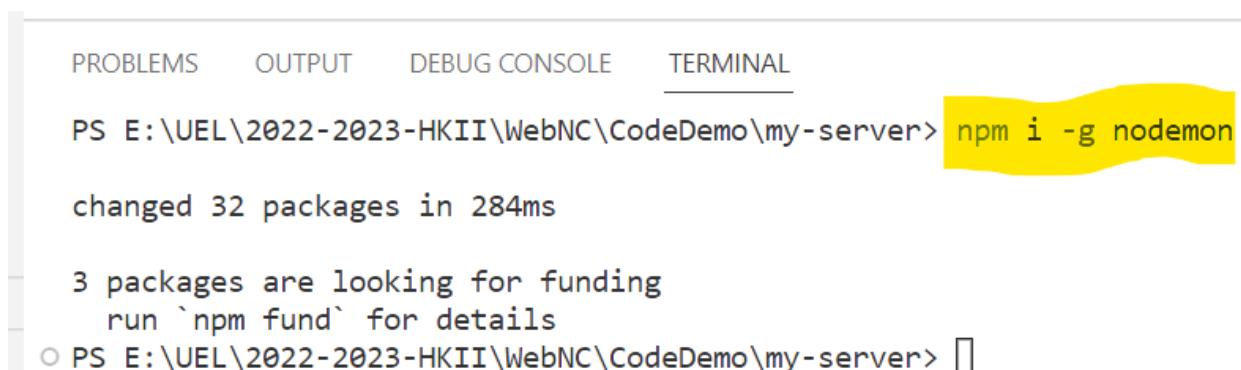
added 32 packages, and audited 92 packages in 3s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> 
```

Bước 2: Thực hiện lệnh

npm i -g nodemon



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm i -g nodemon

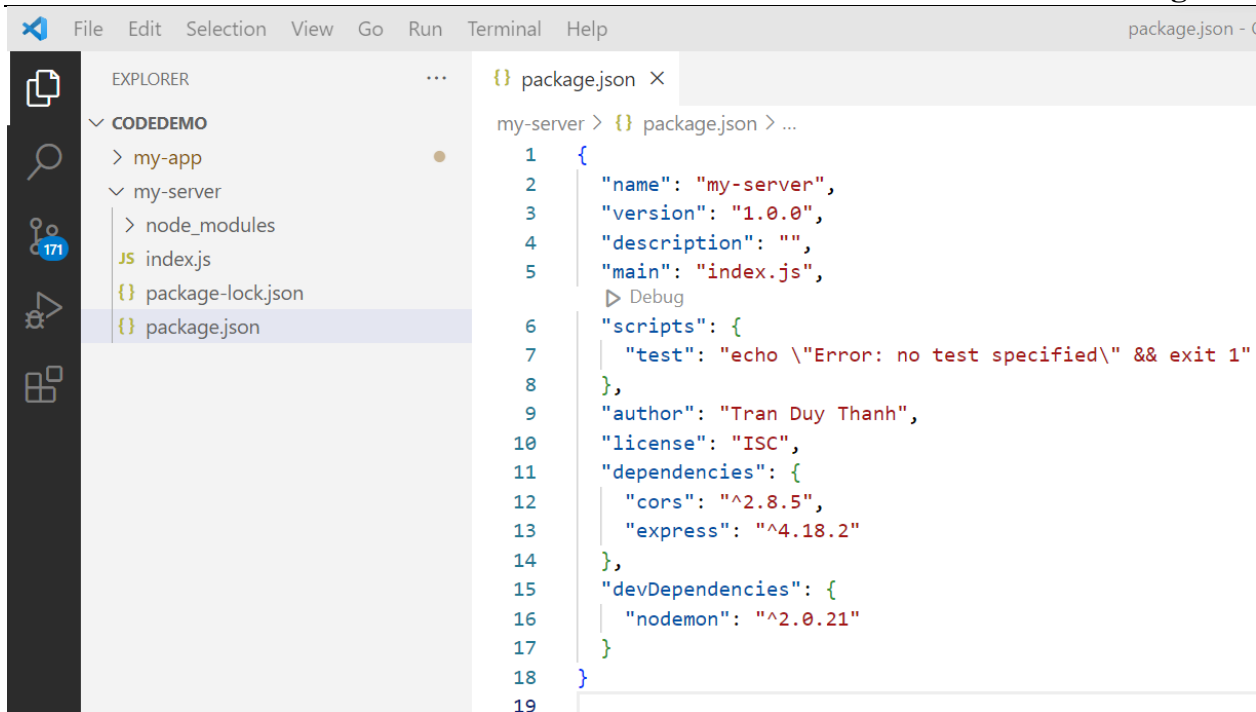
changed 32 packages in 284ms

3 packages are looking for funding
  run `npm fund` for details

PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> 
```

Bước 3: Chỉnh sửa “package.json”

Sau khi cài đặt xong các lệnh nodemon thì file “package.json” sẽ như hình chụp dưới đây:

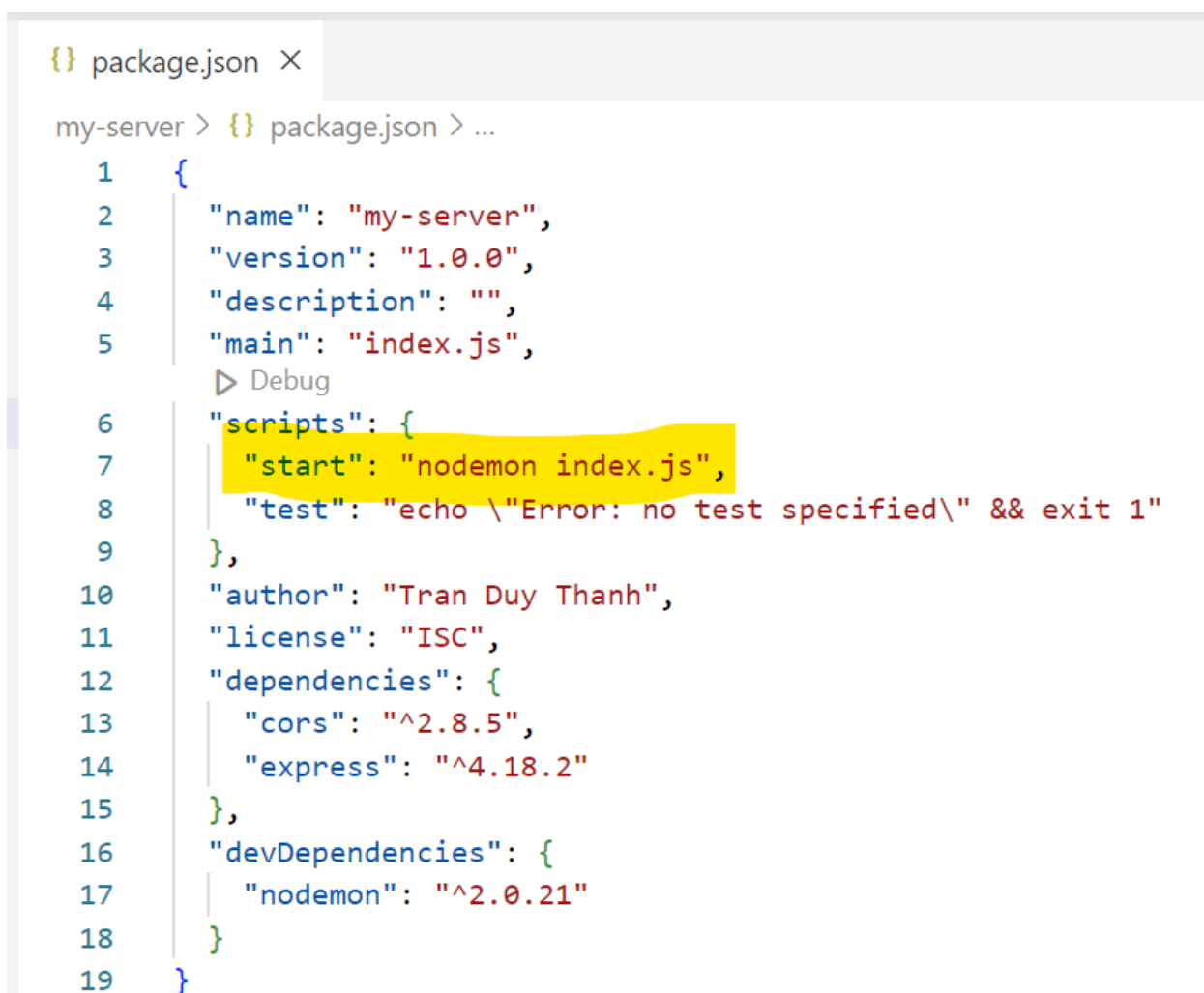


```

1  {
2    "name": "my-server",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "author": "Tran Duy Thanh",
10   "license": "ISC",
11   "dependencies": {
12     "cors": "^2.8.5",
13     "express": "^4.18.2"
14   },
15   "devDependencies": {
16     "nodemon": "^2.0.21"
17   }
18 }
19

```

Bổ sung lệnh “start”: “nodemon index.js” vào thẻ “scripts”



```

1  {
2    "name": "my-server",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "nodemon index.js",
8      "test": "echo \"Error: no test specified\" && exit 1"
9    },
10   "author": "Tran Duy Thanh",
11   "license": "ISC",
12   "dependencies": {
13     "cors": "^2.8.5",
14     "express": "^4.18.2"
15   },
16   "devDependencies": {
17     "nodemon": "^2.0.21"
18   }
19 }

```

Bước 4: Thực hiện lại lệnh chạy Web Server cho API này.

Thay vì viết

node index.js

thì ta viết:

npm start

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
○ [nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
My Server listening on port 3000
□
```

Như vậy Server đã được start ở port 3000 với cơ chế nodemon để tự động restart khi mã nguồn thay đổi, ví dụ nếu ta thay đổi bất kỳ mã nguồn:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
○ [nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
My Server listening on port 3000
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
My Server listening on port 3000
□
```

Bài 37 – Cài đặt morgan cho Web Server Project

Yêu cầu:

Ngoài ra ta có thể dùng lệnh morgan để xem http request logger

Hướng dẫn:

Bước 1: Thực hiện lệnh

npm i --save-dev morgan

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

● PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm i --save-dev morgan

added 5 packages, and audited 97 packages in 4s

10 packages are looking for funding
  run `npm fund` for details

○ found 0 vulnerabilities
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> 
```

File “**package.json**” sẽ được cập nhật:

```
{ } package.json X  JS index.js

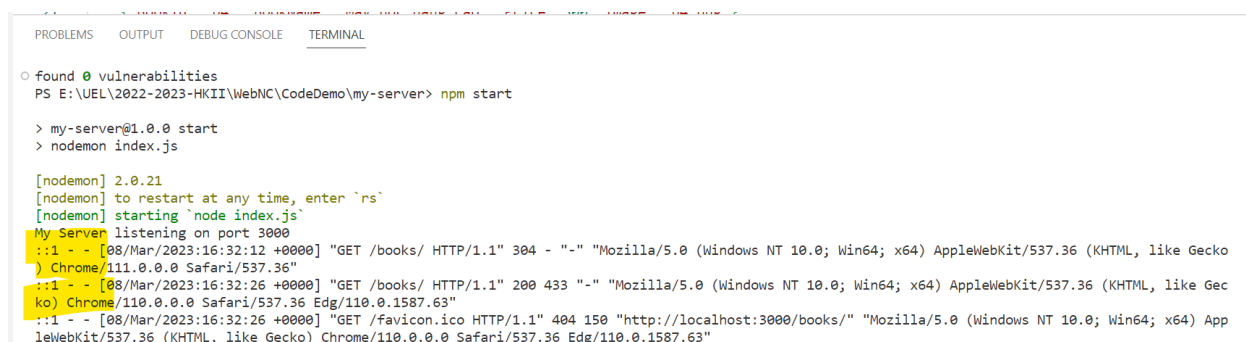
my-server > { } package.json > ...
1  {
2    "name": "my-server",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "nodemon index.js",
8      "test": "echo \"Error: no test specified\" && exit 1"
9    },
10   "author": "Tran Duy Thanh",
11   "license": "ISC",
12   "dependencies": {
13     "cors": "^2.8.5",
14     "express": "^4.18.2"
15   },
16   "devDependencies": {
17     "morgan": "^1.10.0",
18     "nodemon": "^2.0.21"
19   }
20 }
```

Bước 2: Chỉnh sửa index.js để sử dụng morgan

```
const express=require("express")
const app=express()
const port=3000
const morgan=require("morgan")
app.use(morgan("combined"))
//create default API
```

```
app.get("/", (req, res) => {
  res.send("Hello Restful API")
})
app.listen(port, () => {
  console.log(`My Server listening on port ${port}`)
})
```

Bước 3: Khởi chạy Server và ta có kết quả http request logger như hình bên dưới:



```
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm start

> my-server@1.0.0 start
> nodemon index.js

[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] starting `node index.js`
My Server listening on port 3000
::1 - - [08/Mar/2023:16:32:12 +0000] "GET /books/ HTTP/1.1" 304 - "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/111.0.0.0 Safari/537.36"
::1 - - [08/Mar/2023:16:32:26 +0000] "GET /books/ HTTP/1.1" 200 433 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36 Edg/110.0.1587.63"
::1 - - [08/Mar/2023:16:32:26 +0000] "GET /favicon.ico HTTP/1.1" 404 150 "http://localhost:3000/books/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/110.0.0.0 Safari/537.36 Edg/110.0.1587.63"
```

Bài 38 – Tạo HTTP GET – List of Book

Yêu cầu:

Trong file index.js, tiếp tục tạo một danh sách Sách, viết API để lấy danh sách này.

Hướng dẫn:

Giả sử trong file index.js ta bổ sung khai báo danh sách Sách như dưới đây:

```
let database=[
  {"BookId":"b1","BookName":"Kỹ thuật lập trình cơ bản",
  "Price":70,"Image":"b1.png"},
  {"BookId":"b2","BookName":"Kỹ thuật lập trình nâng cao",
  "Price":100,"Image":"b2.png"},
  {"BookId":"b3","BookName":"Máy học cơ bản","Price":200,"Image":"b3.png"},
  {"BookId":"b4","BookName":"Máy học nâng cao","Price":300,"Image":"b4.png"},
  {"BookId":"b5","BookName":"Lập trình Robot cơ bản","Price":250,"Image":"b5.png"},
]
```

Để tạo API lấy danh sách Sách thì thường ta khai báo pattern “/books” như sau:

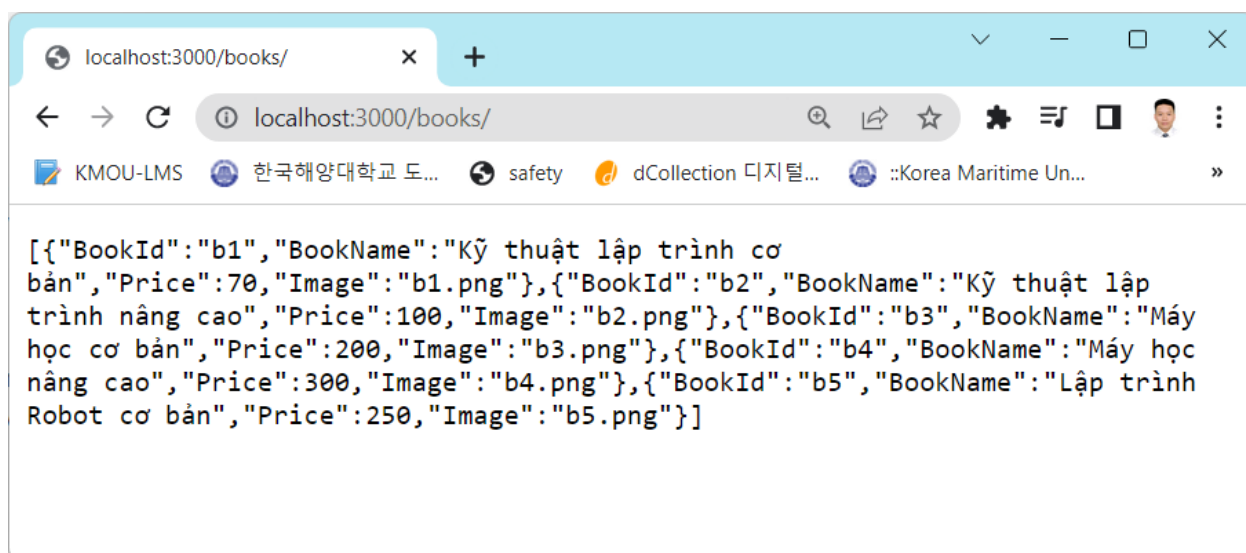
```
app.get("/books", (req, res) => {
  res.send(database)
})
```

Mã lệnh đầy đủ của chương trình (index.js) sau khi bổ sung API này là:

```
const express=require("express")
const app=express()
const port=3000
const morgan=require("morgan")
app.use(morgan("combined"))

app.get("/",(req,res)=>{
    res.send("Hello Restful API")
})
app.listen(port,()=>{
    console.log(`My Server listening on port ${port}`)
})
let database=[
    {"BookId":"b1","BookName":"Kỹ thuật lập trình cơ bản","Price":70,"Image":"b1.png"},
    {"BookId":"b2","BookName":"Kỹ thuật lập trình nâng cao","Price":100,"Image":"b2.png"},
    {"BookId":"b3","BookName":"Máy học cơ bản","Price":200,"Image":"b3.png"},
    {"BookId":"b4","BookName":"Máy học nâng cao","Price":300,"Image":"b4.png"},
    {"BookId":"b5","BookName":"Lập trình Robot cơ bản","Price":250,"Image":"b5.png"},
]
app.get("/books",(req,res)=>{
    res.send(database)
})
```

Tiến hành chạy lại lệnh: **node index.js** (hoặc nếu trước đó đã chạy **npm start** rồi thì hệ thống tự động restart, ta không phải chạy lại) và triệu gọi api books ta sẽ có kết quả:

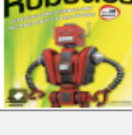


Bài 39 - Triệu gọi HTTP GET – List of Book

Yêu cầu:

Từ API **books** được tạo ở trên. Hãy viết chương trình triệu gọi API này trong AngularJS.

Coi như trong assets/book_images có 5 hình có tên như trong Server trả về. giao diện sau khi triệu gọi xong API books:

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	

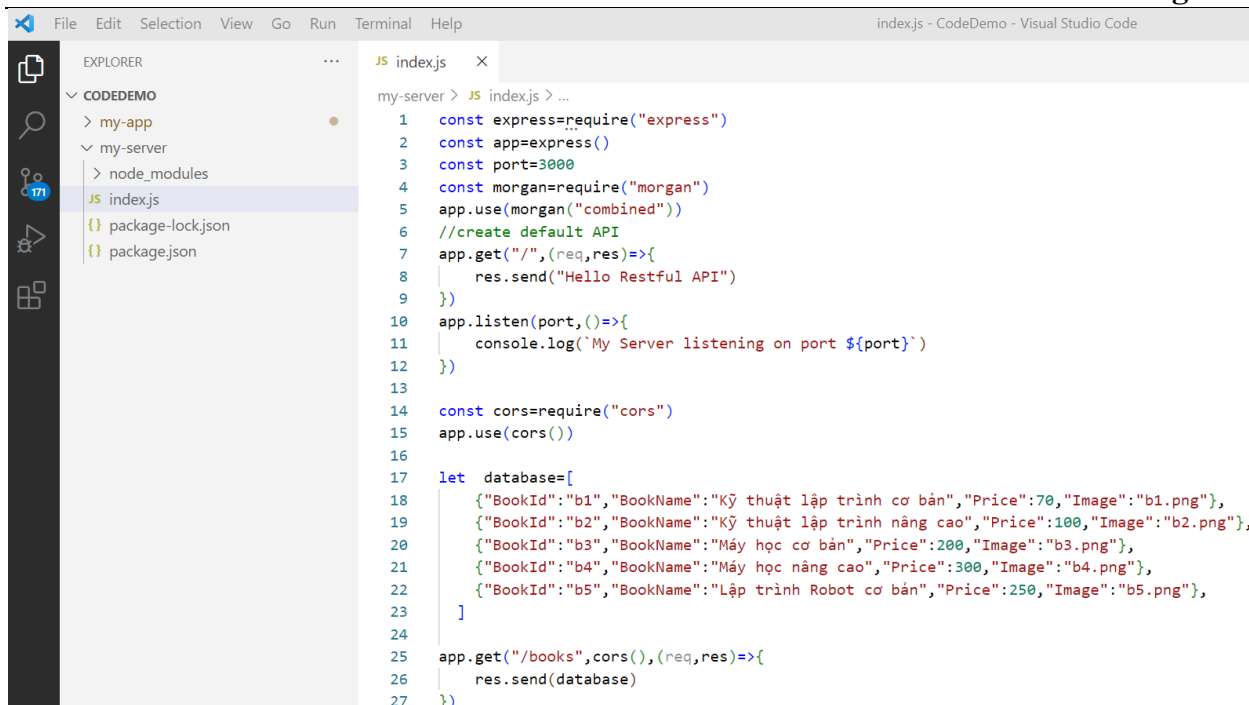
Hướng dẫn:

*Trong project “**my-server**”. Thông thường ta sẽ gặp báo lỗi CORS Policy, nên trên Server ta bổ sung các lệnh sau:

Bước 1: cài đặt lệnh cors(Cross-Origin Resource Sharing) cho Web server có chứa api books.

npm i cors

Bước 2: Bổ sung thêm lệnh liên quan tới cors, mã lệnh cuối cùng của index.js:



```

1  const express=require("express")
2  const app=express()
3  const port=3000
4  const morgan=require("morgan")
5  app.use(morgan("combined"))
6  //create default API
7  app.get("/",(req,res)=>{
8    res.send("Hello Restful API")
9  })
10 app.listen(port,()=>{
11   console.log(`My Server listening on port ${port}`)
12 })
13
14 const cors=require("cors")
15 app.use(cors())
16
17 let database=[
18   {"BookId":"b1","BookName":"Kỹ thuật lập trình cơ bản","Price":70,"Image":"b1.png"},
19   {"BookId":"b2","BookName":"Kỹ thuật lập trình nâng cao","Price":100,"Image":"b2.png"},
20   {"BookId":"b3","BookName":"Máy học cơ bản","Price":200,"Image":"b3.png"},
21   {"BookId":"b4","BookName":"Máy học nâng cao","Price":300,"Image":"b4.png"},
22   {"BookId":"b5","BookName":"Lập trình Robot cơ bản","Price":250,"Image":"b5.png"},
23 ]
24
25 app.get("/books",cors(),(req,res)=>{
26   res.send(database)
27 })
  
```

Dòng lệnh 14, 15 khai báo thư viện cors và enable cors

Dòng lệnh 25 bổ sung thêm cors() cho api books

*Trong project “my-app”:

Bước 1: Tạo interface Ibook trong file Book.ts

```

export interface IBook{
  BookId:string,
  BookName:string,
  Price:number,
  Image:string
}
  
```

Bước 2: Tạo một service mang tên “BookAPI”, trong file “book-api.service.ts” viết các lệnh sau:

```

import { HttpClient, HttpResponse, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, map, Observable, retry, throwError } from 'rxjs';
import { IBook } from '../interfaces/Book';

@Injectable({
  providedIn: 'root'
})
export class BookAPIService {

  constructor(private _http: HttpClient) { }

  getBooks():Observable<any>
  
```

```
{
  const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
  const requestOptions:Object={
    headers:headers,
    responseType:"text"
  }
  return this._http.get<any>("/books",requestOptions).pipe(
    map(res=>JSON.parse(res) as Array<IBook>),
    retry(3),
    catchError(this.handleError))
}

handleError(error:HttpErrorResponse){
  return throwError(()=>new Error(error.message))
}
}
```

Bước 3: Cấu hình “proxy.conf.json”, tiếp tục bổ sung vào cuối file:

```
[
  {
    "context": ["/exchange"],
    "target": "https://www.dongabank.com.vn",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  },
  {
    "context": ["/products"],
    "target": "https://fakestoreapi.com",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  },
  {
    "context": ["/books"],
    "target": "http://localhost:3000",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  }
]
```

Bước 4: Tạo một component “Books”.

Trong file “books.component.ts” bổ sung lệnh:

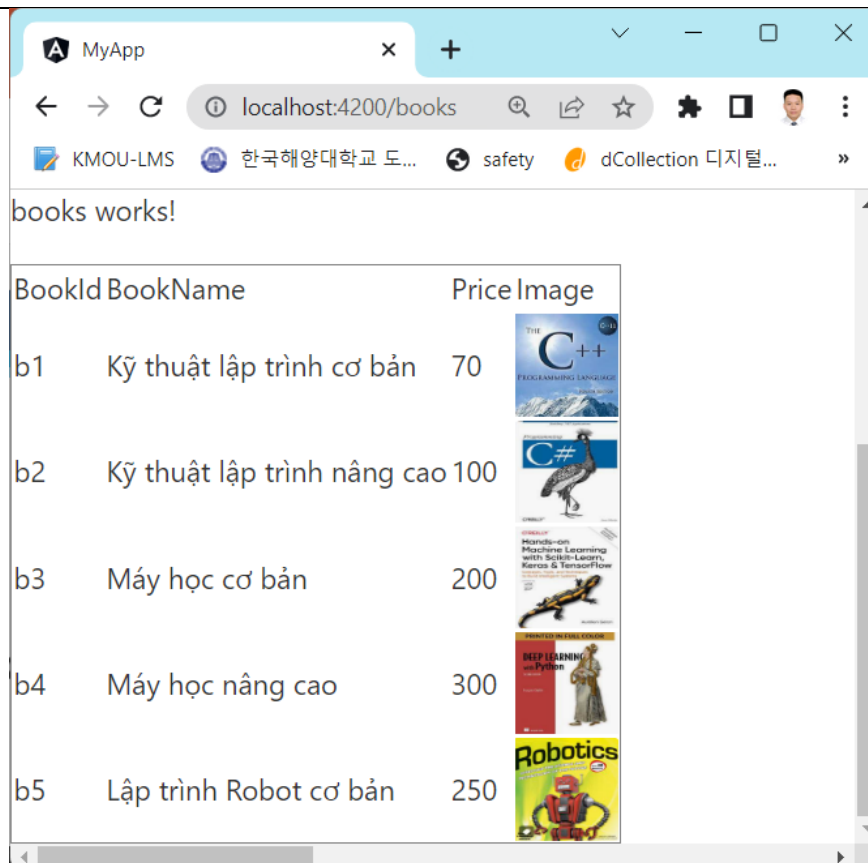
```
import { Component } from '@angular/core';
import { BookAPIService } from '../book-api.service';

@Component({
  selector: 'app-books',
  templateUrl: './books.component.html',
  styleUrls: ['./books.component.css']
})
export class BooksComponent {
  books:any;
  errMessage:string=''
  constructor(private _service: BookAPIService){
    this._service.getBooks().subscribe({
      next:(data)=>{this.books=data},
      error:(err)=>{this.errMessage=err}
    })
  }
}
```

Trong file “books.component.html” bổ sung lệnh:

```
<p>books works!</p>
{{errMessage}}
<table border="1">
  <tr>
    <td>BookId</td>
    <td>BookName</td>
    <td>Price</td>
    <td>Image</td>
  </tr>
  <tbody>
    <tr *ngFor="let book of books">
      <td>{{book.BookId}}</td>
      <td>{{book.BookName}}</td>
      <td>{{book.Price}} </td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>
```

Bước 5: Khai báo Routing hoặc tham chiếu trực tiếp trong “app.component.html”, chạy ứng dụng “my-app” ta có kết quả như đề bài:



Như vậy dữ liệu trong books API được xây dựng trong “my-server” đã được nạp lên front end Angularjs “my-app” thành công.

Bài 40 – Tạo HTTP GET – a Book

Yêu cầu:

Xây dựng API lấy chi tiết thông tin một Book khi biết BookId. Ví dụ

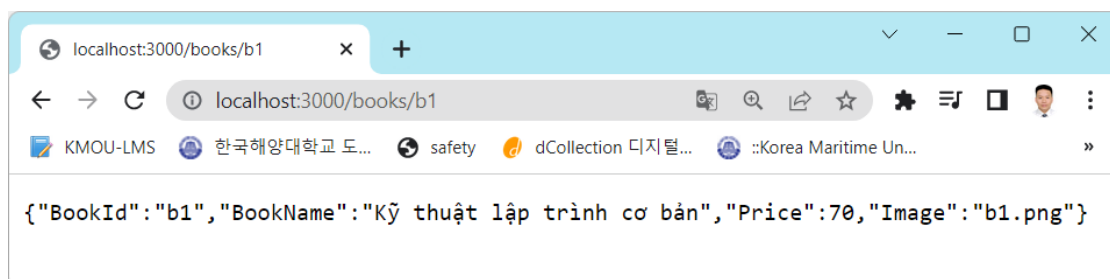
<http://localhost:3000/books/b1>

sẽ trả về thông tin chi tiết của Book có BookId=b1

<http://localhost:3000/books/b2>

sẽ trả về thông tin chi tiết của Book có BookId=b2

ví dụ khi nhập b1, thì JsonObject của Book sẽ cho ra kết quả:



Hướng dẫn:

Trong “index.js” bổ sung lệnh sau vào cuối tập tin:

```
app.get("/books/:id", cors(), (req, res) => {  
  id = req.params["id"]  
  let p = database.find(x => x.BookId == id)  
  res.send(p)  
})
```

Mã lệnh đầy đủ:

```
const express = require("express")  
const app = express()  
const port = 3000  
const morgan = require("morgan")  
app.use(morgan("combined"))  
//create default API  
app.get("/", (req, res) => {  
  res.send("Hello Restful API")  
})  
app.listen(port, () => {  
  console.log(`My Server listening on port ${port}`)  
})  
  
const cors = require("cors")  
app.use(cors())  
  
let database = [  
  { "BookId": "b1", "BookName": "Kỹ thuật lập trình cơ bản",  
    "Price": 70, "Image": "b1.png" },  
  { "BookId": "b2", "BookName": "Kỹ thuật lập trình nâng cao",  
    "Price": 100, "Image": "b2.png" },  
  { "BookId": "b3", "BookName": "Máy học cơ bản", "Price": 200, "Image": "b3.png" },  
  { "BookId": "b4", "BookName": "Máy học nâng cao", "Price": 300, "Image": "b4.png" },  
  { "BookId": "b5", "BookName": "Lập trình Robot cơ bản",  
    "Price": 250, "Image": "b5.png" },  
]  
  
app.get("/books", cors(), (req, res) => {  
  res.send(database)  
})  
app.get("/books/:id", cors(), (req, res) => {  
  id = req.params["id"]  
  let p = database.find(x => x.BookId == id)  
  res.send(p)  
})
```

Lưu ý cú pháp:

"/books/:id"

Bài 41– Triệu gọi HTTP GET – a Book

Yêu cầu:

Xây dựng giao diện bên AngularJS, cho phép nhập vào BookId để tìm kiếm thông tin chi tiết của Book này bằng cách triệu gọi API Get a Book ở bài trước.



Hướng dẫn:

Bước 1: Tiếp tục mở lại file “book-api.service.ts” để hiệu chỉnh mã nguồn cho lớp “BookAPIService” service.

```
getBook(bookId:string):Observable<any>
{
  const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
  const requestOptions:Object={
    headers:headers,
    responseType:"text"
  }
  return this._http.get<any>("/books/"+bookId,requestOptions).pipe(
    map(res=>JSON.parse(res) as IBook),
    retry(3),
    catchError(this.handleError))
}
```

Mã lệnh đầy đủ của Service này sau khi cập nhật hàm getBook(bookId:string)

```
import { HttpClient, HttpResponse, HttpHeaders } from
'@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, map, Observable, retry, throwError } from 'rxjs';
```

```
import { IBook } from './interfaces/Book';

@Injectable({
  providedIn: 'root'
})
export class BookAPIService {

  constructor(private _http: HttpClient) { }

  getBooks():Observable<any>
  {
    const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
    const requestOptions:Object={
      headers:headers,
      responseType:"text"
    }
    return this._http.get<any>("/books",requestOptions).pipe(
      map(res=>JSON.parse(res) as Array<IBook>),
      retry(3),
      catchError(this.handleError))
  }

  handleError(error:HttpErrorResponse){
    return throwError(()=>new Error(error.message))
  }

  getBook(bookId:string):Observable<any>
  {
    const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
    const requestOptions:Object={
      headers:headers,
      responseType:"text"
    }
    return this._http.get<any>("/books/"+bookId,requestOptions).pipe(
      map(res=>JSON.parse(res) as IBook),
      retry(3),
      catchError(this.handleError))
  }
}
```

Bước 2: Tạo component “BookDetail” để hiển thị thông tin chi tiết của Book

File “book-detail.component.ts” bổ sung mã lệnh:

```
import { Component } from '@angular/core';
import { BookAPIService } from '../book-api.service';

@Component({
  selector: 'app-book-detail',
  templateUrl: './book-detail.component.html',
  styleUrls: ['./book-detail.component.css']
})
export class BookDetailComponent {
  book:any;
  errMessage:string=''
  constructor(private _service: BookAPIService){
  }
  searchBook(bookId:string)
  {
    this._service.getBook(bookId).subscribe({
      next:(data)=>{this.book=data},
      error:(err)=>{this.errMessage=err}
    })
  }
}
```

File “book-detail.component.html” bổ sung mã lệnh:

```
<p>book-detail works!</p>
<p>{{errMessage}}</p>
Book Id: <input type="text" #bookId> <button
(click)="searchBook(bookId.value)">Search</button>
<table>
  <tr>
    <td>Book Id:</td>
    <td>{{book.BookId}}</td>
  </tr>
  <tr>
    <td>Book Name:</td>
    <td>{{book.BookName}}</td>
  </tr>
  <tr>
    <td>Price:</td>
    <td>{{book.Price}} </td>
  </tr>
  <tr>
    <td colspan="2">
      
    </td>
  </tr>
</table>
```

Khai báo routing hoặc tham chiếu trực tiếp component này trong
“app.component.html” để test. Chạy ứng dụng ta có kết quả như yêu cầu.

Bài 42 – Tạo HTTP POST – Create a Book

Yêu cầu:

Bổ sung thêm API cho phép thêm mới một Book.

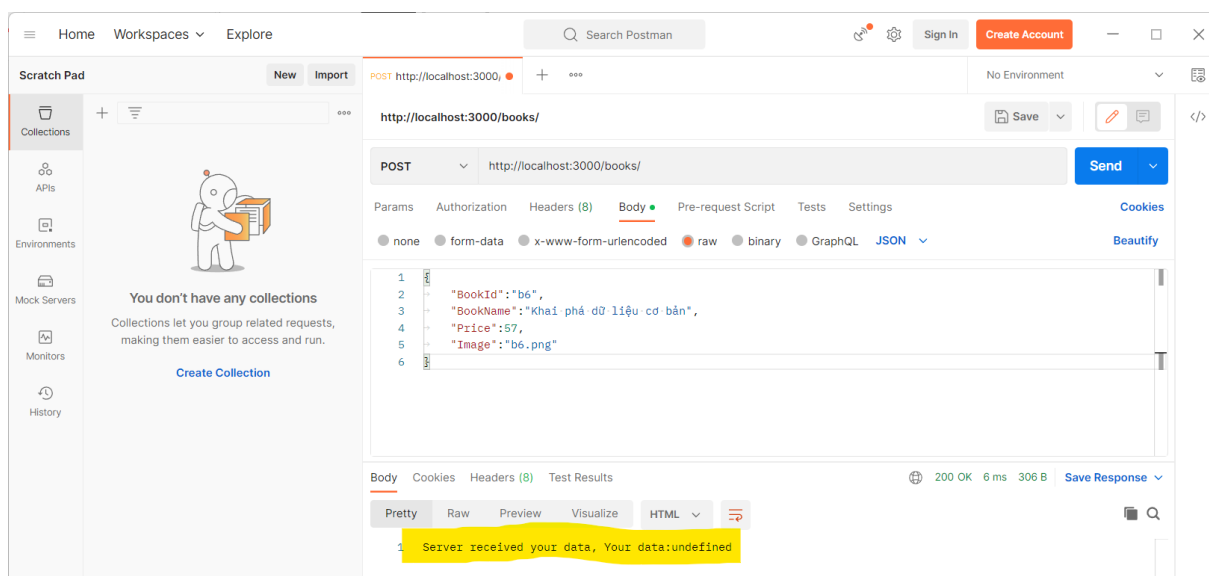
Hướng dẫn:

Bước 1: trong dự án “my-server”, tiếp tục bổ sung mã lệnh sau vào cuối tập tin “index.js”

```
app.post("/books", cors(), (req, res) => {
  console.log(req.body)
  res.send("Server received your data, Your data:" + req.body)
})
```

Bước 2: Cài đặt Postman và sử dụng để test các method: POST, PUT, DELETE (vì chỉ có GET mới test được trên trình duyệt). Lưu ý rằng nếu Postman test thành công thì xem như các lệnh API ở “my-server” đã viết đúng. Công cụ này rất tiện lợi, nó giúp ta kiểm thử API trước khi sử dụng. Ngoài ra còn có công cụ Swagger(<https://swagger.io/>). Trong nội dung của môn học này chúng ta sẽ sử dụng Postman. Tải ở đây: <https://www.postman.com/downloads/>

Test thử nghiệm api books với method POST, cấu hình như bên dưới, nhấn send.



Rõ ràng thấy đã triệu gọi API books (method POST) thành công nhưng body chưa lấy được (undefined).

Bước 3: Cài đặt body-parser cho “my-server”

npm i body-parser

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
● PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> npm i body-parser

added 2 packages, changed 2 packages, and audited 99 packages in 1s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server> █
```

sau khi cài thư viện xong thì tiến hành chỉnh sửa **index.js**:

```
const bodyParser=require("body-parser")
app.use(bodyParser.json())
```

Bước 4: Chỉnh sửa lại API post a book

```
app.post("/books",cors(),(req,res)=>{
  //put json book into database
  database.push(req.body);
  //send message to client(send all database to client)
  res.send(database)
})
```

Với lệnh ở trên, mỗi lần POST 1 Book lên server nó sẽ lưu vào database. Sau đó ta có thể gọi GET books để xem lại toàn bộ dữ liệu mới nhất. Đây là minh họa thao tác thêm mới 1 đối tượng lên Server, nó sẽ bị mất khi restart server. Do đó ta thường phải lưu vào Cơ sở dữ liệu là vậy, chương trình học sẽ dùng MongoDB để lưu trữ, sẽ được hướng dẫn ở các module sau.

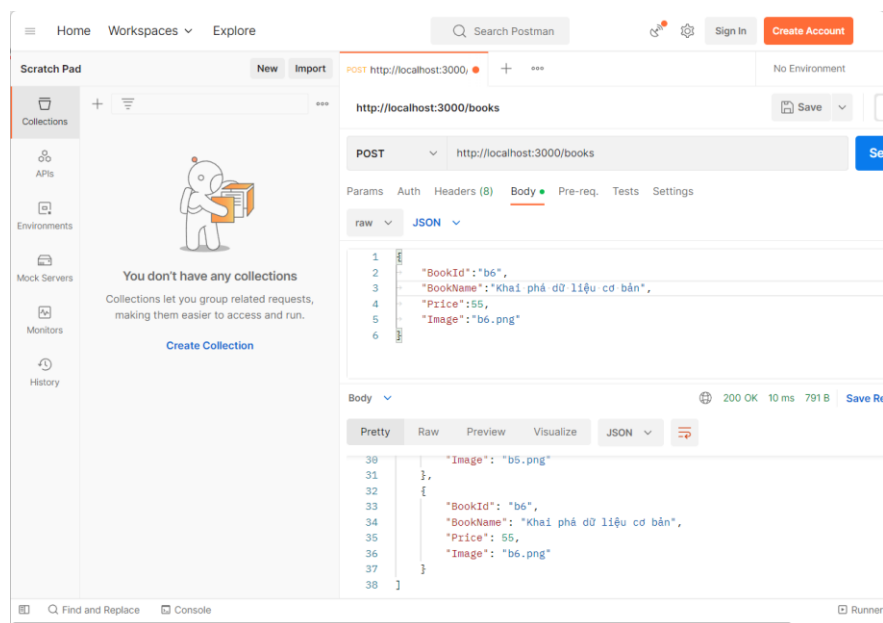
Mã lệnh đầy đủ của index.js sau khi bổ sung API POST 1 Book:

```
const express=require("express")
const app=express()
const port=3000
const morgan=require("morgan")
app.use(morgan("combined"))
const bodyParser=require("body-parser")
app.use(bodyParser.json())
//create default API
app.get("/",(req,res)=>{
  res.send("Hello Restful API")
})
app.listen(port,()=>{
  console.log(`My Server listening on port ${port}`)
})

const cors=require("cors")
app.use(cors())
```

```
let database=[
  {"BookId":"b1","BookName":"Kỹ thuật lập trình cơ
bản","Price":70,"Image":"b1.png"},
  {"BookId":"b2","BookName":"Kỹ thuật lập trình nâng
cao","Price":100,"Image":"b2.png"},
  {"BookId":"b3","BookName":"Máy học cơ bản","Price":200,"Image":"b3.png"},
  {"BookId":"b4","BookName":"Máy học nâng cao","Price":300,"Image":"b4.png"},
  {"BookId":"b5","BookName":"Lập trình Robot cơ bản","Price":250,"Image":"b5.png"},
]
app.get("/books",cors(),(req,res)=>{
  res.send(database)
})
app.get("/books/:id",cors(),(req,res)=>{
  id=req.params["id"]
  let p=database.find(x=>x.BookId==id)
  res.send(p)
})
app.post("/books",cors(),(req,res)=>{
  //put json book into database
  database.push(req.body);
  //send message to client(send all database to client)
  res.send(database)
})
```

Bước 5: thử nghiệm lại bằng Postman



Chọn method là POST, thẻ body chọn raw/json ta nhập dữ liệu có cấu trúc:

```
{
  "BookId":"b6",
  "BookName":"Khai phá dữ liệu cơ bản",
  "Price":55,
  "Image":"b6.png"
}
```

Sau đó bấm nút “Send”, nó sẽ triệu gọi API Post a Book, nếu thành công nó sẽ đồng thời trả về toàn bộ dữ liệu mới nhất (cho chúng ta test), dĩ nhiên ta chỉ cần trả về true/false là được rồi. Trong ví dụ này tài liệu trả về toàn bộ Book để sinh viên dễ quan sát. Sinh viên có thể test lại bằng API GET all Books đã trình bày ở bài trước. Ví dụ chỉ cần nhấn lại chức năng xem Books trên giao diện Web, nó sẽ tải lại dữ liệu mới nhất:

books works!

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	
b6	Khai phá dữ liệu cơ bản	55	

Dữ liệu đã được hiển thị cho sách có mã {b6}, hình ảnh ta chưa xử lý, qua phần MongoDB ta sẽ xử lý hình ảnh.

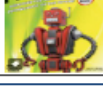
Bài 43 – Triệu gọi HTTP POST – Create a Book

Yêu cầu:

Thiết kế giao diện Front End (my-app) để nhập dữ liệu Book rồi triệu gọi API ở bài trước để tạo mới 1 Book:

- Mặc định khởi động màn hình sẽ tải toàn bộ Books ở Server về (Sinh viên có thể tái sử dụng **BooksComponent** hoặc lặp lại thao tác triệu gọi API lấy toàn bộ Books)
- Mỗi lần submit thành công 1 Book thì màn hình sẽ cập nhật lại danh sách.

Book Id:
Book Name:
Price:

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	

Hướng dẫn:

Bước 1: Trong tập tin “Book.ts”, bổ sung lớp Book:

```
export interface IBook{
    BookId:string,
    BookName:string,
    Price:number,
    Image:string
}
export class Book{
    constructor(
        public BookId:string="",
        public BookName:string="",
        public Price:number=0,
        public Image:string="")
    {}
}
```

Bước 2: Trong tập tin “book-api.service.ts”, Bổ sung mã lệnh cho BookAPIService:

```
postBook(aBook:any):Observable<any>
{
    const headers=new HttpHeaders().set("Content-
Type","application/json;charset=utf-8")
```

```
const requestOptions:Object={
  headers:headers,
  responseType:"text"
}
return
this._http.post<any>("/books",JSON.stringify(aBook),requestOptions).pipe(
  map(res=>JSON.parse(res) as Array<IBook>),
  retry(3),
  catchError(this.handleError))
}
```

Vì ở my-server ta nhận json nên content-type chính là **application/json**.

aBook là 1 model (Book) nhận từ giao diện, nó cần được đưa về định dạng Json as String bằng lệnh **JSON.stringify(aBook)** để gửi lệnh Server (API HTTP POST) mã lệnh đầy đủ của BookAPIService sau khi chỉnh sửa:

```
import { HttpClient, HttpResponse, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { catchError, map, Observable, retry, throwError } from 'rxjs';
import { IBook } from '../interfaces/Book';

@Injectable({
  providedIn: 'root'
})
export class BookAPIService {

  constructor(private _http: HttpClient) { }

  getBooks():Observable<any>
  {
    const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
    const requestOptions:Object={
      headers:headers,
      responseType:"text"
    }
    return this._http.get<any>("/books",requestOptions).pipe(
      map(res=>JSON.parse(res) as Array<IBook>),
      retry(3),
      catchError(this.handleError))
  }

  handleError(error:HttpResponse){
    return throwError(()=>new Error(error.message))
  }

  getBook(bookId:string):Observable<any>
  {
    const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")
    const requestOptions:Object={
      headers:headers,
```

```
        responseType: "text"
      }
      return this._http.get<any>("/books/" + bookId, requestOptions).pipe(
        map(res => JSON.parse(res) as IBook),
        retry(3),
        catchError(this.handleError))
    }
    postBook(aBook: any): Observable<any>
    {
      const headers = new HttpHeaders().set("Content-Type", "application/json; charset=utf-8")
      const requestOptions: Object = {
        headers: headers,
        responseType: "text"
      }
      return this._http.post<any>("/books", JSON.stringify(aBook), requestOptions).pipe(
        map(res => JSON.parse(res) as Array<IBook>),
        retry(3),
        catchError(this.handleError))
    }
  }
}
```

Bước 3: Tạo thêm BookNewComponent.

Chỉnh sửa mã lệnh trong tập tin “**book-new.component.ts**”:

```
import { Component } from '@angular/core';
import { FormControl, FormGroup } from '@angular/forms';
import { BookAPIService } from '../book-api.service';
import { Book } from '../interfaces/Book';

@Component({
  selector: 'app-book-new',
  templateUrl: './book-new.component.html',
  styleUrls: ['./book-new.component.css']
})
export class BookNewComponent {
  book = new Book();
  books: any
  errMessage: string = ''
  constructor(private _service: BookAPIService){
    this._service.getBooks().subscribe({
      next: (data) => {this.books = data},
      error: (err) => {this.errMessage = err}
    })
  }
  postBook()
  {
    this._service.postBook(this.book).subscribe({
      next: (data) => {this.books = data},
      error: (err) => {this.errMessage = err}
    })
  }
}
```

```

    })
  }
}

```

-Biến **book** là biến dùng để mapping theo ngModel. Khi người dùng nhập dữ liệu trên giao diện nó sẽ tự động mapping các thuộc tính và ô nhập liệu.

-Biến **books** để tải toàn bộ book trên server nhằm phục vụ cho việc hiển thị lại danh sách.

-Để mặc định khi mở giao diện thì danh sách Book được hiển thị thì trong constructor ta triệu gọi lại API HTTP GET books (ở đây nếu ta tái sử dụng BookComponent thì không cần gọi lại)

-Hàm postBook() sẽ triệu gọi API POST Book để đẩy dữ liệu từ Giao diện lên Server để lưu trữ. Nếu thực hiện thành công nó sẽ trả về toàn bộ dữ liệu mới nhất, lúc này ta cập nhật lại danh sách.

Chỉnh sửa mã lệnh trong tập tin “**book-new.component.html**”:

```

<p>book-new works!</p>
<p>{{errorMessage}}</p>
<table>
  <tr>
    <td>Book Id:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel" name="BookId"
id="BookId" placeholder="Nhập Book Id"
[(ngModel)]="book.BookId"/>
    </td>
  </tr>
  <tr>
    <td>Book Name:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel"
name="BookName" id="BookName" placeholder="Nhập Book Name"
[(ngModel)]="book.BookName"/>
    </td>
  </tr>
  <tr>
    <td>Price:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel"
name="BookPrice" id="BookPrice" placeholder="Nhập Price"
[(ngModel)]="book.Price"/>
    </td>
  </tr>
  <tr>
    <td colspan="2">

```

```
<button (click)="postBook()">Submit</button>
</td>
</tr>
</table>
<table border="1">
  <tr>
    <td>BookId</td>
    <td>BookName</td>
    <td>Price</td>
    <td>Image</td>
  </tr>
  <tbody>
    <tr *ngFor="let book of books">
      <td>{{book.BookId}}</td>
      <td>{{book.BookName}}</td>
      <td>{{book.Price}} </td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>
```

Table thứ nhất để cho nhập liệu thông tin chi tiết của Book, dùng [(ngModel)] để binding.

Table thứ hai để hiển thị danh sách Book lấy từ server. Nếu tái sử dụng component thì ta có thể thay thế table này thành 1 lệnh duy nhất:

```
<div app-books></div>
```

Cấu hình routing hoặc tham chiếu trực tiếp vào “app.component.html” để chạy component này ta sẽ có kết quả như đề bài yêu cầu.

Bài 44 – Tạo HTTP PUT – Update a Book

Yêu cầu:

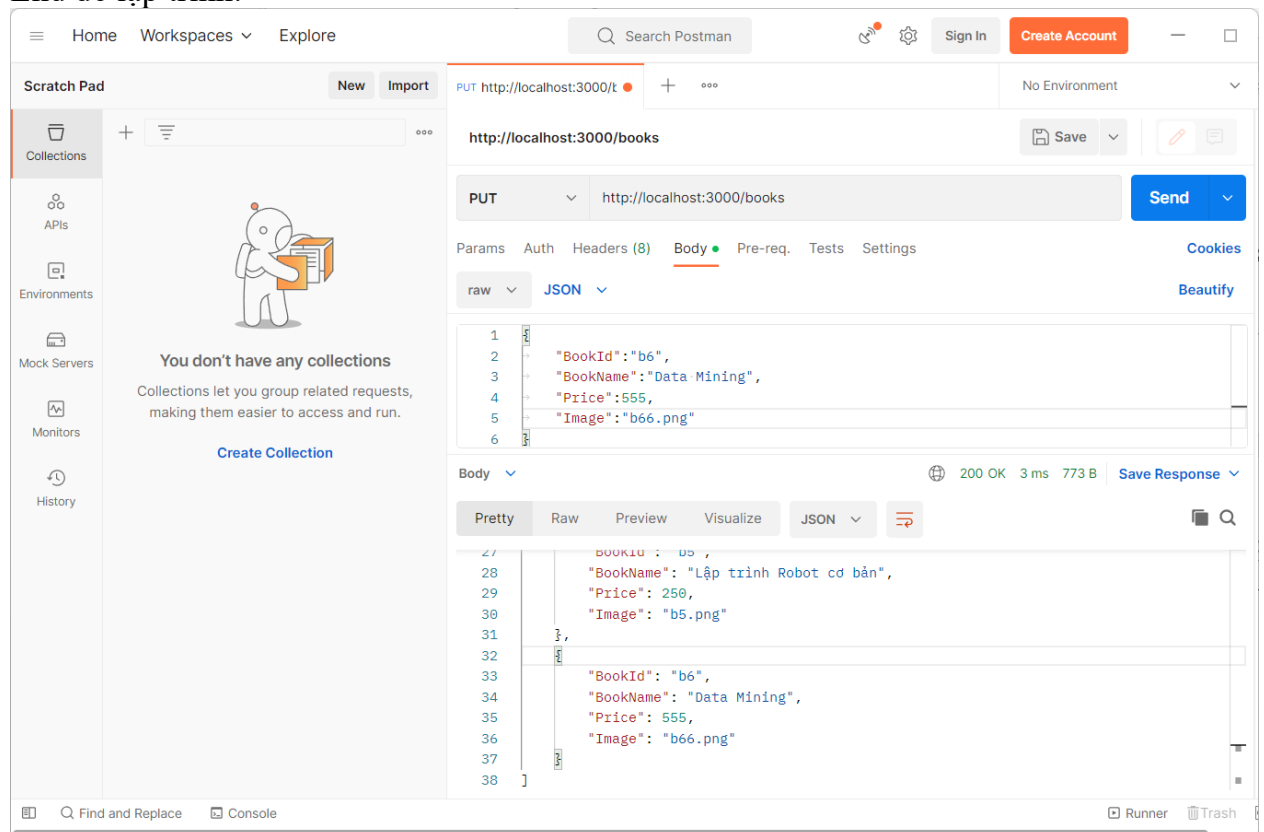
Viết Restful API HTTP PUT để cập nhật thông tin của 1 Book khi biết BookId.

Hướng dẫn:

Bước 1: Mở file **index.js** trong project “my-server”, bổ sung Restful API cập nhật book như dưới đây:

```
app.put("/books", cors(), (req, res) => {
  book = database.find(x => x.BookId === req.body.BookId)
  if (book !== null) {
    book.BookName = req.body.BookName
    book.Price = req.body.Price
    book.Image = req.body.Image
  }
  res.send(database)
})
```

Bước 2: Thử nghiệm Postman, để đảm bảo rằng API chạy được trước khi gắn vào Front End để lập trình:



- Chọn method là PUT
- Chọn raw/JSON
- Thả body nhập cấu trúc Book Json để gửi lên Server

```
{
  "BookId": "b6",
  "BookName": "Data Mining",
  "Price": 555,
  "Image": "b66.png"
}
```

-Nhấn nút “send” ta thấy Server đã chỉnh sửa dữ liệu thành công khi trả về danh sách dữ liệu mới nhất với các thông tin vừa được cập nhật.

Bài 45 – Triệu gọi HTTP PUT – Update a Book

Yêu cầu:

Thiết kế giao diện tương tự như triệu gọi HTTP POST, tuy nhiên trường hợp này là gửi dữ liệu lên để chỉnh sửa:

Book Id:

Book Name:

Price:

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	
b6	Data Mining	555	

Hướng dẫn:

Bước 1:

Mở file “**book-api.service.ts**” để bổ sung hàm **putBook** cho lớp **BookAPIService**.

```

putBook(aBook: any): Observable<any>
{
    const headers = new HttpHeaders().set("Content-Type", "application/json; charset=utf-8")
    const requestOptions: Object = {
        headers: headers,
        responseType: "text"
    }
    return
    this._http.put<any>("/books", JSON.stringify(aBook), requestOptions).pipe(
        map(res => JSON.parse(res) as Array<IBook>),
        retry(3),
        catchError(this.handleError))
}

```

Bước 2:

Tạo thêm một component **BookUpdate**.

File “**book-update.component.ts**” chỉnh mã lệnh:

```
export class BookUpdateComponent {
  book=new Book();
  books:any
  errMessage:string=''
  constructor(private _service: BookAPIService){
    this._service.getBooks().subscribe({
      next:(data)=>{this.books=data},
      error:(err)=>{this.errMessage=err}
    })
  }
  putBook()
  {
    this._service.putBook(this.book).subscribe({
      next:(data)=>{this.books=data},
      error:(err)=>{this.errMessage=err}
    })
  }
}
```

File “**book-update.component.html**” chỉnh mã lệnh:

```
<p>book-update works!</p>
<p>{{errMessage}}</p>
<table>
  <tr>
    <td>Book Id:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel" name="BookId"
id="BookId" placeholder="Nhập Book Id"
[(ngModel)]=book.BookId"/>
    </td>
  </tr>
  <tr>
    <td>Book Name:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel"
name="BookName" id="BookName" placeholder="Nhập Book Name"
[(ngModel)]=book.BookName"/>
    </td>
  </tr>
  <tr>
    <td>Price:</td>
    <td>
      <input type="text" class="form-control" #name="ngModel"
name="BookPrice" id="BookPrice" placeholder="Nhập Price"
[(ngModel)]=book.Price"/>
    </td>
  </tr>
</table>
```

```

</tr>
<tr>
  <td colspan="2">
    <button (click)="putBook()">Update</button>
  </td>
</tr>
</table>
<table border="1">
  <tr>
    <td>BookId</td>
    <td>BookName</td>
    <td>Price</td>
    <td>Image</td>
  </tr>
  <tbody>
    <tr *ngFor="let book of books">
      <td>{{book.BookId}}</td>
      <td>{{book.BookName}}</td>
      <td>{{book.Price}} </td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>

```

Cấu hình routing hoặc tham chiếu trực tiếp vào “app.component.html” để chạy component này ta sẽ có kết quả như đề bài yêu cầu.

Book Id:
 Book Name:
 Price:

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	
b6	Data Mining	555	

Trước khi sửa

Book Id:
 Book Name:
 Price:

BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Introduction Machine Learning	350	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	
b6	Data Mining	555	

Sau khi sửa

Bài 46– Tạo HTTP DELETE– Remove a Book

Yêu cầu:

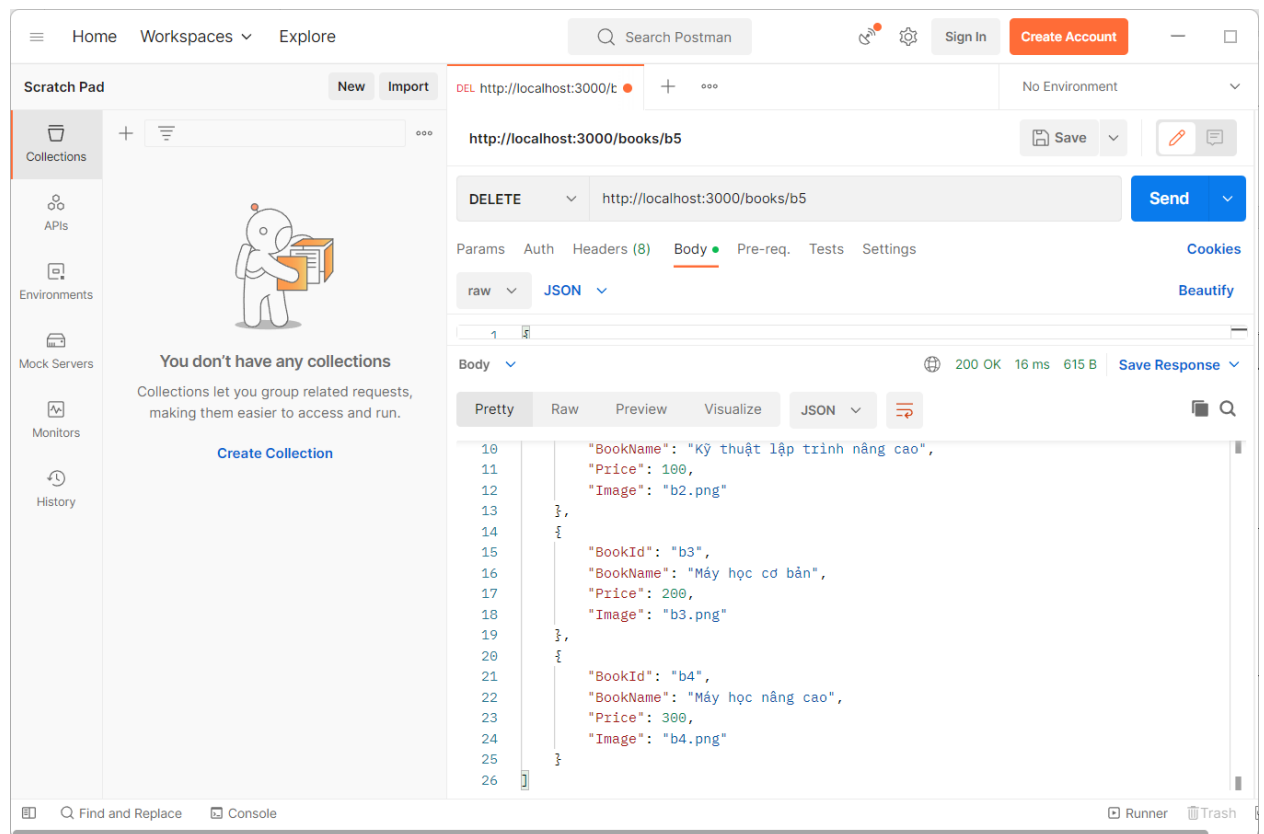
Viết Restful API để xóa 1 Book.

Hướng dẫn:

Trong project “my-server”, chỉnh sửa **index.js**:

```
app.delete("/books/:id", cors(), (req, res) => {
  id = req.params["id"]
  database = database.filter(x => x.BookId !== id);
  res.send(database)
})
```

Trong Postman, method chọn DELETE, và url = “http://localhost:3000/books/b5”
Sau đó nhấn nút Send, chương trình sẽ xóa thành công Book có BookId=b5.



Bài 47 – Triệu gọi HTTP DELETE– Remove a Book

Yêu cầu:

Thiết kế giao diện để xóa Book bằng cách triệu gọi HTTP Delete API.

Book Id:	b5	Delete Book	
BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	

-Mặc định danh sách Book sẽ hiển thị lên giao diện

-Nhập Book Id, nhấn nút Delete Book để xóa đồng thời cập nhật lại danh sách nếu xóa thành công

Hướng dẫn:

Bước 1: Mở file “book-api.service.ts” và Bổ sung mã lệnh cho **BookAPIService**:

```
deleteBook(bookId:string):Observable<any>
{
  const headers=new HttpHeaders().set("Content-Type","application/json;charset=utf-8")
  const requestOptions:Object={
    headers:headers,
    responseType:"text"
  }
  return this._http.delete<any>("/books/"+bookId,requestOptions).pipe(
    map(res=>JSON.parse(res) as Array<IBook>),
    retry(3),
    catchError(this.handleError))
}
```

Bước 2: Thêm component BookDelete.

Bổ sung mã lệnh cho “book-delete.component.ts”:

```
import { Component } from '@angular/core';
import { BookAPIService } from '../book-api.service';

@Component({
```

```

selector: 'app-book-delete',
templateUrl: './book-delete.component.html',
styleUrls: ['./book-delete.component.css']
})
export class BookDeleteComponent {
  books: any
  errorMessage: string = ''
  constructor(private _service: BookAPIService){
    this._service.getBooks().subscribe({
      next: (data) => {this.books = data},
      error: (err) => {this.errorMessage = err}
    })
  }
  deleteBook(bookId: any)
  {
    this._service.deleteBook(bookId).subscribe({
      next: (data) => {this.books = data},
      error: (err) => {this.errorMessage = err}
    })
  }
}

```

Bổ sung mã lệnh cho “book-delete.component.html”:

```

<p>book-delete works!</p>
<p>{{errorMessage}}</p>
Book Id: <input type="text" #bookId> <button
(click)="deleteBook(bookId.value)">Delete Book</button>
<table border="1">
  <tr>
    <td>BookId</td>
    <td>BookName</td>
    <td>Price</td>
    <td>Image</td>
  </tr>
  <tbody>
    <tr *ngFor="let book of books">
      <td>{{book.BookId}}</td>
      <td>{{book.BookName}}</td>
      <td>{{book.Price}} </td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>

```

Cấu hình routing hoặc tham chiếu trực tiếp vào “app.component.html” để chạy component này ta sẽ có kết quả như đề bài yêu cầu.

Book Id:	b5	Delete Book	
BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	
b5	Lập trình Robot cơ bản	250	

Trước khi xóa

Book Id:	b5	Delete Book	
BookId	BookName	Price	Image
b1	Kỹ thuật lập trình cơ bản	70	
b2	Kỹ thuật lập trình nâng cao	100	
b3	Máy học cơ bản	200	
b4	Máy học nâng cao	300	

Sau khi xóa

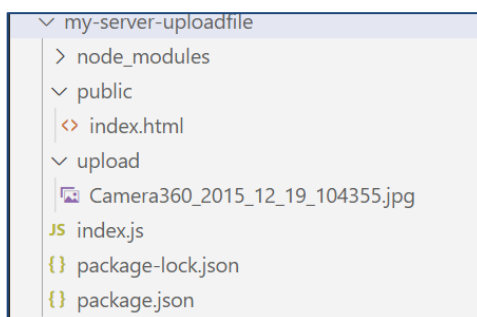
Bài 48 – Upload files to Server-1

Yêu cầu:

Tạo một Project NodeJs mới mang tên “my-server-uploadfile”, viết lệnh để upload file lên Server. Trường hợp này màn hình load file nằm trong Server

Hướng dẫn:

Bước 1: Tạo Project “my-server-uploadfile” có cấu trúc như hình bên dưới:



-Tạo các thư mục public, upload

Bước 2: Cài đặt các lệnh:

```
npm install express
npm install express-fileupload
npm i --save-dev nodemon
npm i --save-dev morgan
npm i body-parser
```

-Sau khi cài đặt xong và cấu hình “package.json”

```
{
  "name": "my-server-uploadfile",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Tran Duy Thanh",
  "license": "ISC",
  "dependencies": {
    "express": "^4.18.2",
    "express-fileupload": "^1.4.0"
  },
  "devDependencies": {
    "morgan": "^1.10.0",
    "nodemon": "^2.0.21"
  }
}
```

Bước 3: Viết mã lệnh cho index.js:

```
const express = require('express');
const fileUpload = require('express-fileupload');
const app = express();
const port = 3001;
const morgan=require("morgan")
app.use(morgan("combined"))
const bodyParser=require("body-parser")
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({extended: true}));

app.use(
  fileUpload({
    limits: {
      fileSize: 10000000,
    },
    abortOnLimit: true,
  })
);

// Add this line to serve our index.html page
app.use(express.static('public'));

app.get('/', (req, res) => {
  res.sendFile('index.html');
});

app.post('/upload', (req, res) => {
  // Get the file that was set to our field named "image"
  const { image } = req.files;
```

```
// If no image submitted, exit
if (!image) return res.sendStatus(400);

// Move the uploaded image to our upload folder
image.mv(__dirname + '/upload/' + image.name);

// All good
res.sendStatus(200);
});

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`);
});
```

-Bước 4: Tạo index.html trong thư mục public

```
<!DOCTYPE html>
<form action="/upload" method="POST" enctype="multipart/form-data">
  <input type="file" name="image" />
  <button type="submit">Upload</button>
</form>
```

-Chạy ứng dụng Server: npm start

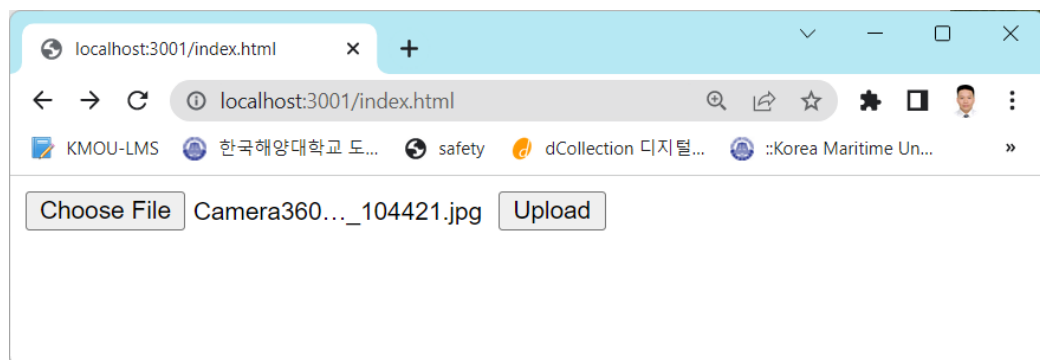
```
PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server-uploadfile> npm start
```

```
> my-server-uploadfile@1.0.0 start
> nodemon index.js
```

```
[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
Example app listening on port 3001
```

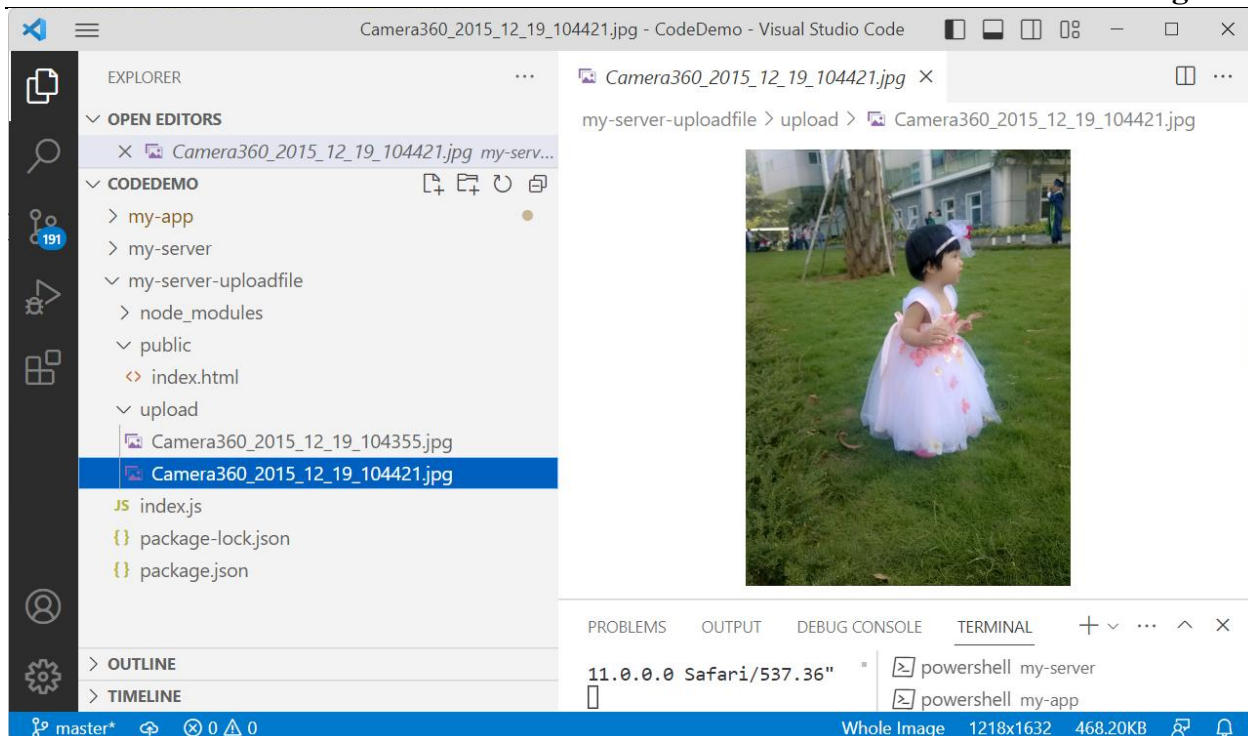
Chạy trình duyệt:

<http://localhost:3001/index.html>



Bấm Choose File, chọn hình rồi bấm upload

Kết quả sau khi upload thành công:



Bài 49 – Upload files to Server-2

Yêu cầu:

Tái sử dụng “my-server-uploadfile” xem như là 1 Service độc lập dùng để upload hình ảnh(và ta không cần phải lập trình lại hàm upload), hoặc ta có thể di chuyển các lệnh trong “index.js” của project “my-server-uploadfile” sang “index.js” của project “my-server”.

Sinh viên áp dụng bài này để cập nhật hình ảnh cho Books ở các bài trước.

Khi upload được hình ảnh lên Server rồi thì ta đã có đường dẫn vật lý lưu trữ trên Server (mà không phải trong thư mục assets), khi nạp dữ liệu Json trả về cho HTML hiển thị hình ảnh thì ta chỉ cần sửa lại giá trị thuộc tính “Image” của đối tượng trả về cho đúng đường dẫn của hình ảnh đã đưa lên server là được.

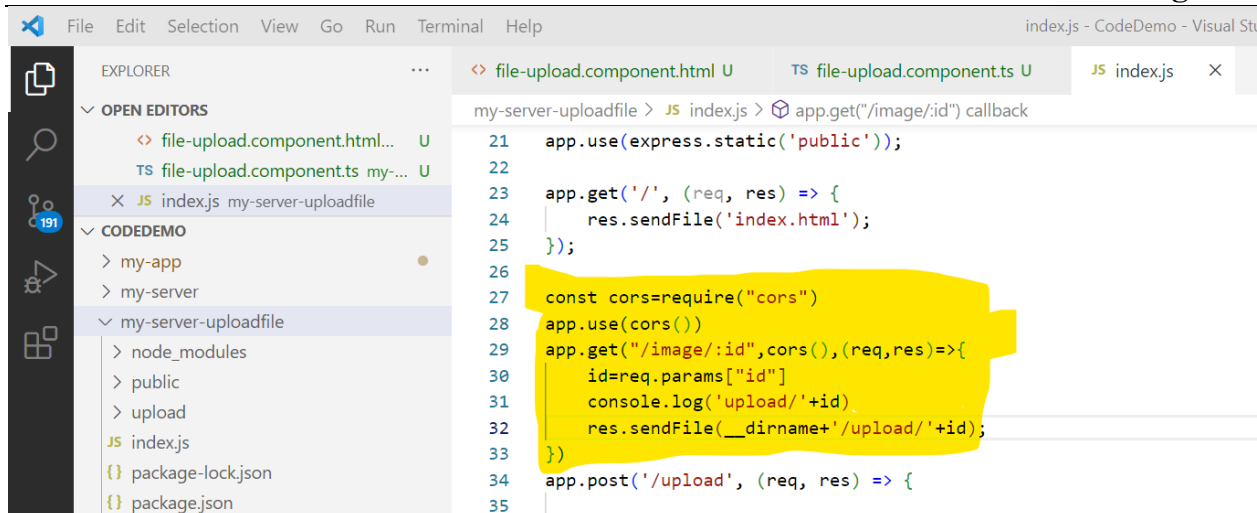
Hướng dẫn:

Trong bài này, tài liệu sẽ hướng dẫn tái sử dụng “my-server-uploadfile” và trình thêm cách xem lại hình ảnh đã tải lên. Dựa vào bài này Sinh viên có thể dễ dàng áp dụng cho phần cập nhật hình Ảnh của Books ở các bài trước.

Bước 1: Cài đặt thêm thư viện cors cho “my-server-uploadfile”

npm i cors

Bước 2: Mở file index.js của project “my-server-uploadfile” và bổ sung thêm API Get để tải hình ảnh từ server về client khi biết tên hình nhằm hiển thị nó lên HTML



```

21 app.use(express.static('public'));
22
23 app.get('/', (req, res) => {
24   res.sendFile('index.html');
25 });
26
27 const cors=require("cors")
28 app.use(cors())
29 app.get("/image/:id",cors(),(req,res)=>{
30   id=req.params["id"]
31   console.log('upload/'+id)
32   res.sendFile(__dirname+'/upload/'+id);
33 })
34 app.post('/upload', (req, res) => {
35

```

Ví dụ, khi có 1 tên hình nào đó thì ở client (AngularJS) ở các component ta sẽ gọi:

```
<img src='http://localhost:3001/image/metaverse.png'>
```

Nó sẽ tự động gọi API GET để hiển thị chi tiết hình ảnh lên giao diện HTML

Tức là nếu áp dụng để lưu hình ảnh cho **book** thì ta chỉ cần lưu tên hình khi tạo một Book mới lên Server. Ở ví dụ này thì **book.image**= “metaverse.png”, lúc này

Thay vì các bài trước viết là:

```

```

Thì chuyển thành:

```

```

Dưới đây là mã lệnh đầy đủ của index.js trong server “my-server-uploadfile”, lưu ý Webserver này ta chạy ở port 3001

```

const express = require('express');
const fileUpload = require('express-fileupload');
const app = express();
const port = 3001;
const morgan=require("morgan")
app.use(morgan("combined"))
const bodyParser=require("body-parser")
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({extended: true}));

app.use(
  fileUpload({
    limits: {
      fileSize: 10000000,
    },
    abortOnLimit: true,
  })
);

// Add this line to serve our index.html page
app.use(express.static('public'));

```

```
app.get('/', (req, res) => {
  res.sendFile('index.html');
});

const cors=require("cors")
app.use(cors())
app.get("/image/:id",cors(),(req,res)=>{
  id=req.params["id"]
  console.log('upload/'+id)
  res.sendFile(__dirname+'/upload/'+id);
})
app.post('/upload', (req, res) => {

  // Get the file that was set to our field named "image"
  const { image } = req.files;

  // If no image submitted, exit
  if (!image) return res.sendStatus(400);

  // Move the uploaded image to our upload folder
  image.mv(__dirname + '/upload/' + image.name);

  // All good
  res.sendStatus(200);
});

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`);
});
```

Bước 3: Mở dự án “my-app” để tham chiếu sử dụng Restful API trong “my-server-uploadfile”, như vậy “my-app” đã sử dụng nhiều Restful API ở các Web server khác nhau (nó cũng là đúng với thực tế).

-Mở file “proxy.conf.json” bổ sung khai báo:

```
{
  "context": ["/upload"],
  "target": "http://localhost:3001",
  "secure": true,
  "changeOrigin":true,
  "logLevel":"debug"
}
```

Khai báo đầy đủ của “proxy.conf.json”:

```
[
  {
    "context": ["/exchange"],
    "target": "https://www.dongabank.com.vn",
    "secure": true,
```

```

    "changeOrigin":true,
    "logLevel":"debug"
  },
  {
    "context": ["/products"],
    "target": "https://fakestoreapi.com",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  },
  {
    "context": ["/books"],
    "target": "http://localhost:3000",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  },
  {
    "context": ["/upload"],
    "target": "http://localhost:3001",
    "secure": true,
    "changeOrigin":true,
    "logLevel":"debug"
  }
]

```

Bước 4: Tạo thêm một component mới với tên “FileUpload”.

-Mã lệnh “file-upload.component.html” như sau:

```

<input type="file" class="file-input"
      (change)="onFileSelected($event)" #fileUpload name="image">

<div class="file-upload">

  {{fileName || "No file uploaded yet."}}

  <button mat-mini-fab color="primary" class="upload-btn"
    (click)="fileUpload.click()">
    Upload
  </button>
</div>

```

-Mã lệnh của “file-upload.component.ts” như sau:

```

import { HttpClient, HttpEventType } from '@angular/common/http';
import { Component, Input } from '@angular/core';
import { finalize, Subscription } from 'rxjs';

@Component({
  selector: 'app-file-upload',
  templateUrl: './file-upload.component.html',

```

```
styleUrls: ['./file-upload.component.css']
})
export class FileUploadComponent {
  @Input()
  requiredFileType:any;

  fileName = '';
  uploadProgress:number=0;
  uploadSub: Subscription=new Subscription();

  constructor(private http: HttpClient) {}

  onFileSelected(event:any) {
    const file:File = event.target.files[0];

    if (file) {
      this.fileName = file.name;
      const formData = new FormData();
      formData.append("image", file);

      const upload$ = this.http.post("/upload", formData, {
        reportProgress: true,
        observe: 'events'
      })
      .pipe(
        finalize(() => this.reset())
      );

      this.uploadSub = upload$.subscribe(event => {
        if (event.type == HttpEventType.UploadProgress) {
          this.uploadProgress = Math.round(100 * (event.loaded /
event.total!));
        }
      })
    }
  }

  cancelUpload() {
    this.uploadSub.unsubscribe();
    this.reset();
  }

  reset() {
    this.uploadProgress = 0;
    this.uploadSub = new Subscription();
  }
}
```

Cấu hình Routing hoặc tham chiếu trực tiếp “app.component.html” để chạy ta sẽ có kết quả như mong muốn. Sinh viên suy nghĩ để tích hợp phần hình ảnh này vào cho Book.

Bài 50 – Bài tập tổng hợp Restful API(*)

Yêu cầu:

- Xây dựng Restful API cung cấp các chức năng: lấy toàn bộ thông tin sách, xem chi tiết sách, tạo sách mới, chỉnh sửa sách, xóa sách.
- Thiết kế màn hình quản lý thông tin sách như hình minh họa:

QUẢN LÝ THÔNG TIN SÁCH								
Create New								
Tensach	Giaban	Mota	Anhbìa	Ngaycapnhat	Soluongton	MaCD	MaNXB	
Giáo trình Tin học cơ bản	26000.00	Nội dung của cuốn: Tin Học Cơ Bản Windows XP gồm có 7 chương: Chương 1: Một số vấn đề cơ bản. Chương 2: Sử dụng nhanh thành công cụ và thanh thực đơn trong My Computer và Windows Explorer. Chương 3: Các thao tác trong windows XP. Chương 4: Các thiết lập trong Windows XP. Chương 5: Bảo trì máy tính. Chương 6: Các phím tắt Chương 7: Hỏi và đáp các thắc mắc. Xin trân trọng giới thiệu cuốn sách cùng bạn	THCB.jpg	25/10/2014 12:00:00 SA	120	7	1	Edit Details Delete
Giáo trình Cơ Sở Dữ Liệu Với Visual Basic 2005 Và ADO.NET 2.0	12000.00	Cuốn sách này gồm 3 phần sau: Phần 1: Xử lý văn bản trong Microsoft thiếu các nội dung sau: Chương 1: Căn bản về cơ sở dữ liệu. Chương 2: Các bộ kết nối và tương tác dữ liệu. Chương 3: Bộ chứa dữ liệu DataSet. Chương 4: Bộ điều hợp dữ liệu DataAdapter. Chương 5: Sử dụng các điều khiển ràng buộc dữ liệu. Chương 6: Tạo báo cáo với Crystal Report. Chương 7: ADO.NET và XML. Xin trân trọng giới thiệu cùng các bạn.	TH004.jpg	23/10/2013 12:00:00 SA	25	3	2	Edit Details Delete
Visual Basic 2005 Tập 3, Quyển 2: Lập Trình Web	20000.00	"Visual Basic 2005 Tập 3, Quyển 2: Lập Trình Web Với Cơ Sở Dữ Liệu" sẽ cung cấp kỹ thuật và hướng dẫn bạn: Tự học xây dựng ứng dụng Web quản lý CSDL toàn diện với ADO.NET 2.0 và ASP.NET. Khai thác các đối tượng và nguồn dữ liệu dành cho WebForm. Sử dụng các điều khiển dữ liệu đặc thù dành riêng cho ASP.NET và Web. Làm quen với những khái niệm xử lý dữ liệu hoàn toàn mới. Ràng buộc dữ liệu với các thành phần giao diện Web Forms. Thiết kế ứng dụng Web "Quản lý CD Shop" trực quan và thực tế. Cung cấp một kiến thức hoàn chỉnh về Web cho các bạn yêu thích ngôn ngữ Visual Basic và .NET Framework. Sách có kèm theo CD-ROM bài tập.	LTWeb2005.jpg	15/09/2014 12:00:00 SA	240	8	4	Edit Details Delete

- +Mặc định chương trình sẽ gọi API GET all Books để hiển thị.
- +Nhấn nút “Create New” hiển thị màn hình thêm sách, khi thêm thành công sẽ quay lại màn hình danh sách ban đầu
- +Nhấn nút “Edit” hiển thị màn hình chỉnh sửa thông tin sách, chỉnh sửa thành công sẽ quay lại màn hình danh sách ban đầu
- + Nhấn nút “Details” hiển thị thông tin chi tiết của sách đang chọn
- + Nhấn nút “Delete” sẽ xác nhận hỏi xóa hay không, nếu xóa thành công thì nạp lại danh sách sau khi xóa.
- Sinh viên tự thiết kế cấu trúc JSon cho Book để đáp ứng yêu cầu đề bài.
- Xử lý hình ảnh cho Book.

Module 6 –Restful API với MongoDB

Nội dung kiến thức thực hành:

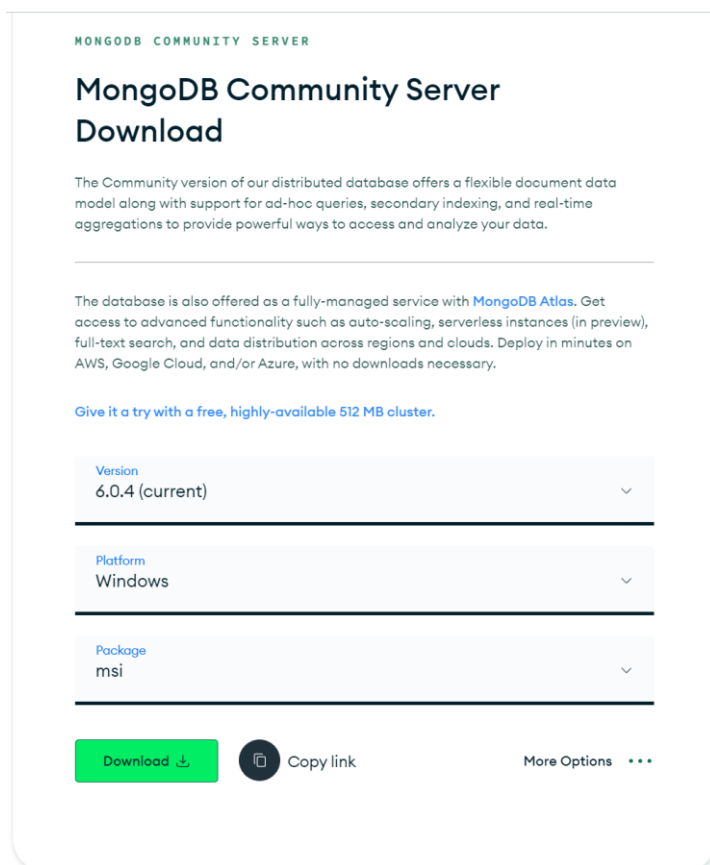
- + củng cố các kiến thức và kỹ năng lập trình Restful API
- + Sử dụng MongoDB và MongoDB Compass
- + Các thư viện kết nối MongoDB
- + Tạo Restful API với MongoDB
- + Lưu trữ hình ảnh trong MongoDB

Bài 51- Cài đặt và sử dụng MongoDB

Yêu cầu:

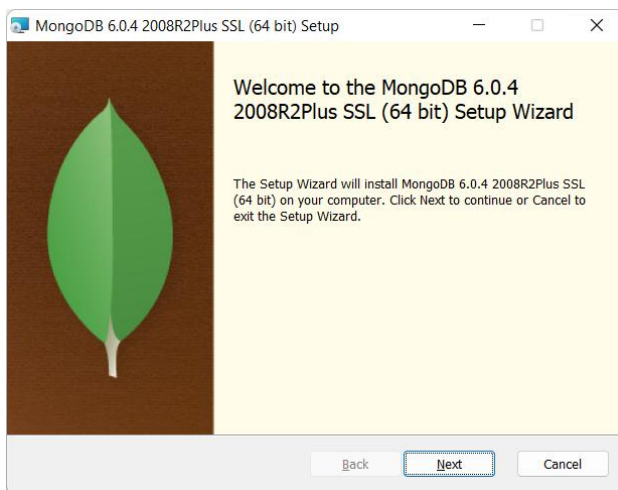
Để tải MongoDB, ta vào link

<https://www.mongodb.com/try/download/community>

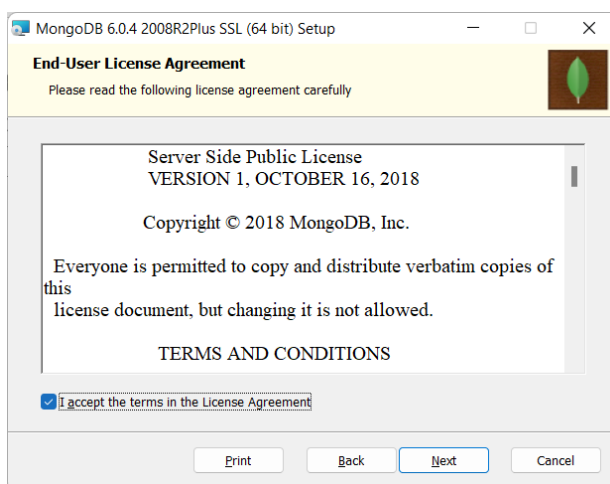


Nhấn Download ta được file “mongodb-windows-x86_64-6.0.4-signed.msi”

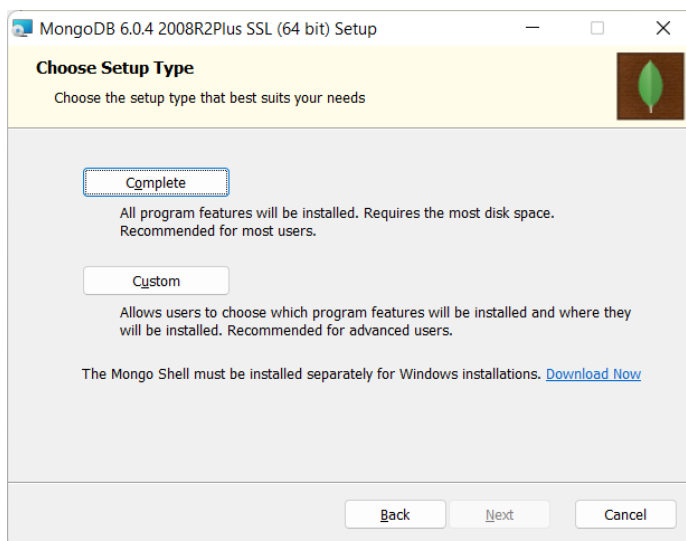
Bấm vào file để cài đặt



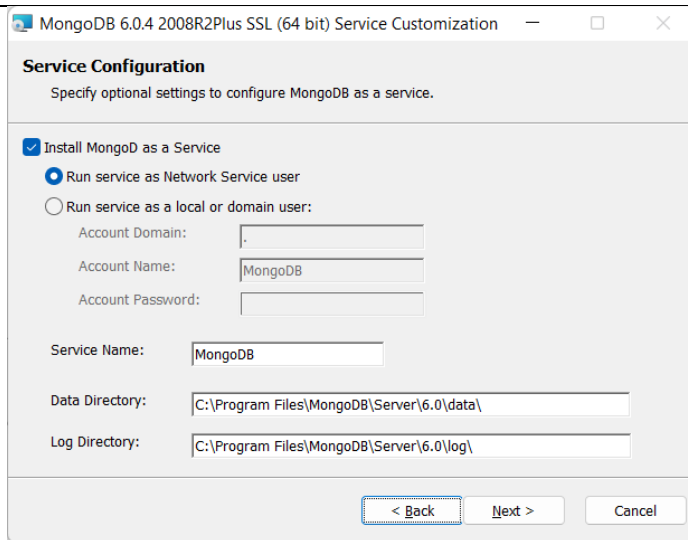
Nhấn Next



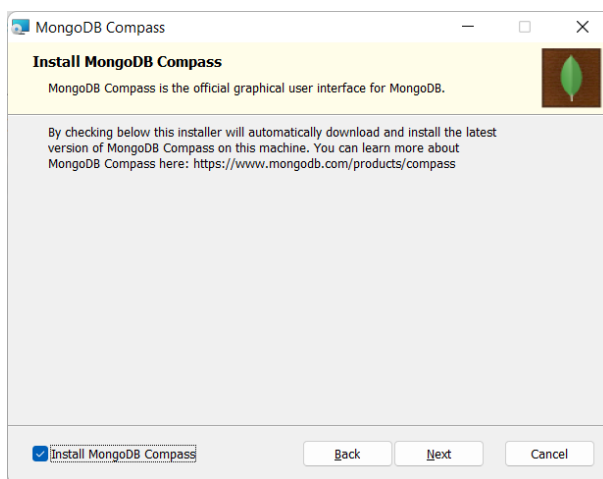
Checked “I accept the terms in the License Agreement” và nhấn Next



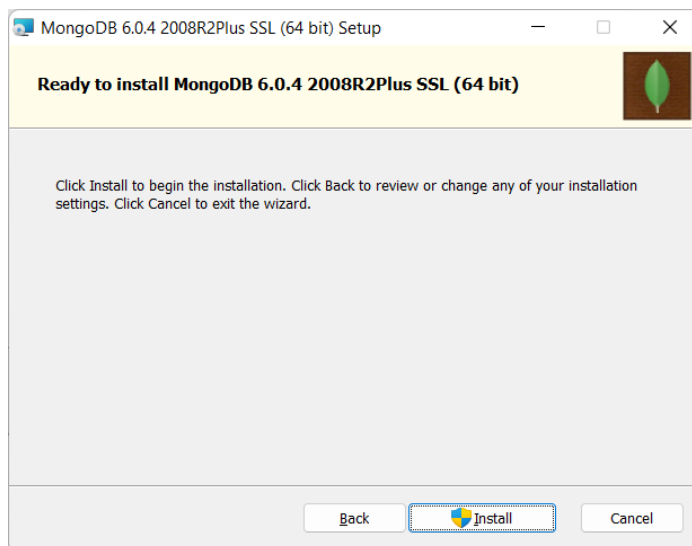
Nhấn nút complete



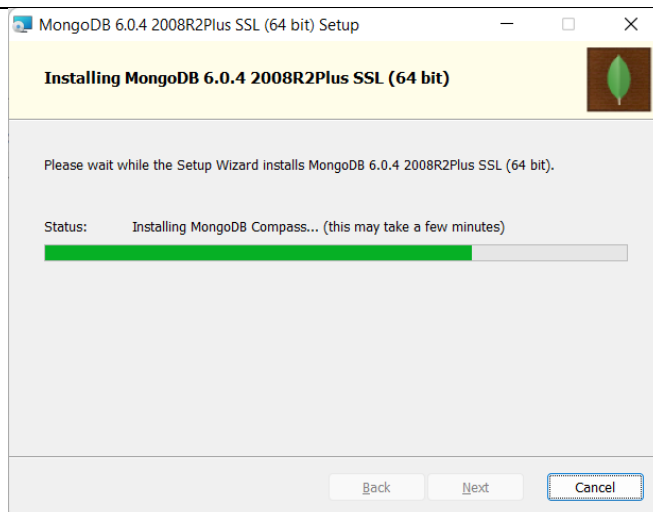
Để mặc định như hình trên và bấm Next



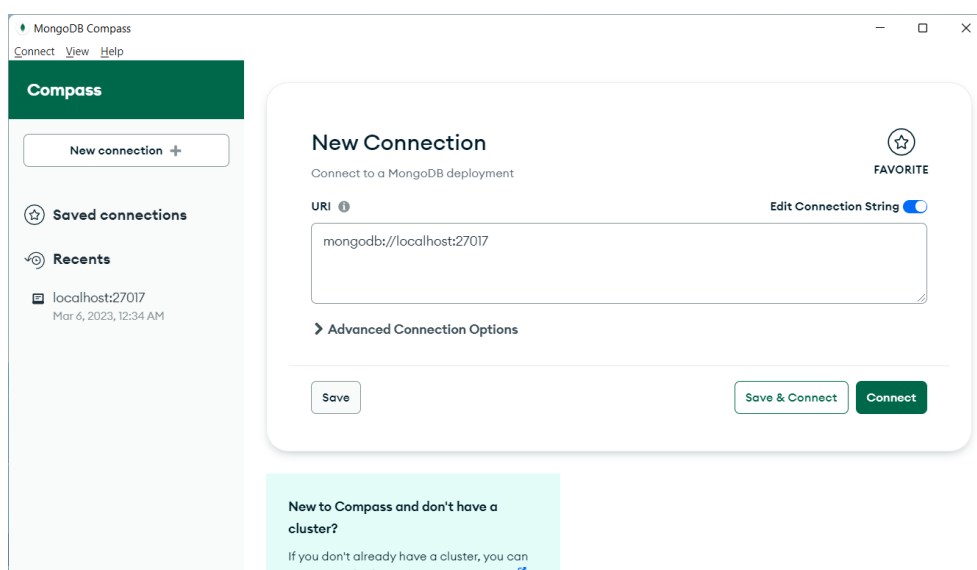
Checked “Install MongoDB Compass” rồi bấm Next. Compass là công cụ cho ta thao tác tới MongoDB Server



Bấm Install



Chờ hệ thống cài đặt cho tới lúc thành công. Đây là màn hình MongoDB Compass khi cài đặt xong.

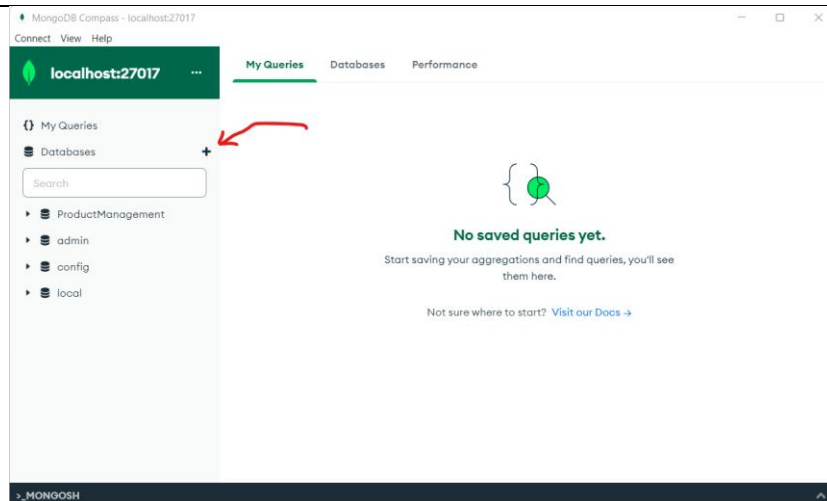


Đường dẫn kết nối:

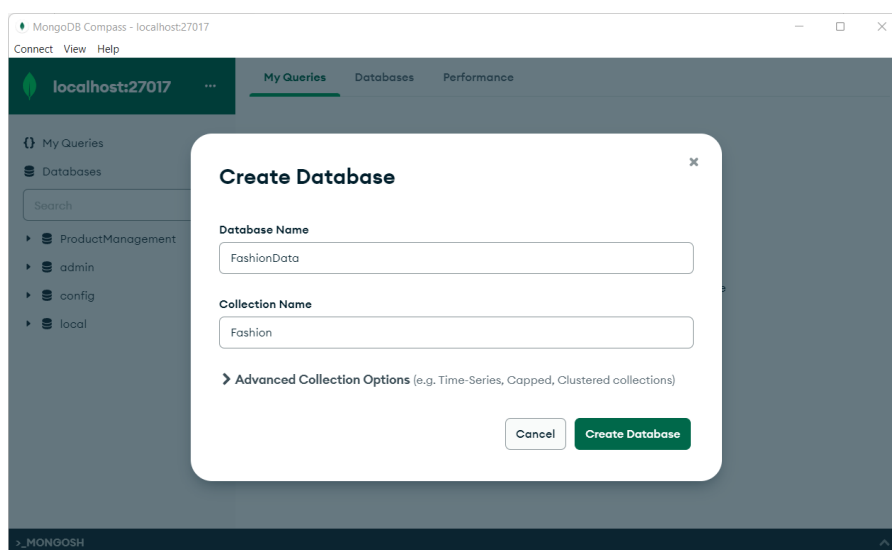
`mongodb://localhost:27017`

Đôi khi trong quá trình lập trình, có thể một số thư viện set mặc định IPv6 nếu để localhost. Như vậy máy tính sẽ không thể lập trình được nếu nó là IPv4. Lúc này ta nên thay localhost bằng 127.0.0.1

Bấm connect để kết nối:



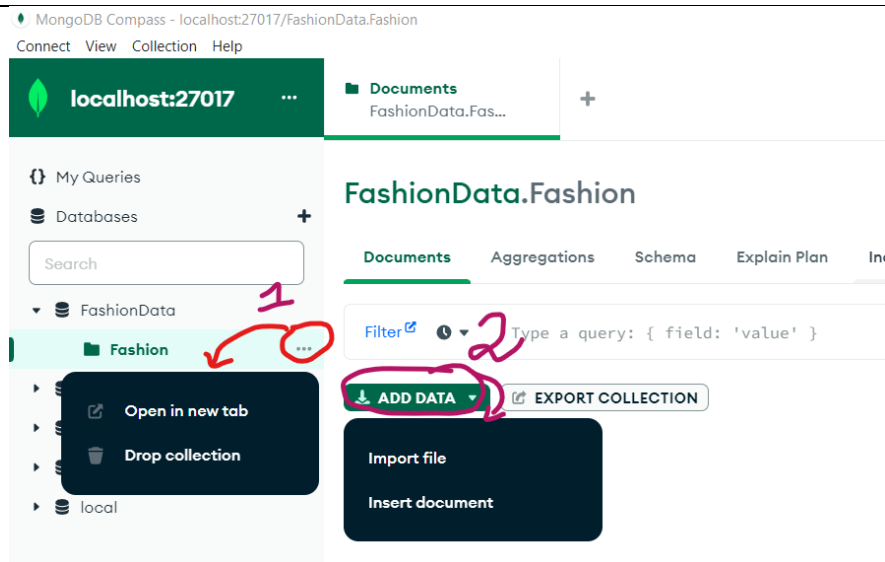
Để tạo Cơ sở dữ liệu ta bấm vào biểu tượng dấu “+” kế bên chữ “Databases” và đặt tên cơ sở dữ liệu như hình:



Database Name: **FashionData**

Collection Nam: **Fashion**

SQL		NoSQL
Database	➔	Database
Table	➔	Collection
Row	➔	Document
Column	➔	Field/Attribute



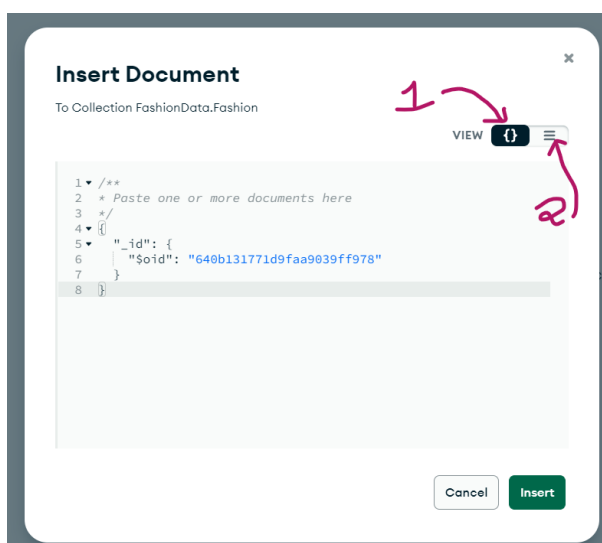
Mục số 1:

- “Open in new tab”: Mở thêm 1 Tab mới để làm việc
- “Drop collection”: Xóa collection đang chọn

Mục số 2:

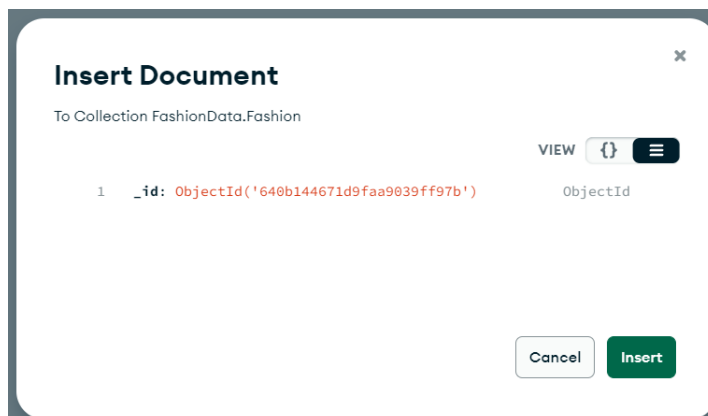
- “Import File”: Nhập dữ liệu từ một file có sẵn trước đó
- “Insert document”: Thêm mới một dữ liệu, nó thường là 1 JSon (ta có thể tạo 1 cấu trúc phức tạp chứ không nhất thiết là 1 JSonObject)

Bây giờ ta chọn “Insert document”:

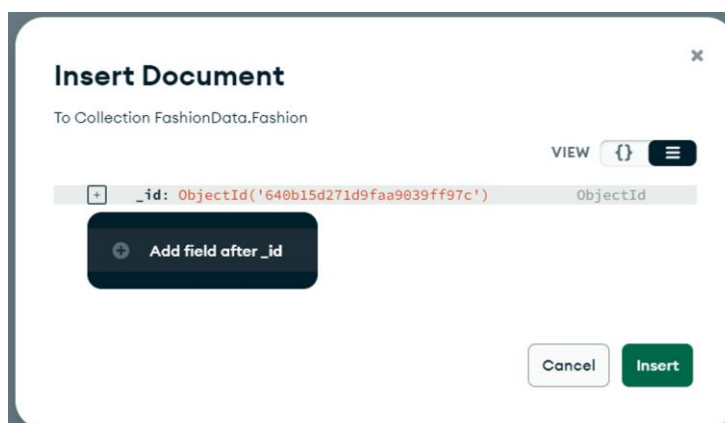


- Cách nhập theo số 1 là định dạng cấu trúc Json trực tiếp
- Cách nhập theo số 2 là định dạng Visualization dạng Field và cho lựa chọn kiểu dữ liệu của mỗi Field.

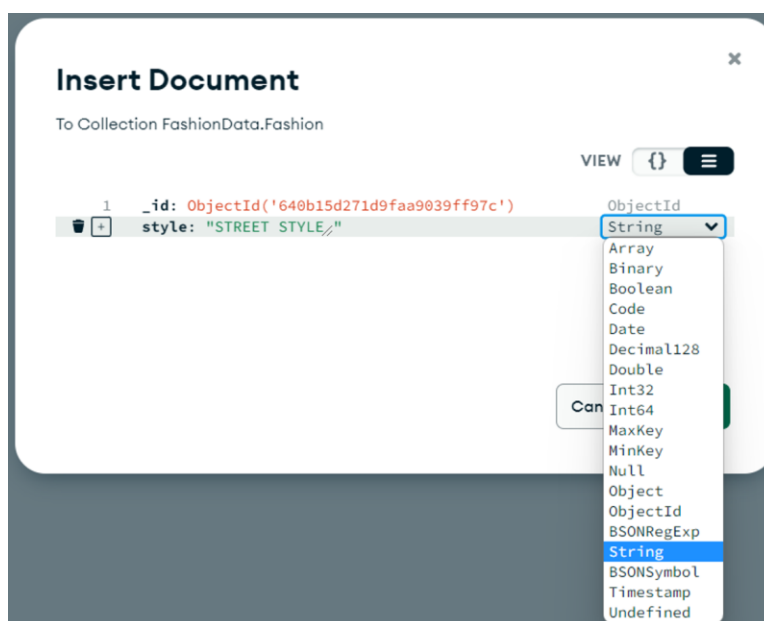
Ta lựa chọn cách số 2 để dễ thiết kế:



Di chuyển chuột tới đằng trước 1 Field bất kỳ, nó hiển thị ra dấu “+”, bấm chuột vào dấu “+” và chọn “Add field after _id” như hình dưới đây:



Ta tạo một Field tên là style và giá trị “STREET STYLE” và kiểu dữ liệu là String như hình minh họa dưới đây:

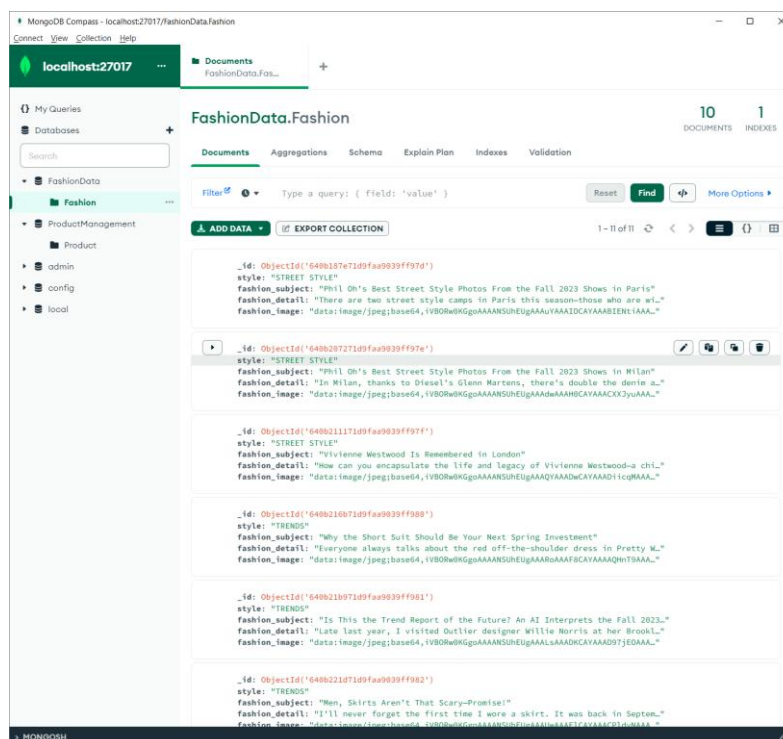


Tương tự như vậy, ta tiếp tục tạo thêm một số Field cho Collection Fashion này:



Hình ảnh là Base64 String. Dùng tool này để đưa hình về string rồi dán vào MongoDB: <https://codebeautify.org/image-to-base64-converter>

Đây là minh họa tạo cấu trúc và nhập document cho collection Fashion:



Sinh viên có thể tải dữ liệu được Export ở đây:

<http://teacher.uelstore.com/resource/Fashion.json>

Sau đó dùng chức năng import collection để tiết kiệm thời gian, cách import:

-Đầu tiên tạo 1 cơ sở dữ liệu tên “FashionData”

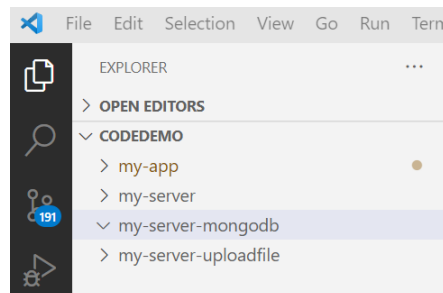
-Sau đó tạo 1 collection tên “Fashion”

-Cuối cùng chọn “Fashion” và import file, trở tới “Fashion.json”

Bài 52 - Cấu hình và Kết nối NodeJS với MongoDB

Yêu cầu:

Tạo một thư mục đồng cấp “my-server-mongodb” để build Project cho Webserver tương tác MongoDB.



Bước 1: Cài đặt các lệnh:

```
npm install express
npm install express-fileupload
npm install mongodb
npm i --save-dev nodemon
npm i --save-dev morgan
npm i body-parser
npm i cors
```

Ngoài ra nếu dùng **mongoose** kết hợp thì ta có thể tải đặt thêm 2 thư viện:

```
npm i mongoose
npm install --save mongoclient
```

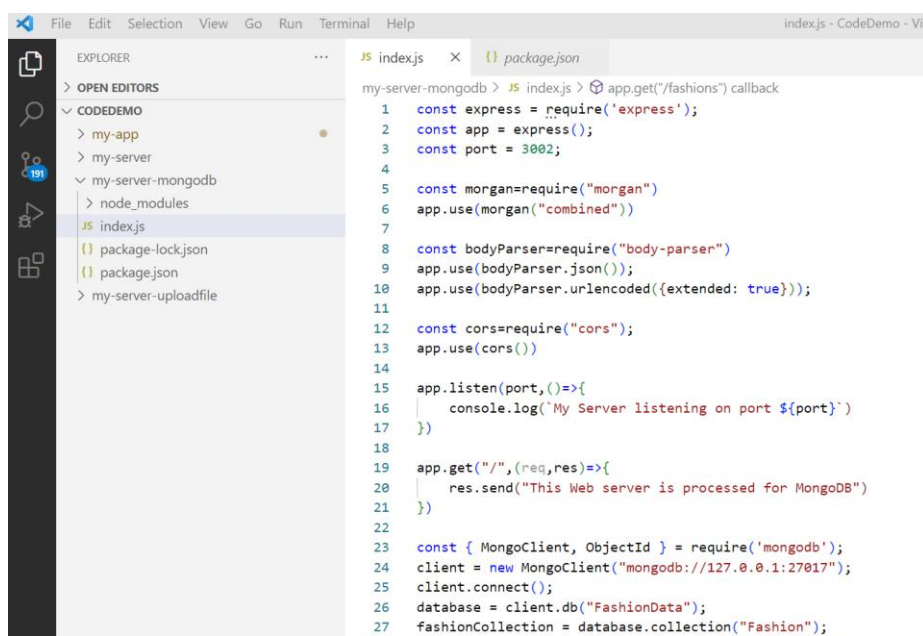
Trong bài này, tài liệu hướng dẫn dùng thư viện “**mongodb**”
File “package.json”:

```
{
  "name": "my-server-mongodb",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Tran Duy Thanh",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.2",
    "cors": "^2.8.5",
    "express": "^4.18.2",
    "express-fileupload": "^1.4.0",
    "mongodb": "^5.1.0"
  },
}
```

```
"devDependencies": {
  "morgan": "^1.10.0",
  "nodemon": "^2.0.21"
}
```

Bước 2: Tạo file Index.js để viết API Service trong “my-server-mongodb”

Tạo file **index.js** cấu hình một số thư viện và một số hàm như đã học ở các bài trước, chạy ở Port 3002:



```
1 const express = require('express');
2 const app = express();
3 const port = 3002;
4
5 const morgan=require("morgan")
6 app.use(morgan("combined"))
7
8 const bodyParser=require("body-parser")
9 app.use(bodyParser.json());
10 app.use(bodyParser.urlencoded({extended: true}));
11
12 const cors=require("cors");
13 app.use(cors())
14
15 app.listen(port,()=>{
16   console.log(`My Server listening on port ${port}`)
17 })
18
19 app.get("/",(req,res)=>{
20   res.send("This Web server is processed for MongoDB")
21 })
22
23 const { MongoClient, ObjectId } = require('mongodb');
24 client = new MongoClient("mongodb://127.0.0.1:27017");
25 client.connect();
26 database = client.db("FashionData");
27 fashionCollection = database.collection("Fashion");
```

Các dòng lệnh từ số 1 tới 21 là cấu hình giống như đã học ở bài trước để xử lý các vấn đề liên quan tới Restful API.

Các dòng lệnh từ 23 tới 27 là để kết nối mongodb database:

- Dòng 23 gọi thư viện **mongodb**
- Dòng 24 khai báo chuỗi kết nối tới Server, thay localhost bằng 127.0.0.1 nếu các thư viện lấy Ipv6 của máy tính
- Dòng 25 gọi lệnh kết nối tới Server
- Dòng 26 kết nối tới Database “FashionData” mà ta thiết kế ở bài trước.
- Dòng 27 truy suất collection Fashion. Nếu cơ sở dữ liệu có nhiều collection mà ta muốn truy suất thì cứ thêm các dòng khai báo mới.

Dưới đây là mã lệnh đầy đủ index.js:

```
const express = require('express');
const app = express();
const port = 3002;

const morgan=require("morgan")
app.use(morgan("combined"))

const bodyParser=require("body-parser")
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({extended: true}));

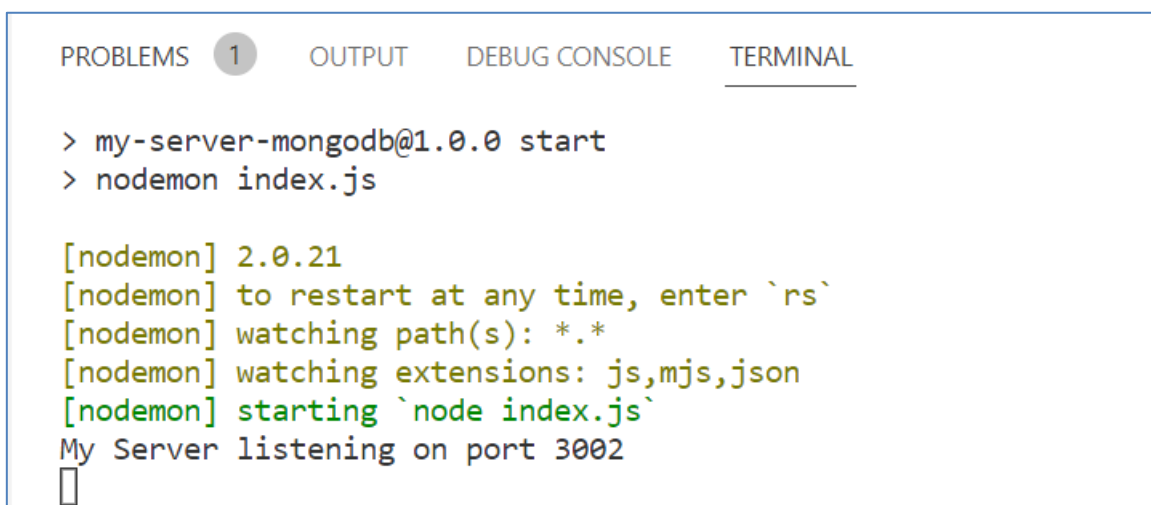
const cors=require("cors");
app.use(cors())

app.listen(port,()=>{
  console.log(`My Server listening on port ${port}`)
})

app.get("/",(req,res)=>{
  res.send("This Web server is processed for MongoDB")
})

const { MongoClient, ObjectId } = require('mongodb');
client = new MongoClient("mongodb://127.0.0.1:27017");
client.connect();
database = client.db("FashionData");
fashionCollection = database.collection("Fashion");
```

Bước 3: Gõ lệnh **npm start** để chạy Service



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

> my-server-mongodb@1.0.0 start
> nodemon index.js

[nodemon] 2.0.21
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node index.js`
My Server listening on port 3002
█
```

Như vậy tới đây chương trình đã chạy ở port 3002 và kết nối thành công tới MongoDB

Bài 53 – Tạo và triệu gọi API - HTTP GET – List Fashion

Yêu cầu:

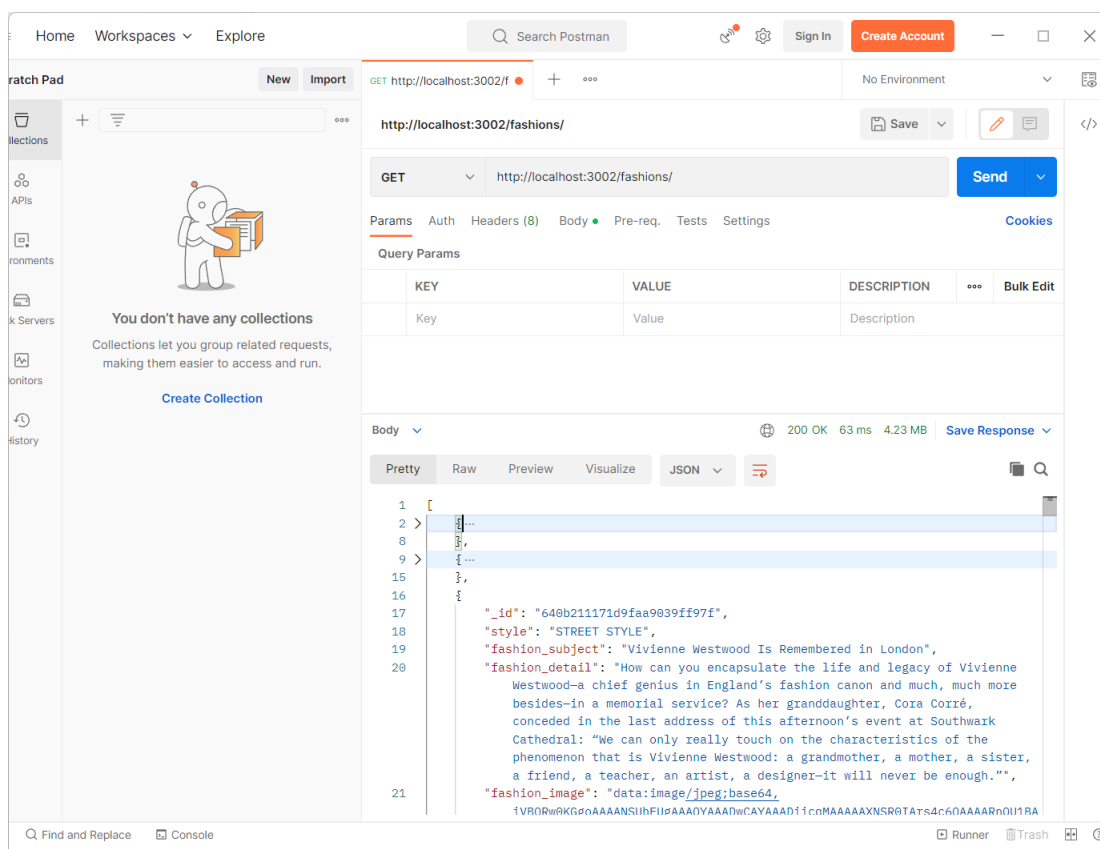
Tiếp tục cập nhật mã lệnh cho index.js trong project “my-server-mongodb”, khai báo API lấy toàn bộ Fashion và hiển thị danh sách Fashion này lên giao diện trong project “my-app”.

Hướng dẫn:

Bước 1: Vào “my-server-mongodb”, bổ sung hàm sau vào cuối file index.js (bổ sung đăng sau các lệnh kết nối tới MongoDB):

```
app.get("/fashions", cors(), async (req, res) => {
  const result = await fashionCollection.find({}).toArray();
  res.send(result)
})
```

Lệnh trên truy vấn toàn bộ Fashion và trả về JsonArray. Vì ta đã cấu hình tự động restart server khi mã nguồn thay đổi nên chỉ cần mở Postman lên test chức năng này:



Ra được kết quả như trên là đã viết API fashions thành công.

Bước 2: Trong “my-app”, thêm mới 1 model class **Fashion** trong file “Fashion.ts”:

```
export class Fashion{  
    constructor(  
        public _id:any=null,  
        public style:string="",  
        public fashion_subject:string="",  
        public fashion_detail:string="",  
        public fashion_image:string=""){  
    }  
}
```

Bước 3: Trong “my-app”, bổ sung Service FashionAPIService.

Viết mã lệnh cho “fashion-api.service.ts”:

```
import { HttpClient, HttpResponse, HttpHeaders } from  
'@angular/common/http';  
import { Injectable } from '@angular/core';  
import { catchError, map, Observable, retry, throwError } from 'rxjs';  
import { Fashion } from '../models/Fashion';  
  
@Injectable({  
    providedIn: 'root'  
})  
export class FashionAPIService {  
  
    constructor(private _http: HttpClient) { }  
    getFashions():Observable<any>  
    {  
        const headers=new HttpHeaders().set("Content-Type","text/plain;charset=utf-8")  
        const requestOptions={  
            headers:headers,  
            responseType:"text"  
        }  
        return this._http.get<any>("/fashions",requestOptions).pipe(  
            map(res=>JSON.parse(res) as Array<Fashion>),  
            retry(3),  
            catchError(this.handleError))  
    }  
    handleError(error:HttpResponse){  
        return throwError(()=>new Error(error.message))  
    }  
}
```

Bước 4: Trong “my-app”, bổ sung Componen FashionComponent.

Viết mã lệnh cho “fashion.component.ts”:

```
import { Component } from '@angular/core';
import { FashionAPIService } from '../fashion-api.service';

@Component({
  selector: 'app-fashion',
  templateUrl: './fashion.component.html',
  styleUrls: ['./fashion.component.css']
})
export class FashionComponent {
  fashions:any;
  errMessage:string=''
  constructor(public _service: FashionAPIService){
    this._service.getFashions().subscribe({
      next:(data)=>{this.fashions=data},
      error:(err)=>{this.errMessage=err}
    })
  }
}
```

Viết mã lệnh cho “fashion.component.html”:

```
<p>fashion works!</p>
{{errMessage}}
<table border="1">
  <tr>
    <td>Id</td>
    <td>Fashion Subject</td>
    <td>Fashion Detail</td>
    <td>Image</td>
  </tr>
  <tbody>
    <tr *ngFor="let fashion of fashions">
      <td>{{fashion._id}}</td>
      <td>{{fashion.fashion_subject}}</td>
      <td>{{fashion.fashion_detail}} </td>
      <td>
        
      </td>
    </tr>
  </tbody>
</table>
```

Bước 5: Trong “my-app”, bổ sung khai báo cho “proxy.conf.json”:

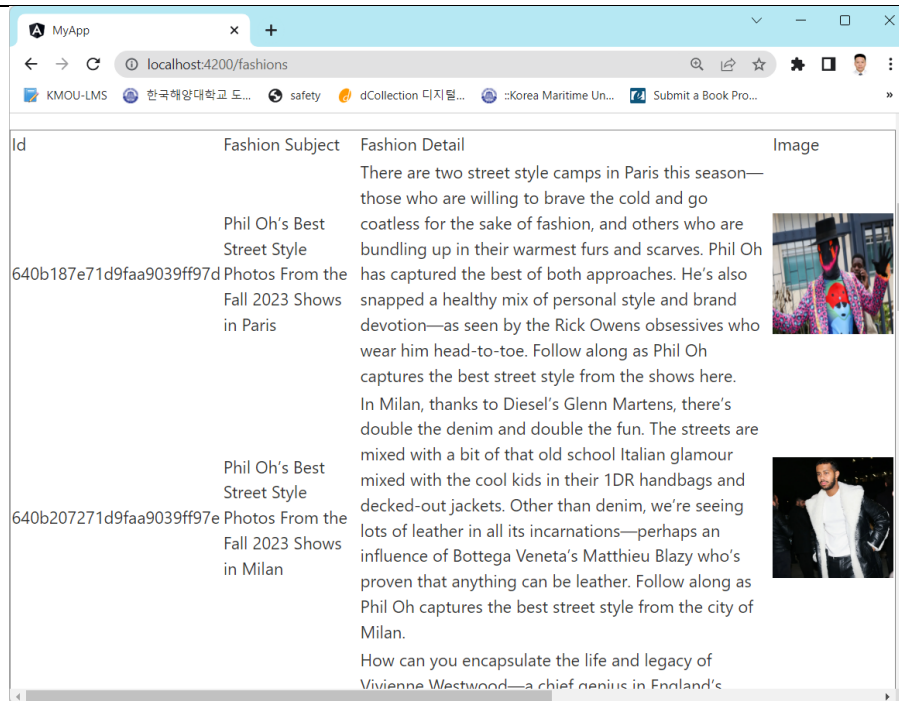
```
{
  "context": ["/fashions"],
  "target": "http://localhost:3002",
  "secure": true,
  "changeOrigin":true,
  "logLevel":"debug"
}
```

Mã khai báo đầy đủ:

```
[
  {
    "context": ["/exchange"],
    "target": "https://www.dongabank.com.vn",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  },
  {
    "context": ["/products"],
    "target": "https://fakestoreapi.com",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  },
  {
    "context": ["/books"],
    "target": "http://localhost:3000",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  },
  {
    "context": ["/upload"],
    "target": "http://localhost:3001",
    "secure": true,
    "changeOrigin": true,
    "logLevel": "debug"
  }
],
{
  "context": ["/fashions"],
  "target": "http://localhost:3002",
  "secure": true,
  "changeOrigin": true,
  "logLevel": "debug"
}
]
```

Bước 6: Khai báo routing hoặc chạy component này trực tiếp trong

“app.component.html”, ta có kết quả như mong muốn:



Lưu ý hình là Base64string, nên ta dùng cú pháp:

```

```

Như vậy khi tạo API lưu mới 1 Fashion thì ta chuyển hình qua Base64String

Bài 54 – Tạo và triệu gọi API - HTTP GET – A Fashion

Yêu cầu:

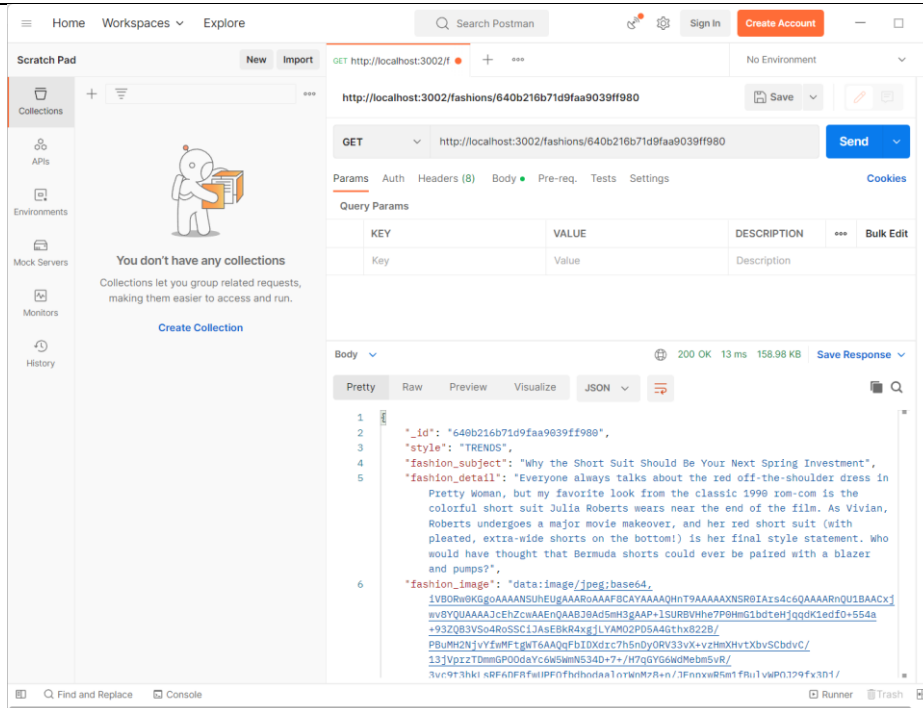
- trong “my-server-mongodb” cập nhật index.js để cung cấp API lấy 1 Fashion khi biết Id
- trong “my-app” triệu gọi API để hiển thị chi tiết Fashion
- Sinh viên suy nghĩ giao diện phù hợp để triệu gọi API

Hướng dẫn:

“index.js” trong “my-server-mongodb” bổ sung mã lệnh sau vào cuối tập tin:

```
app.get("/fashions/:id", cors(), async (req, res) => {
  var o_id = new ObjectId(req.params["id"]);
  const result = await fashionCollection.find({_id:o_id}).toArray();
  res.send(result[0])
})
```

Dùng Postman để test:

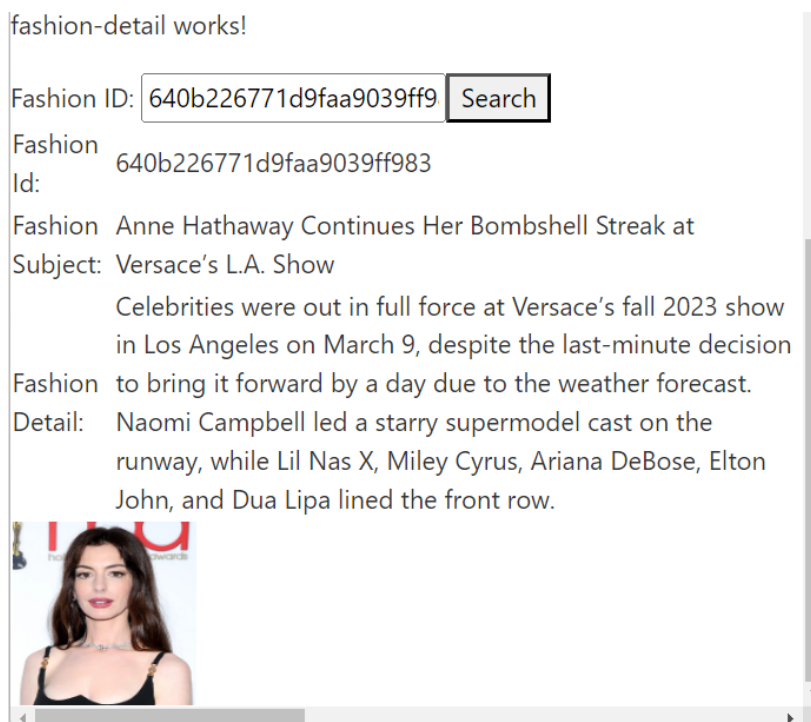


Phần AngularJS Sinh viên xử lý tương tự như các hướng dẫn trước, giống như BookAPIService (hàm getBook(bookId:string)) và BookDetailComponent:

-Bước 1: Bổ sung hàm getFashion(id:string) trong lớp **FashionAPIService**

Hàm này gọi API Restful 1 Fashion

-Bước 2: Bổ sung component “FashionDetailComponent”, giao diện như bên dưới, tuy nhiên Sinh viên nên bố trí cho nó thẩm mỹ:



Bài 55 – Tạo và triệu gọi API - HTTP POST – Create a Fashion

Yêu cầu:

Xây dựng Restful API để thêm mới một Fashion, sử dụng AngularJS để triệu gọi API này, thiết kế giao diện phù hợp.

Hướng dẫn:

Bước 1: Trong “my-server-mongodb”, bổ sung API Post a Fashion trong file index.js (bổ sung cuối file):

```
app.post("/fashions", cors(), async (req, res) => {  
    //put json Fashion into database  
    await fashionCollection.insertOne(req.body)  
    //send message to client(send all database to client)  
    res.send(req.body)  
})
```

Lưu ý trong trường hợp dung lượng gửi lên Server (payload) quá lớn nó sẽ báo lỗi, ta cần xử lý lỗi này bằng cách bổ sung các lệnh sau ở trước các API, trong ví dụ này là cho payload 10mb, bạn có thể tăng lên nếu muốn:

```
const bodyParser = require("body-parser")  
app.use(bodyParser.json({ limit: '10mb' }));  
app.use(bodyParser.urlencoded({ extended: true, limit: '10mb' }));  
app.use(express.json({ limit: '10mb' }));  
app.use(express.urlencoded({ limit: '10mb' }));  
app.use(express.json());
```

Mã lệnh đầy đủ của index.js:

```
const express = require('express');  
const app = express();  
const port = 3002;  
  
const morgan = require("morgan")  
app.use(morgan("combined"))  
  
const bodyParser = require("body-parser")  
app.use(bodyParser.json({ limit: '10mb' }));  
app.use(bodyParser.urlencoded({ extended: true, limit: '10mb' }));  
  
app.use(express.json({ limit: '10mb' }));  
app.use(express.urlencoded({ limit: '10mb' }));  
app.use(express.json());  
  
const cors = require("cors");  
app.use(cors());
```

```
app.listen(port,()=>{
  console.log(`My Server listening on port ${port}`)
})

app.get("/",(req,res)=>{
  res.send("This Web server is processed for MongoDB")
})

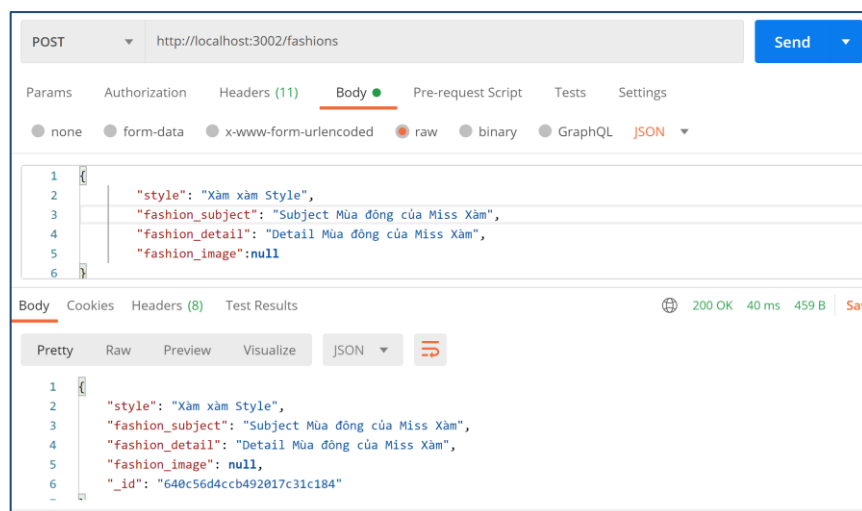
const { MongoClient, ObjectId } = require('mongodb');
client = new MongoClient("mongodb://127.0.0.1:27017");
client.connect();
database = client.db("FashionData");
fashionCollection = database.collection("Fashion");

app.get("/fashions",cors(),async (req,res)=>{
  const result = await fashionCollection.find({}).toArray();
  res.send(result)
})

app.get("/fashions/:id",cors(),async (req,res)=>{
  var o_id = new ObjectId(req.params["id"]);
  const result = await fashionCollection.find({_id:o_id}).toArray();
  res.send(result[0])
})

app.post("/fashions",cors(),async(req,res)=>{
  //put json Fashion into database
  await fashionCollection.insertOne(req.body)
  //send message to client(send all database to client)
  res.send(req.body)
})
```

Bước 2: Dùng postman để test, nếu nó thành công thì qua bước 3



Lưu ý đầu vào là 1 Json không có `_id` (vì nó tự động tạo ra), đầu ra chương trình sẽ trả về cấu trúc input đồng thời có thêm `_id` cùng với giá trị của nó. Việc lập trình như vậy giúp chúng ta dễ dàng truy cập tới dữ liệu vừa đưa vào thông qua `_id`.

Input	Output
<pre>{ "style": "Xàm xàm Style", "fashion_subject": "Subject Mùa đông của Miss Xàm", "fashion_detail": "Detail Mùa đông của Miss Xàm", "fashion_image": null }</pre>	<pre>{ "style": "Xàm xàm Style", "fashion_subject": "Subject Mùa đông của Miss Xàm", "fashion_detail": "Detail Mùa đông của Miss Xàm", "fashion_image": null, "_id": "640c56d4ccb492017c31c184" }</pre>

Bước 3: Trong “my-app”, thêm hàm **postFashion** trong **FashionAPIService**

```
postFashion(aFashion:any):Observable<any>
{
  const headers=new HttpHeaders().set("Content-Type","application/json;charset=utf-8")
  const requestOptions:Object={
    headers:headers,
    responseType:"text"
  }
  return
  this._http.post<any>("/fashions",JSON.stringify(aFashion),requestOptions).pipe(
    map(res=>JSON.parse(res) as Fashion),
    retry(3),
    catchError(this.handleError))
}
```

Bước 4: Bổ sung **FashionNewComponent** trong “my-app”, thiết kế giao diện và xử lý triệu gọi thêm mới một Fashion:

Fashion Style:

Fashion Subject:

Fashion Detail:

Fashion Image:

No file chosen

File “fashion-new.component.html” thiết kế giao diện như sau:

```
<p>fashion-new works!</p>
<p>{{errorMessage}}</p>
<table>
  <tr>
    <td>Fashion Style:</td>
```

```

        <td>
            <input type="text" class="form-control" #name="ngModel" name="style"
id="style" placeholder="Nhập Style"
[(ngModel)]="fashion.style"/>
        </td>
    </tr>
    <tr>
        <td>Fashion Subject:</td>
        <td>
            <input type="text" class="form-control" #name="ngModel"
name="fashion_subject" id="fashion_subject" placeholder="Nhập Subject"
[(ngModel)]="fashion.fashion_subject"/>
        </td>
    </tr>
    <tr>
        <td>Fashion Detail:</td>
        <td>
            <input type="text" class="form-control" #name="ngModel"
name="fashion_detail" id="fashion_detail" placeholder="Nhập Detail"
[(ngModel)]="fashion.fashion_detail"/>
        </td>
    </tr>
    <tr>
        <td>Fashion Image:</td>
        <td>
            <input type="file" class="file-input"
#name="ngModel" name="fashion_image" id="fashion_image"
[(ngModel)]="fashion.fashion_image"
(change)="onFileSelected($event, fashion)"
>
        </td>
    </tr>
    <tr>
        <td colspan="2">
            <button (click)="postFashion()">Submit</button>
        </td>
    </tr>
</table>

```

File “fashion-new.component.ts” coding xử lý chuyển hình vật lý thành Base64String và đẩy lên Server như sau:

```

import { Component } from '@angular/core';
import { from } from 'rxjs';
import { FashionAPIService } from '../fashion-api.service';
import { Fashion } from '../models/Fashion';

@Component({
  selector: 'app-fashion-new',
  templateUrl: './fashion-new.component.html',
  styleUrls: ['./fashion-new.component.css']
})
export class FashionNewComponent {
  fashion=new Fashion();
  errMessage:string=''

```

```
constructor(private _service: FashionAPIService){  
  
}  
public setFashion(f:Fashion)  
{  
    this.fashion=f  
}  
onFileSelected(event:any,fashion:Fashion) {  
    let me = this;  
    let file = event.target.files[0];  
  
    let reader = new FileReader();  
    reader.readAsDataURL(file);  
    reader.onload = function () {  
        fashion.fashion_image=reader.result!.toString()  
    };  
    reader.onerror = function (error) {  
        console.log('Error: ', error);  
    };  
}  
  
postFashion()  
{  
    this._service.postFashion(this.fashion).subscribe({  
        next:(data)=>{this.fashion=data},  
        error:(err)=>{this.errorMessage=err}  
    })  
}  
}
```

Cấu hình routing hoặc tham chiếu trong **app.component.html** để chạy, ta có kết quả như mong muốn.

Bài 56 – Tạo và triệu gọi API - HTTP PUT – Update a Fashion

Yêu cầu:

Xây dựng API để cập nhật Fashion. Lưu ý bài này tương tự như bài tạo API thêm mới một Fashion.

Hướng dẫn:

- Mọi thứ làm tương tự như bài thêm mới một Fashion.

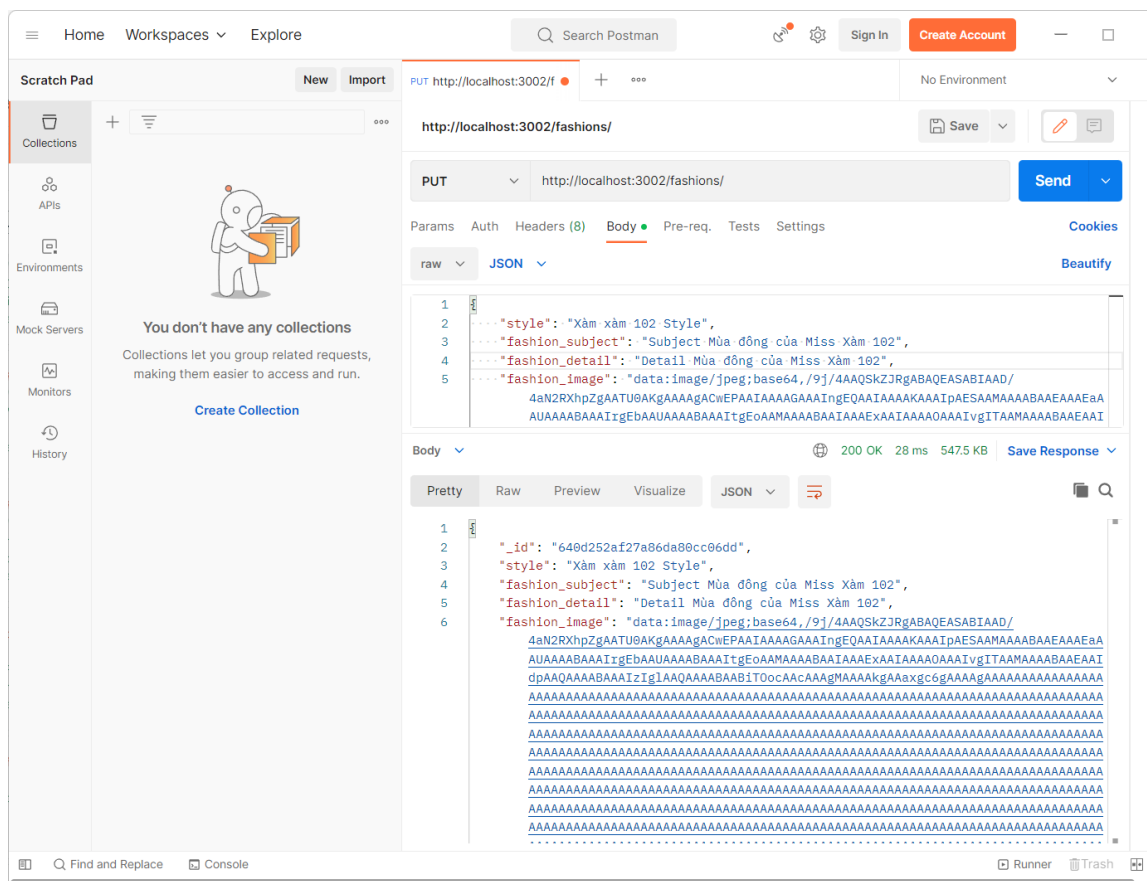
Cập nhật “my-server-mongodb”, trong file index.js bổ sung API sau:

```
app.put("/fashions", cors(), async (req, res) => {

    //update json Fashion into database
    await fashionCollection.updateOne(
        { _id: new ObjectId(req.body._id) }, //condition for update
        { $set: { //Field for updating
            style: req.body.style,
            fashion_subject: req.body.fashion_subject,
            fashion_detail: req.body.fashion_detail,
            fashion_image: req.body.fashion_image
        }
    })

    //send Fashion after updating
    var o_id = new ObjectId(req.body._id);
    const result = await fashionCollection.find({ _id: o_id }).toArray();
    res.send(result[0])
})
```

Hàm trên cập nhật thông tin của một Fashion, sau đó trả về Fashion đã được cập nhật. Ta dùng Postman để test nhằm đảm bảo mã lệnh được viết đúng:



Các giao diện và service trong “my-app” làm tương tự như bài trước nên tài liệu không cung cấp chi tiết ở đây (sao chép qua rồi đổi chỗ gọi API thôi).

Bài 57 – Tạo và triệu gọi API - HTTP DELETE – Remove a Fashion

Yêu cầu:

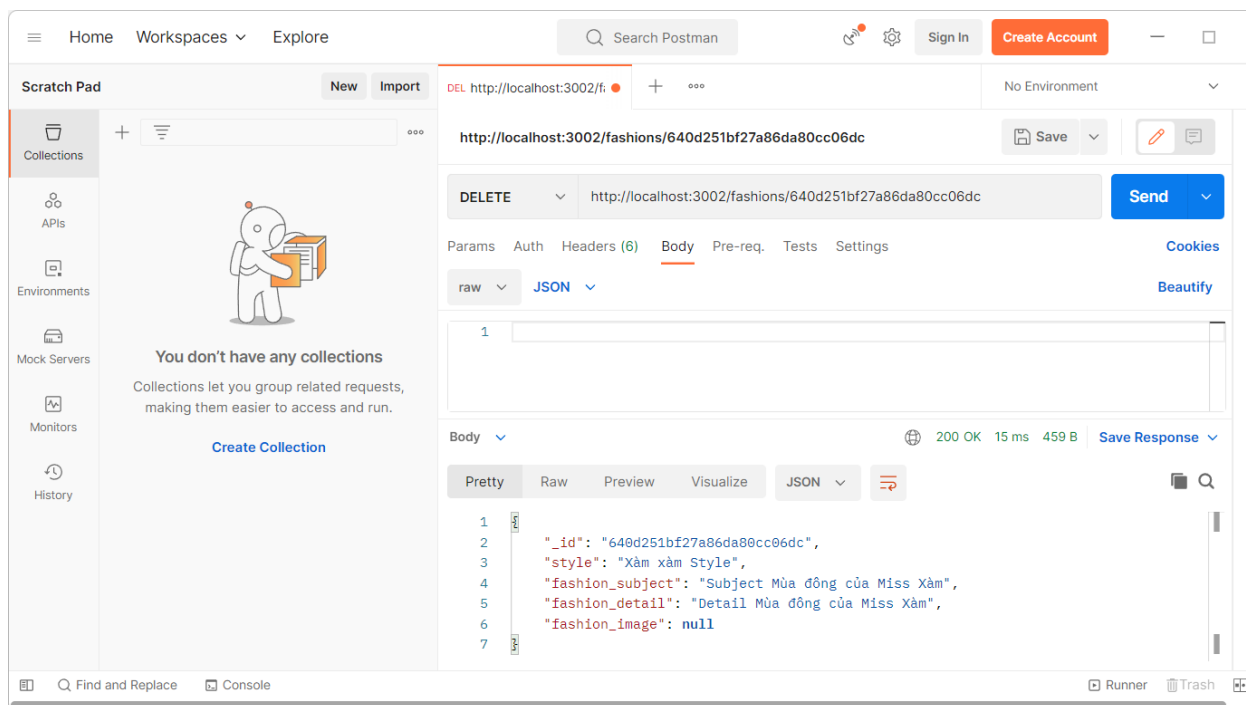
Viết API xóa một Fashion khi biết `_id`

Hướng dẫn:

Trong “my-server-mongodb” bổ sung API DELETE ở cuối “index.js”

```
app.delete("/fashions/:id", cors(), async (req, res) => {
  //find detail Fashion with id
  var o_id = new ObjectId(req.params["id"]);
  const result = await fashionCollection.find({_id:o_id}).toArray();
  //update json Fashion into database
  await fashionCollection.deleteOne(
    {_id:o_id}
  )
  //send Fashion after remove
  res.send(result[0])
})
```

Test Postman để kiểm chứng mã lệnh có đúng không, nếu như màn hình bên dưới là đúng (chọn method DELETE):



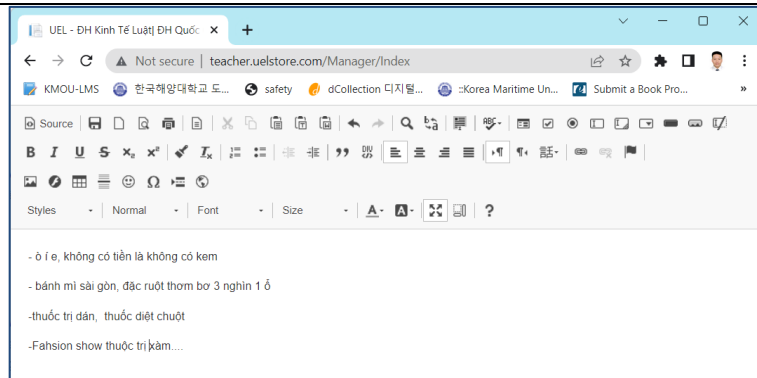
Phần “my-app” là tương tự như `BookDeleteComponent` và `BookAPIService` (`deleteBook(bookId:string):Observable<any>`). Tài liệu không trình bày chi tiết phần triệu gọi API xóa này.

Bài 58 – Bài tập tổng hợp(*)

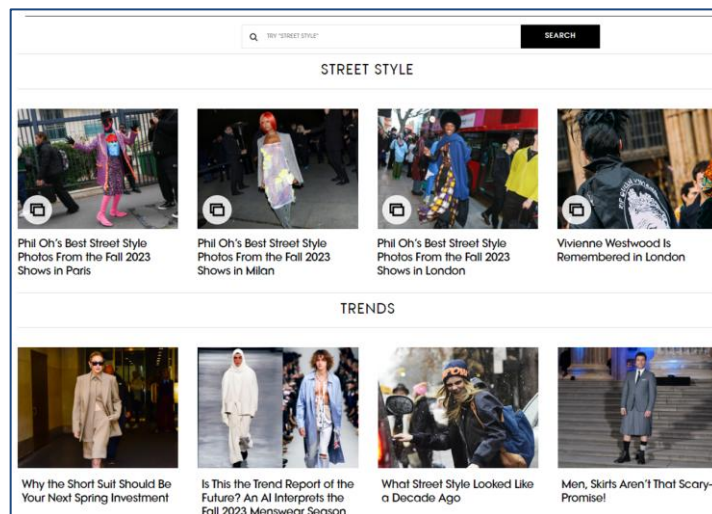
Yêu cầu:

Áp dụng các bài học trong Module này để Xây dựng Website Fashion tổng hợp theo mô tả:

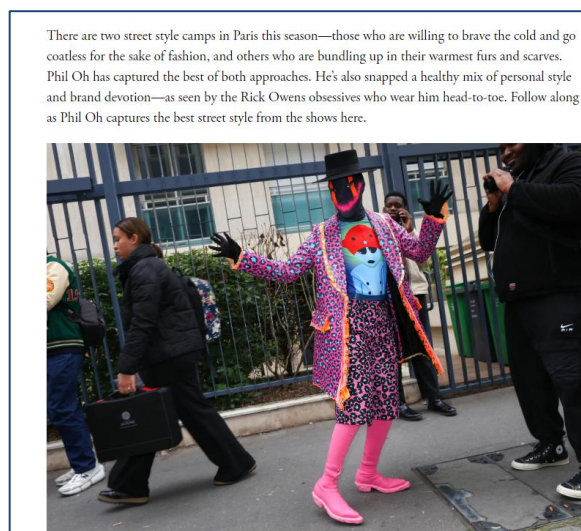
- Cơ sở dữ liệu MongoDB tên **FashionData**, Sinh viên cần suy luận và tự đề xuất ra cấu trúc dữ liệu phù hợp để thiết kế trong MongoDB:
 - o Có một collection duy nhất tên là **Fashion** gồm các thông tin: ObjectId (MongoDB tự động tạo ra), tiêu đề fashion, chi tiết fashion, hình thumbnail, Style của Fashion, ngày tạo.
 - o Nhập dữ liệu mẫu khoảng 3 Style, mỗi Style có khoảng 3 tới 5 fashion.
- Module Restful API (NodeJS, ExpressJS), bao gồm các API (tạo một project tên “**server-fashion**”, port 4000):
 - o API trả về toàn bộ Fashion, sắp xếp theo ngày tạo giảm dần
 - o API lọc danh sách các Fashion theo 1 Style nào đó
 - o API trả về một Fashion dựa vào ObjectId (gọi chung là id)
 - o API thêm mới một Fashion
 - o API chỉnh sửa một Fashion
 - o API xóa một Fashion dựa vào id
- Module Admin (AngularJs Front End) để quản trị hệ thống, bao gồm các chức năng (tạo một project tên “**admin-fashion**”, port 4001):
 - o Xem toàn bộ danh sách Fashion, mỗi dòng có nút xem chi tiết, nút sửa, và nút xóa (phải xác nhận trước khi xóa).
 - o Ngoài ra trong trang xem danh sách Fashion có nút Thêm mới một Fashion.
 - o Khi nhấn nút sửa hay nút thêm mới đều hiển thị một màn hình chi tiết để nhập thông tin cho Fashion, sau khi lưu thành công thì quay trở lại màn hình xem danh sách Fashion, yêu cầu phải sử dụng **WYSIWYG** (What You See Is What You Get) Html Editor cho field **chi tiết fashion**, Sinh viên tìm các công cụ tương tự:



- Module Client (AngularJs Front End) để trình diễn, bao gồm các chức năng(tạo một project tên “**client-fashion**” , port 4002):
 - o Hiển thị Fashion theo nhóm Style như hình dưới đây, hình đại diện là thuộc tính thumbnail.



- o Khi nhấn vào từng Fashion thì hiển thị chi tiết, hình và nội dung nếu có nó thuộc field chi tiết được nhập bằng **WYSIWYG**:



- o Tìm kiếm Fashion theo Style, người dùng nhập Style hoặc chọn từ dropdownlist thì lọc ra các Fashion thuộc Style đã chọn.

Module 7 – Cookie và Session

Nội dung kiến thức thực hành:

- + Hiểu được ý nghĩa của Cookie và Session
- + Lập trình Cookie để lưu trữ dữ liệu phía client
- + Lập trình Session để lưu trữ dữ liệu phía Server
- + Tạo giỏ hàng với Session

Bài 59 – Trình bày ý nghĩa của Cookie và Session

Yêu cầu:

Hãy trình bày ý nghĩa của Cookie và Session trong các hệ thống Website thương mại điện tử. So sánh sự khác biệt về cơ chế hoạt động của 2 đối tượng này.

Liệt kê một số thư viện hỗ trợ xử lý Cookie và Session.

Hướng dẫn:

- Sử dụng thư viện **cookie-parser** để mình xử lý Cookie
- Sử dụng thư viện **express-session** để xử lý Session

Bài 60 – Lập trình Cookie

Yêu cầu:

Sử dụng cookie-parser minh họa các chức năng:

- Khởi tạo đối tượng Cookie
- Thêm mới Cookie
- Truy xuất Cookie
- Xóa Cookie

Hướng dẫn:

Bước 1:

- Mở project “**my-server-mongodb**”
- Sau đó cài đặt **cookie-parser** bằng lệnh:

```
npm install --save cookie-parser
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

● PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server-mongodb> npm install --save cookie-parser
added 2 packages, removed 1 package, and audited 120 packages in 865ms

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server-mongodb> █
```

Bước 2:

Hiệu chỉnh “index.js” để cài đặt khởi tạo đối tượng Cookie:

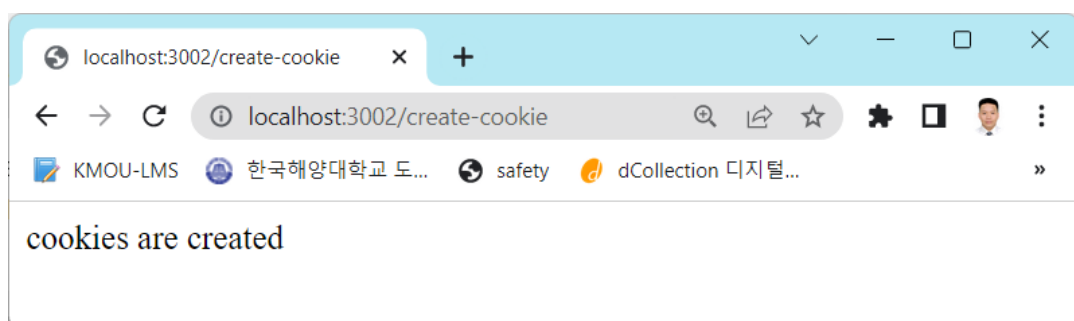
```
var cookieParser = require('cookie-parser');
app.use(cookieParser());
```

Bước 3:

Minh họa 1 API để tạo Cookie, dữ liệu đơn và dữ liệu JsonObject:

```
app.get("/create-cookie", cors(), (req, res) => {
  res.cookie("username", "tranduythanh")
  res.cookie("password", "123456")
  account = { "username": "tranduythanh",
    "password": "123456" }
  res.cookie("account", account)
  res.send("cookies are created")
})
```

Khởi tạo server và kiểm thử api create-cookie, ta có kết quả:

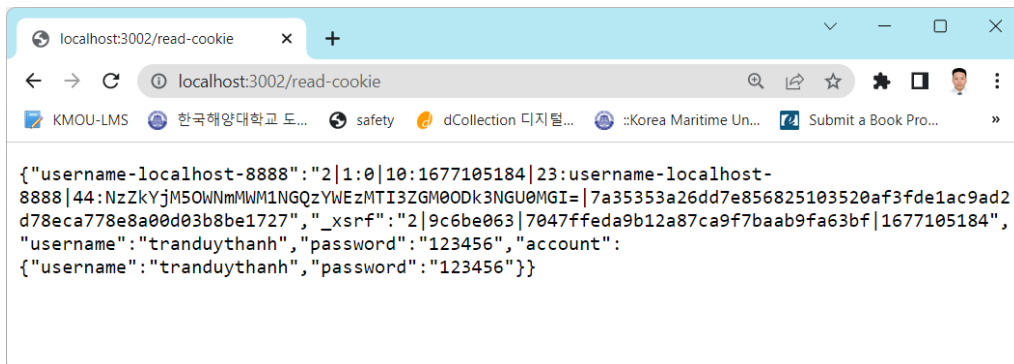


Bước 4:

Minh họa 1 API để đọc Cookie:

```
app.get("/read-cookie", cors(), (req, res) => {
  //cookie is stored in client, so we use req
  res.send(req.cookies)
})
```

Chạy chương trình và gọi api read-cookie ta có kết quả:



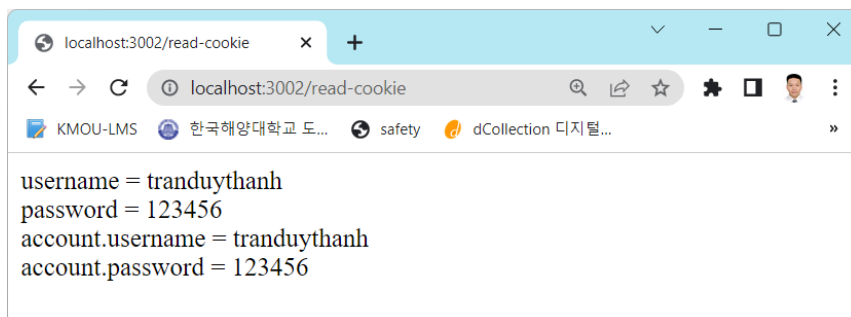
Quan sát kết quả, rõ ràng nó là một JsonObject, ta có thể dễ dàng truy cập tới từng Cookie mà ta đã lưu trước đó.

Bước 5:

Hiệu chỉnh api read-cookie để truy suất và bóc tách ra từng biến cookie mà ta đã lưu (chú ý dùng req):

```
app.get("/read-cookie", cors(), (req, res) => {
  //cookie is stored in client, so we use req
  username = req.cookies.username
  password = req.cookies.password
  account = req.cookies.account
  infor = "username = " + username + "<br/>"
  infor += "password = " + password + "<br/>"
  infor += "account.username = " + account.username + "<br/>"
  infor += "account.password = " + account.password + "<br/>"
  res.send(infor)
})
```

Chạy chương trình và gọi api read-cookie ta có kết quả:



Ngoài ra để tạo cookie với timeout (cookie có thời hạn) thì ta có thể code:

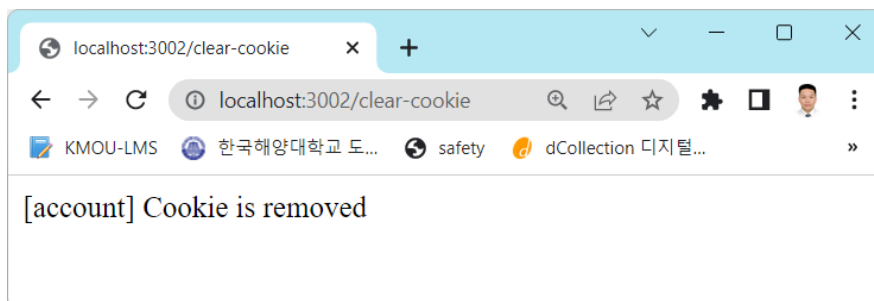
```
//Expires after 360000 ms from the time it is set.
res.cookie("infor_limit1", 'I am limited Cookie - way 1', {expire: 360000 + Date.now()});
res.cookie("infor_limit2", 'I am limited Cookie - way 2', {maxAge: 360000});
```

Bước 6:

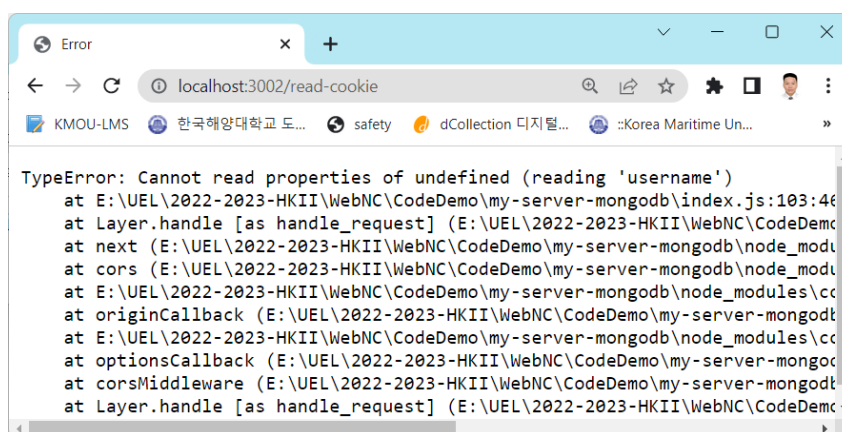
Thêm api clear-cookie để xóa Cookie đã lưu (chú ý dùng **res**):

```
app.get("/clear-cookie", cors(), (req, res) => {
  res.clearCookie("account")
  res.send("[account] Cookie is removed")
})
```

Chạy api ta có kết quả:



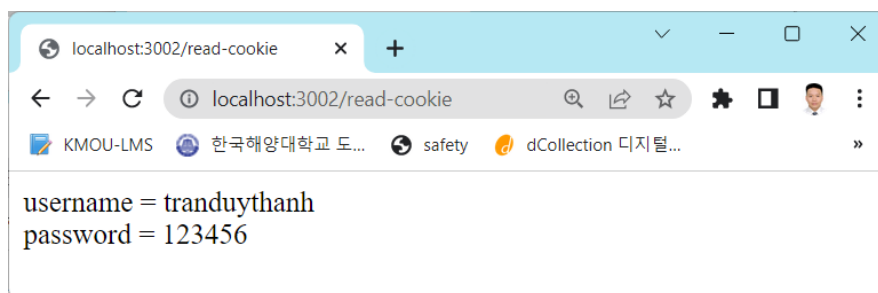
Lưu ý sau khi xóa Cookie mà ta cố tính truy suất thì nó bị null, và báo lỗi:



Do đó ta cần phải kiểm tra Cookie đọc ra, nếu nó null thì không xử lý, hiệu chỉnh lại:

```
app.get("/read-cookie", cors(), (req, res) => {
  //cookie is stored in client, so we use req
  username=req.cookies.username
  password=req.cookies.password
  account=req.cookies.account
  infor="username = "+username+"<br/>"
  infor+="password = "+password+"<br/>"
  if(account!=null)
  {
    infor+="account.username = "+account.username+"<br/>"
    infor+="account.password = "+account.password+"<br/>"
  }
  res.send(infor)
})
```

Kết quả sau khi hiệu chỉnh kiểm tra null:



Lúc này không còn bị báo lỗi nữa, chương trình chỉ hiển thị cookie của username và password, còn cookie account thì bị xóa rồi, nó bị null nên không hiển thị.

Lưu ý Cookie là lưu phía client (lưu phía trình duyệt), chạy trình duyệt nào thì cookie lưu theo trình duyệt đó và không hiểu các trình duyệt khác. Ví dụ nếu chạy Chrome và lưu cookie bên Chrome thì bên Firefox hay Microsoft Edge không có truy suất được. Và nhớ rằng cho dù lập trình Cookie có đúng nhưng nếu người sử dụng cấm mọi trình duyệt lưu cookie thì các mã lệnh của ta cũng vô nghĩa.

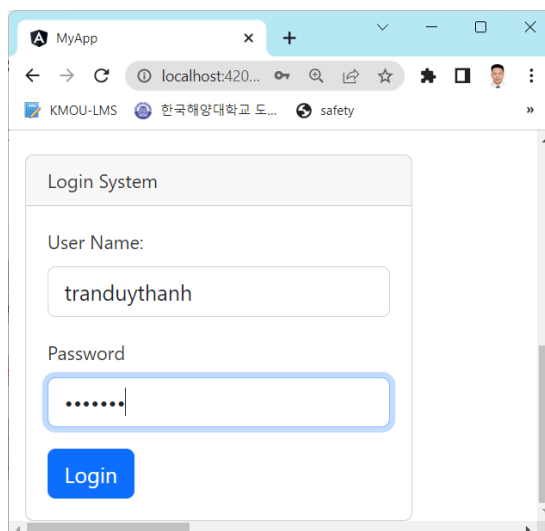
Bài 61 – Lập trình Cookie – Lưu thông tin đăng nhập

Yêu cầu:

Trong cơ sở dữ liệu “FashionData”, bổ sung thêm 1 collection tên là “User”, nó có 2 field là “username” và “password”. Hãy viết các Restful API để đăng nhập, nếu đăng nhập thành công thì lưu Cookie thông tin đăng nhập, các lần sau mở trình duyệt lên thì đọc cookie và hiển thị các thông tin này lên các ô nhập user name và password.

Hướng dẫn:

Tạo LoginComponent, thiết kế giao diện như hình dưới đây:



Dưới đây là mã lệnh của “login.component.html”:

```
<div class="container mt-4">
  <div class="card" style="width: 18rem;">
    <div class="card-header">Login System</div>
    <div class="card-body">
      <form action="/auth/register" method="POST">
        <div class="mb-3">
          <label for="name-reg" class="form-label">User Name:</label>
          <input type="text" class="form-control" id="name-reg"
name="name">
        </div>
        <div class="mb-3">
          <label for="password-reg" class="form-label">Password</label>
          <input type="password" class="form-control" id="password-reg"
name="password">
        </div>
        <button type="submit" class="btn btn-primary">Login</button>
      </form>
    </div>
  </div>
</div>
```

Yêu cầu Sinh viên áp dụng các bài học trước để tạo Collection “User”, nhập một vài Document mẫu, viết API Restful đăng nhập trong file **index.js** của dự án “my-server-mongodb”. API đăng nhập nên dùng phương thức POST, Khi đăng nhập thành công thì lưu Cookie, lần mở màn hình Login tiếp theo sẽ đọc cookie và hiển thị lên giao diện.

Bài 62 – Lập trình Session

Yêu cầu:

Sử dụng **express-session** để minh họa cách sử dụng Session cơ bản.

Hướng dẫn:

Bước 1:

- Mở project “my-server-mongodb”
- Sau đó cài đặt **express-session** bằng lệnh:

```
npm install --save express-session
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
● PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server-mongodb> npm install --save express-session
added 4 packages, and audited 124 packages in 3s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
○ PS E:\UEL\2022-2023-HKII\WebNC\CodeDemo\my-server-mongodb>

```

Bước 2:

Hiệu chỉnh “index.js” để cài đặt khởi tạo đối tượng Session:

```

var session = require('express-session');
app.use(session({secret: "Shh, its a secret!"}));

```

Bước 3:

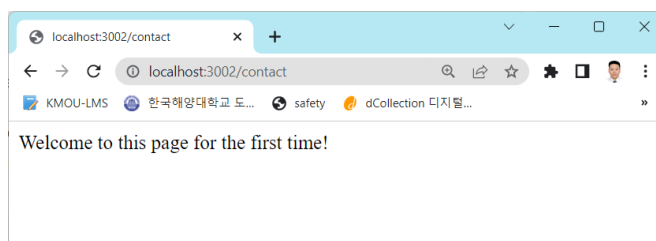
Minh họa 1 API để tạo Session visited

```

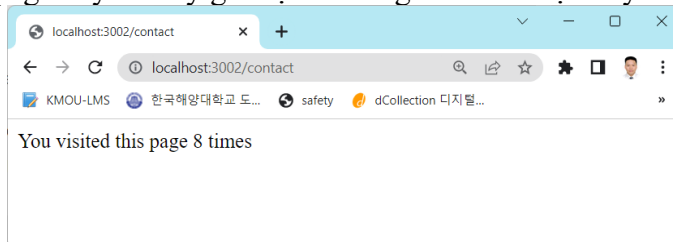
app.get("/contact", cors(), (req, res) => {
  if (req.session.visited !== null) {
    req.session.visited++
    res.send("You visited this page " + req.session.visited + " times")
  }
  else {
    req.session.visited = 1
    res.send("Welcome to this page for the first time!")
  }
})

```

Khởi tạo server và kiểm thử api **contact**, ta có kết quả:



Nhấn F5 nhiều lần ở page này ta thấy giá trị lưu trong session được thay đổi:



Lưu ý tương tự Cookie, ta có thể lưu cấu trúc dữ liệu bất kỳ cho Session.

Bài 63 – Lập trình Session – Bài tập tổng hợp Shopping Cart(*)

Yêu cầu:

Sử dụng Session minh họa xử lý giỏ hàng. Trong các trang thương mại điện tử, khi người sử dụng chọn các mặt hàng và đưa vào giỏ hàng (chưa thực sự mua) thì ta dùng Session để xử lý lưu trữ tạm thời. Hoặc chúng ta vẫn lưu vào cơ sở dữ liệu ở bảng tạm để xử lý trường hợp khách hàng đặt hàng dở dang: Đang đặt thì tạm dừng, hoặc đặt xong tới khâu thanh toán thì tạm hoãn... mọi thao tác ta cần lưu trữ lại để chăm sóc khách hàng (ví dụ khách hàng X đặt 5 món hàng nhưng tình trạng đơn hàng vẫn chưa hoàn tất, sau một thời gian kiểm tra hệ thống thì chủ cửa hàng nên chủ động liên hệ để push đơn hàng).

Trong bài này, yêu cầu Sinh viên chỉ sử dụng Session để lưu dữ liệu tạm thời các thao tác của khách hàng trên Server (bộ nhớ Server). Chỉ khi nào khách hàng xác nhận thanh toán thì mới lưu vào cơ sở dữ liệu các lựa chọn có trong đơn hàng. Trường hợp này thì khi tắt hệ thống Website thì các dữ liệu của khách hàng đang lưu trữ trong session thường sẽ bị mất hoặc sau thời gian timeout nó cũng bị mất. Yêu cầu Sinh viên thực hiện và minh họa các mô tả sau:

- Trong MongoDB tạo một collection Product, các Field tùy thuộc vào nhu cầu thiết kế của sinh viên để tạo cho phù hợp theo ý mình
- Viết các API để xử lý hiển thị Product, cho các khách lựa chọn Product và lưu vào Session (Session này tạo 1 biến đối tượng là **cart** để lưu JsonArray các Product mà khách hàng lựa chọn)
- Thiết kế giao diện cho khách hàng lựa chọn sản phẩm để đưa vào **cart**.
- Thiết kế giao diện để hiển thị chi tiết Product trong **cart**

Hướng dẫn:

Ví dụ màn hình hiển thị Product để khách hàng chọn mua:



Diamond Promise Ring 1/6 ct tw
Round-cut 10K White Gold

☆☆☆☆☆

\$399.99

ADD TO CART



Diamond Promise Ring 1/4 ct tw
Round/Baguette 10K White Gold

☆☆☆☆☆

\$529.00

ADD TO CART



Diamond Promise Ring 1/6 ct tw
Black/White Sterling Silver

☆☆☆☆☆

\$159.00

ADD TO CART



Diamond Promise Ring 1/5 ct tw
Round-cut Sterling Silver

☆☆☆☆☆

\$289.00

ADD TO CART



Diamond Promise Ring 1/5 ct tw
Round-cut Sterling Silver

☆☆☆☆☆

\$289.00

ADD TO CART



Diamond Promise Ring 1/8 ct tw
Round-cut Sterling Silver Ring

☆☆☆☆☆

\$229.00

ADD TO CART



Mỗi lần bấm “ADD TO CART” thì đưa Product vào giỏ hàng (biến **cart**)

Khi vào chức năng xem giỏ hàng, hiển thị giao diện như bên dưới:

Shopping cart

Remove	SKU	Image	Product(s)	Price	Qty.	Total
☐			Diamond Promise Ring 1/5 ct tw Round-cut Sterling Silver	\$289.00	1	\$289.00
☐			Diamond Promise Ring 1/5 ct tw Round-cut Sterling Silver	\$289.00	1	\$289.00
☐			Diamond Promise Ring 1/8 ct tw Round-cut Sterling Silver Ring	\$229.00	1	\$229.00
				Update shopping cart Continue shopping		

-Cho phép checked Product để xóa khỏi giỏ hàng, cập nhật số lượng → bấm “Update shopping cart” để cập nhật giỏ hàng.

-Muốn mua tiếp (quay trở lại màn hình Product) → bấm “Continue shopping”